LibMuse iOS 8.0.9

Generated by Doxygen 1.14.0

1 Muse SDK Documentation 1
1.0.1 Quick start . . . 1
1.0.1.1 iOS . . . 1
1.0.2 Threading . . . 1
1.0.3 Exceptions . . . 1

2 Deprecated List 3

3 Hierarchical Index 5
3.1 Class Hierarchy . . . 5

4 Class Index 7
4.1 Class List . . . 7

5 File Index 9
5.1 File List . . . 9

6 Class Documentation 11
6.1 <IXNAction> Protocol Reference . . . 11
6.1.1 Detailed Description . . . 11
6.1.2 Method Documentation . . . 11
6.1.2.1 run . . . 11
6.2 IXNAdvertisingStats Class Reference . . . 12
6.2.1 Detailed Description . . . 12
6.2.2 Method Documentation . . . 12
6.2.2.1 advertisingStatsWithNumAdvertisingPackets:avgAdvertisingInterval:sigmaAdvertisingInterval:ma
6.2.2.2 initWithNumAdvertisingPackets:avgAdvertisingInterval:sigmaAdvertisingInterval:maxAdvertising
6.2.3 Property Documentation . . . 13
6.2.3.1 avgAdvertisingInterval . . . 13
6.2.3.2 hasBadMac . . . 13
6.2.3.3 isLost . . . 13
6.2.3.4 maxAdvertisingInterval . . . 14
6.2.3.5 numAdvertisingPackets . . . 14
6.2.3.6 sigmaAdvertisingInterval . . . 14
6.3 IXNAnnotationData Class Reference . . . 14
6.3.1 Detailed Description . . . 15
6.3.2 Method Documentation . . . 15
6.3.2.1 annotationDataWithData:format:eventType:eventId:parentId: . . . 15
6.3.2.2 initWithData:format:eventType:eventId:parentId: . . . 15
6.3.3 Property Documentation . . . 15
6.3.3.1 data . . . 15
6.3.3.2 eventId . . . 16
6.3.3.3 eventType . . . 16
6.3.3.4 format . . . 16
6.3.3.5 parentId . . . 16

6.4 IXNApiVersion Class Reference . . . 16
6.4.1 Detailed Description . . . 17
6.4.2 Method Documentation . . . 17
6.4.2.1 getApi . . . 17
6.4.2.2 getMajor . . . 17
6.4.2.3 getMinor . . . 17
6.4.2.4 getMonotonic . . . 18
6.4.2.5 getPatch . . . 18
6.4.2.6 getString . . . 18
6.5 IXNComputingDeviceConfiguration Class Reference . . . 18
6.5.1 Detailed Description . . . 19
6.5.2 Method Documentation . . . 19
6.5.2.1 computingDeviceConfigurationWithOsType:osVersion:hardwareModelName:hardwareModelId:pro
6.5.2.2 initWithOsType:osVersion:hardwareModelName:hardwareModelId:processorName:processorSpeed
6.5.3 Property Documentation . . . 20
6.5.3.1 bluetoothVersion . . . 20
6.5.3.2 hardwareModelId . . . 20
6.5.3.3 hardwareModelName . . . 20
6.5.3.4 memorySize . . . 20
6.5.3.5 numberOfProcessors . . . 20
6.5.3.6 osType . . . 21
6.5.3.7 osVersion . . . 21
6.5.3.8 processorName . . . 21
6.5.3.9 processorSpeed . . . 21
6.5.3.10 timeZone . . . 21
6.5.3.11 timeZoneOffsetSeconds . . . 21
6.6 IXNComputingDeviceConfigurationFactory Class Reference . . . 21
6.6.1 Detailed Description . . . 22
6.6.2 Method Documentation . . . 22
6.6.2.1 getInstance . . . 22
6.6.3 Property Documentation . . . 22
6.6.3.1 computingDeviceConfiguration . . . 22
6.7 IXNDspData Class Reference . . . 22
6.7.1 Detailed Description . . . 23
6.7.2 Method Documentation . . . 23
6.7.2.1 dspDataWithType:floatArray:intArray:version: . . . 23
6.7.2.2 initWithType:floatArray:intArray:version: . . . 23
6.7.3 Property Documentation . . . 23
6.7.3.1 floatArray . . . 23
6.7.3.2 intArray . . . 24
6.7.3.3 type . . . 24
6.7.3.4 version . . . 24
6.8 IXNError Class Reference . . . 24

6.8.1 Detailed Description . . . 25
6.8.2 Method Documentation . . . 25
6.8.2.1 errorWithType:code:info: . . . 25
6.8.2.2 initWithType:code:info: . . . 25
6.8.3 Property Documentation . . . 25
6.8.3.1 code . . . 25
6.8.3.2 info . . . 25
6.8.3.3 type . . . 26
6.9 <IXNEventLoop> Protocol Reference . . . 26
6.9.1 Detailed Description . . . 26
6.9.2 Method Documentation . . . 26
6.9.2.1 cancel . . . 26
6.9.2.2 post: . . . 26
6.9.2.3 postDelayed:delayMilliseconds: . . . 27
6.10 IXNLibmuseVersion Class Reference . . . 27
6.10.1 Detailed Description . . . 27
6.10.2 Method Documentation . . . 28
6.10.2.1 instance . . . 28
6.11 <IXNLogListener> Protocol Reference . . . 28
6.11.1 Detailed Description . . . 28
6.11.2 Method Documentation . . . 28
6.11.2.1 receiveLog: . . . 28
6.12 IXNLogManager Class Reference . . . 29
6.12.1 Detailed Description . . . 29
6.12.2 Method Documentation . . . 30
6.12.2.1 getTimestamp . . . 30
6.12.2.2 instance . . . 30
6.12.2.3 makeDefaultLogListener . . . 31
6.12.2.4 setLogListener: . . . 31
6.12.2.5 setMinimumSeverity: . . . 32
6.12.2.6 timeSince: . . . 32
6.12.2.7 writeLog:raw:tag:message: . . . 32
6.13 IXNLogPacket Class Reference . . . 33
6.13.1 Detailed Description . . . 33
6.13.2 Method Documentation . . . 33
6.13.2.1 initWithSeverity:raw:tag:timestamp:message: . . . 33
6.13.2.2 logPacketWithSeverity:raw:tag:timestamp:message: . . . 33
6.13.3 Property Documentation . . . 34
6.13.3.1 message . . . 34
6.13.3.2 raw . . . 34
6.13.3.3 severity . . . 34
6.13.3.4 tag . . . 34
6.13.3.5 timestamp . . . 35

6.14 IXNMuse Class Reference . . . 35
6.14.1 Detailed Description . . . 36
6.14.2 Method Documentation . . . 36
6.14.2.1 connect . . . 36
6.14.2.2 disconnect . . . 36
6.14.2.3 enableDataTransmission: . . . 37
6.14.2.4 enableException: . . . 37
6.14.2.5 enableLedIndicator: . . . 37
6.14.2.6 execute . . . 38
6.14.2.7 getConnectionState . . . 38
6.14.2.8 getLastDiscoveredTime . . . 38
6.14.2.9 getMacAddress . . . 39
6.14.2.10 getModel . . . 39
6.14.2.11 getMuseConfiguration . . . 39
6.14.2.12 getMuseVersion . . . 40
6.14.2.13 getName . . . 40
6.14.2.14 getRssi . . . 40
6.14.2.15 isConnectable . . . 41
6.14.2.16 isLowEnergy . . . 41
6.14.2.17 isPaired . . . 41
6.14.2.18 registerConnectionListener: . . . 41
6.14.2.19 registerDataListener:type: . . . 42
6.14.2.20 registerErrorListener: . . . 42
6.14.2.21 runAsynchronously . . . 42
6.14.2.22 setLicenseData: . . . 43
6.14.2.23 setNotchFrequency: . . . 43
6.14.2.24 setNumConnectTries: . . . 43
6.14.2.25 setPreset: . . . 44
6.14.2.26 setProperty:value: . . . 44
6.14.2.27 unregisterAllListeners . . . 44
6.14.2.28 unregisterConnectionListener: . . . 44
6.14.2.29 unregisterDataListener:type: . . . 44
6.14.2.30 unregisterErrorListener: . . . 45
6.15 IXNMuseArtifactPacket Class Reference . . . 45
6.15.1 Detailed Description . . . 46
6.15.2 Method Documentation . . . 46
6.15.2.1 initWithHeadbandOn:blink:jawClench:timestamp: . . . 46
6.15.2.2 museArtifactPacketWithHeadbandOn:blink:jawClench:timestamp: . . . 46
6.15.3 Property Documentation . . . 46
6.15.3.1 blink . . . 46
6.15.3.2 headbandOn . . . 46
6.15.3.3 jawClench . . . 47
6.15.3.4 timestamp . . . 47

6.16 IXNMuseConfiguration Class Reference . . . 47
6.16.1 Detailed Description . . . 48
6.16.2 Method Documentation . . . 48
6.16.2.1 getAccelerometerSampleFrequency . . . 48
6.16.2.2 getAdcFrequency . . . 48
6.16.2.3 getAfeGain . . . 48
6.16.2.4 getBatteryDataEnabled . . . 48
6.16.2.5 getBatteryPercentRemaining . . . 49
6.16.2.6 getBluetoothMac . . . 49
6.16.2.7 getDownsampleRate . . . 49
6.16.2.8 getDrlRefEnabled . . . 49
6.16.2.9 getDrlRefFrequency . . . 49
6.16.2.10 getEegChannelCount . . . 50
6.16.2.11 getHeadbandName . . . 50
6.16.2.12 getHeadsetSerialNumber . . . 50
6.16.2.13 getLicenseNonce . . . 50
6.16.2.14 getMicrocontrollerId . . . 50
6.16.2.15 getModel . . . 51
6.16.2.16 getNotchFilter . . . 51
6.16.2.17 getNotchFilterEnabled . . . 51
6.16.2.18 getOutputFrequency . . . 51
6.16.2.19 getPreset . . . 51
6.16.2.20 getSerialNumber . . . 52
6.16.2.21 getSeroutMode . . . 52
6.16.2.22 getSwitch . . . 52
6.17 <IXNMuseConnectionListener> Protocol Reference . . . 52
6.17.1 Detailed Description . . . 52
6.17.2 Method Documentation . . . 53
6.17.2.1 receiveMuseConnectionPacket:muse: . . . 53
6.18 IXNMuseConnectionPacket Class Reference . . . 53
6.18.1 Detailed Description . . . 54
6.18.2 Method Documentation . . . 54
6.18.2.1 initWithPreviousConnectionState:currentConnectionState: . . . 54
6.18.2.2 museConnectionPacketWithPreviousConnectionState:currentConnectionState: 54
6.18.3 Property Documentation . . . 54
6.18.3.1 currentConnectionState . . . 54
6.18.3.2 previousConnectionState . . . 54
6.19 <IXNMuseDataListener> Protocol Reference . . . 55
6.19.1 Detailed Description . . . 55
6.19.2 Method Documentation . . . 55
6.19.2.1 receiveMuseArtifactPacket:muse: . . . 55
6.19.2.2 receiveMuseDataPacket:muse: . . . 56
6.20 IXNMuseDataPacket Class Reference . . . 56

6.20.1 Detailed Description . . . 57
6.20.2 Method Documentation . . . 57
6.20.2.1 getAccelerometerValue: . . . 57
6.20.2.2 getBatteryValue: . . . 58
6.20.2.3 getDrlRefValue: . . . 58
6.20.2.4 getEegChannelValue: . . . 59
6.20.2.5 getGyroValue: . . . 59
6.20.2.6 getMagnetometerValue: . . . 60
6.20.2.7 getOpticsChannelValue: . . . 60
6.20.2.8 getPpgChannelValue: . . . 61
6.20.2.9 getPpgMicroamps:ppgValue: . . . 61
6.20.2.10 getPressureValue: . . . 62
6.20.2.11 getTemperatureValue . . . 62
6.20.2.12 getUvValue: . . . 62
6.20.2.13 makePacket:timestamp:values: . . . 63
6.20.2.14 makeUninitializedPacket: . . . 63
6.20.2.15 packetType . . . 63
6.20.2.16 timestamp . . . 64
6.20.2.17 values . . . 64
6.20.2.18 valuesSize . . . 64
6.21 <IXNMuseErrorListener> Protocol Reference . . . 64
6.21.1 Detailed Description . . . 65
6.21.2 Method Documentation . . . 65
6.21.2.1 receiveError:muse: . . . 65
6.22 <IXNMuseFile> Protocol Reference . . . 65
6.22.1 Detailed Description . . . 66
6.22.2 Method Documentation . . . 66
6.22.2.1 close: . . . 66
6.22.2.2 open: . . . 66
6.22.2.3 read: . . . 66
6.22.2.4 write: . . . 67
6.23 IXNMuseFileFactory Class Reference . . . 67
6.23.1 Detailed Description . . . 68
6.23.2 Method Documentation . . . 68
6.23.2.1 museFileReaderWithPathString: . . . 68
6.23.2.2 museFileWithPathString: . . . 68
6.23.2.3 museFileWriterWithPathString: . . . 68
6.24 IXNMuseFileReader Class Reference . . . 69
6.24.1 Detailed Description . . . 70
6.24.2 Method Documentation . . . 70
6.24.2.1 close . . . 70
6.24.2.2 getAnnotation . . . 70
6.24.2.3 getArtifactPacket . . . 70

6.24.2.4 getComputingDeviceConfiguration . . . 71
6.24.2.5 getConfiguration . . . 71
6.24.2.6 getDataPacket . . . 71
6.24.2.7 getDsp . . . 72
6.24.2.8 getFileReader: . . . 72
6.24.2.9 getMessageId . . . 72
6.24.2.10 getMessageTimestamp . . . 73
6.24.2.11 getMessageType . . . 73
6.24.2.12 getVersion . . . 73
6.24.2.13 gotoNextMessage . . . 73
6.24.2.14 open . . . 74
6.25 IXNMuseFileWriter Class Reference . . . 74
6.25.1 Detailed Description . . . 75
6.25.2 Method Documentation . . . 75
6.25.2.1 addAnnotation:annotation: . . . 75
6.25.2.2 addAnnotationString:annotation: . . . 75
6.25.2.3 addArtifactPacket:packet: . . . 76
6.25.2.4 addComputingDeviceConfiguration:configuration: . . . 76
6.25.2.5 addConfiguration:configuration: . . . 76
6.25.2.6 addDataPacket:packet: . . . 77
6.25.2.7 addDsp:dsp: . . . 78
6.25.2.8 addVersion:version: . . . 78
6.25.2.9 close . . . 78
6.25.2.10 discardBufferedPackets . . . 79
6.25.2.11 flush . . . 79
6.25.2.12 getBufferedMessagesSize . . . 79
6.25.2.13 getBufferredMessagesCount . . . 79
6.25.2.14 getFileWriter: . . . 79
6.25.2.15 getTotalBytesWritten . . . 80
6.25.2.16 isOpen . . . 80
6.25.2.17 open . . . 80
6.25.2.18 setTimestamp: . . . 80
6.25.2.19 setTimestampMode: . . . 81
6.26 <IXNMuseListener> Protocol Reference . . . 81
6.26.1 Detailed Description . . . 81
6.26.2 Method Documentation . . . 81
6.26.2.1 museListChanged . . . 81
6.27 <IXNMuseManager> Protocol Reference . . . 82
6.27.1 Detailed Description . . . 82
6.27.2 Method Documentation . . . 82
6.27.2.1 getAdvertisingStats: . . . 82
6.27.2.2 getMuses . . . 83
6.27.2.3 removeFromListAfter: . . . 83

6.27.2.4 resetAdvertisingStats . . . 83
6.27.2.5 setMuseListener: . . . 83
6.27.2.6 startListening . . . 84
6.27.2.7 stopListening . . . 84
6.28 IXNMuseManagerIos Class Reference . . . 84
6.28.1 Detailed Description . . . 85
6.28.2 Method Documentation . . . 85
6.28.2.1 sharedManager . . . 85
6.28.2.2 showMusePickerWithCompletion: . . . 85
6.28.3 Property Documentation . . . 86
6.28.3.1 museListener . . . 86
6.29 IXNMuseVersion Class Reference . . . 86
6.29.1 Detailed Description . . . 86
6.29.2 Method Documentation . . . 87
6.29.2.1 getBleFirmwareVersion . . . 87
6.29.2.2 getBootloaderVersion . . . 87
6.29.2.3 getBspVersion . . . 87
6.29.2.4 getFirmwareBuildNumber . . . 87
6.29.2.5 getFirmwareType . . . 87
6.29.2.6 getFirmwareVersion . . . 88
6.29.2.7 getHardwareVersion . . . 88
6.29.2.8 getProtocolVersion . . . 88
6.29.2.9 getRunningState . . . 88
6.29.2.10 makeDefaultVersion . . . 88
6.29.2.11 makeVersion: . . . 89
6.30 <IXNReaderListener> Protocol Reference . . . 89
6.30.1 Detailed Description . . . 89
6.30.2 Method Documentation . . . 89
6.30.2.1 receiveAnnotation: . . . 89
6.30.2.2 receiveComputingDeviceConfiguration: . . . 89
6.30.2.3 receiveConfiguration: . . . 90
6.30.2.4 receiveVersion: . . . 90
6.31 IXNReaderMuse Class Reference . . . 90
6.31.1 Detailed Description . . . 91
6.31.2 Method Documentation . . . 91
6.31.2.1 asMuse . . . 91
6.31.2.2 currentTime . . . 92
6.31.2.3 getPlaybackSettings . . . 92
6.31.2.4 playback . . . 92
6.31.2.5 run . . . 92
6.31.2.6 runInRealTimespan . . . 93
6.31.2.7 setPlaybackListener: . . . 93
6.31.2.8 setPlaybackSettings: . . . 93

6.31.2.9 setReaderListener: . . . 93
6.31.2.10 stopPlayback . . . 94
6.32 IXNReaderMuseBuilder Class Reference . . . 94
6.32.1 Detailed Description . . . 94
6.32.2 Method Documentation . . . 94
6.32.2.1 build: . . . 94
6.32.2.2 buildWithAsync:asyncLoop: . . . 95
6.32.2.3 get . . . 95
6.32.2.4 skipPacketTypes: . . . 95
6.32.2.5 withEventLoop: . . . 96
6.32.2.6 withModel: . . . 96
6.32.2.7 withPacketTypes: . . . 96
6.32.2.8 withPlaybackSettings: . . . 97
6.33 <IXNReaderPlaybackListener> Protocol Reference . . . 97
6.33.1 Detailed Description . . . 97
6.33.2 Method Documentation . . . 98
6.33.2.1 receivePlaybackDone . . . 98
6.33.2.2 receivePlaybackInterrupted . . . 98
6.34 IXNResult Class Reference . . . 98
6.34.1 Detailed Description . . . 99
6.34.2 Method Documentation . . . 99
6.34.2.1 initWithLevel:type:code:info: . . . 99
6.34.2.2 resultWithLevel:type:code:info: . . . 99
6.34.3 Property Documentation . . . 99
6.34.3.1 code . . . 99
6.34.3.2 info . . . 99
6.34.3.3 level . . . 100
6.34.3.4 type . . . 100
6.35 IXNRunEventLoop Class Reference . . . 100
6.35.1 Detailed Description . . . 100
6.35.2 Method Documentation . . . 100
6.35.2.1 initWithRunLoop: . . . 100
6.36 IXNStringify Class Reference . . . 101
6.36.1 Detailed Description . . . 101
6.36.2 Method Documentation . . . 101
6.36.2.1 connectionState: . . . 101
6.36.2.2 instance . . . 102
6.36.2.3 packetType: . . . 102

7 File Documentation 103
7.1 mainpage.dox File Reference . . . 103
7.2 IXNAccelerometer.h File Reference . . . 103
7.2.1 Detailed Description . . . 103

7.2.2 Enumeration Type Documentation . . . 105
7.2.2.1 IXNAccelerometer . . . 105
7.3 IXNAccelerometer.h . . . 105
7.4 IXNAction.h File Reference . . . 105
7.5 IXNAction.h . . . 106
7.6 IXNAdvertisingStats.h File Reference . . . 106
7.7 IXNAdvertisingStats.h . . . 106
7.8 IXNAnnotationData.h File Reference . . . 106
7.9 IXNAnnotationData.h . . . 107
7.10 IXNAnnotationFormat.h File Reference . . . 107
7.10.1 Detailed Description . . . 107
7.10.2 Enumeration Type Documentation . . . 107
7.10.2.1 IXNAnnotationFormat . . . 107
7.11 IXNAnnotationFormat.h . . . 108
7.12 IXNApiVersion.h File Reference . . . 108
7.13 IXNApiVersion.h . . . 108
7.14 IXNBattery.h File Reference . . . 108
7.14.1 Detailed Description . . . 109
7.14.2 Enumeration Type Documentation . . . 109
7.14.2.1 IXNBattery . . . 109
7.15 IXNBattery.h . . . 109
7.16 IXNComputingDeviceConfiguration.h File Reference . . . 110
7.17 IXNComputingDeviceConfiguration.h . . . 110
7.18 IXNConnectionState.h File Reference . . . 110
7.18.1 Detailed Description . . . 111
7.18.2 Enumeration Type Documentation . . . 111
7.18.2.1 IXNConnectionState . . . 111
7.19 IXNConnectionState.h . . . 111
7.20 IXNDrlRef.h File Reference . . . 111
7.20.1 Detailed Description . . . 112
7.20.2 Enumeration Type Documentation . . . 112
7.20.2.1 IXNDrlRef . . . 112
7.21 IXNDrlRef.h . . . 112
7.22 IXNDspData.h File Reference . . . 112
7.23 IXNDspData.h . . . 113
7.24 IXNEeg.h File Reference . . . 113
7.24.1 Detailed Description . . . 113
7.24.2 Enumeration Type Documentation . . . 113
7.24.2.1 IXNEeg . . . 113
7.25 IXNEeg.h . . . 114
7.26 IXNError.h File Reference . . . 114
7.27 IXNError.h . . . 114
7.28 IXNErrorType.h File Reference . . . 115

7.28.1 Detailed Description . . . 115
7.28.2 Enumeration Type Documentation . . . 115
7.28.2.1 IXNErrorType . . . 115
7.29 IXNErrorType.h . . . 116
7.30 IXNEventLoop.h File Reference . . . 116
7.31 IXNEventLoop.h . . . 116
7.32 IXNGyro.h File Reference . . . 116
7.32.1 Detailed Description . . . 117
7.32.2 Enumeration Type Documentation . . . 118
7.32.2.1 IXNGyro . . . 118
7.33 IXNGyro.h . . . 119
7.34 IXNLibmuseVersion.h File Reference . . . 119
7.35 IXNLibmuseVersion.h . . . 119
7.36 IXNLogListener.h File Reference . . . 119
7.37 IXNLogListener.h . . . 120
7.38 IXNLogManager.h File Reference . . . 120
7.39 IXNLogManager.h . . . 120
7.40 IXNLogPacket.h File Reference . . . 120
7.41 IXNLogPacket.h . . . 121
7.42 IXNMagnetometer.h File Reference . . . 121
7.42.1 Detailed Description . . . 121
7.42.2 Enumeration Type Documentation . . . 122
7.42.2.1 IXNMagnetometer . . . 122
7.43 IXNMagnetometer.h . . . 122
7.44 IXNMessageType.h File Reference . . . 123
7.44.1 Detailed Description . . . 123
7.44.2 Enumeration Type Documentation . . . 123
7.44.2.1 IXNMessageType . . . 123
7.45 IXNMessageType.h . . . 124
7.46 IXNMuse.h File Reference . . . 125
7.47 IXNMuse.h . . . 125
7.48 IXNMuseArtifactPacket.h File Reference . . . 126
7.49 IXNMuseArtifactPacket.h . . . 126
7.50 IXNMuseConfiguration.h File Reference . . . 126
7.51 IXNMuseConfiguration.h . . . 127
7.52 IXNMuseConnectionListener.h File Reference . . . 127
7.53 IXNMuseConnectionListener.h . . . 127
7.54 IXNMuseConnectionPacket.h File Reference . . . 128
7.55 IXNMuseConnectionPacket.h . . . 128
7.56 IXNMuseDataListener.h File Reference . . . 128
7.57 IXNMuseDataListener.h . . . 128
7.58 IXNMuseDataPacket.h File Reference . . . 129
7.59 IXNMuseDataPacket.h . . . 129

7.60 IXNMuseDataPacketType.h File Reference . . . 130
7.60.1 Detailed Description . . . 130
7.60.2 Enumeration Type Documentation . . . 131
7.60.2.1 IXNMuseDataPacketType . . . 131
7.61 IXNMuseDataPacketType.h . . . 138
7.62 IXNMuseErrorListener.h File Reference . . . 139
7.63 IXNMuseErrorListener.h . . . 139
7.64 IXNMuseFile.h File Reference . . . 139
7.65 IXNMuseFile.h . . . 140
7.66 IXNMuseFileReader.h File Reference . . . 140
7.67 IXNMuseFileReader.h . . . 140
7.68 IXNMuseFileWriter.h File Reference . . . 141
7.69 IXNMuseFileWriter.h . . . 141
7.70 IXNMuseListener.h File Reference . . . 142
7.71 IXNMuseListener.h . . . 142
7.72 IXNMuseManager.h File Reference . . . 142
7.72.1 Variable Documentation . . . 142
7.72.1.1 IXNMuseManagerDEFAULTREMOVEFROMLISTAFTER . . . 142
7.73 IXNMuseManager.h . . . 143
7.74 IXNMuseModel.h File Reference . . . 143
7.74.1 Detailed Description . . . 143
7.74.2 Enumeration Type Documentation . . . 143
7.74.2.1 IXNMuseModel . . . 143
7.75 IXNMuseModel.h . . . 144
7.76 IXNMusePreset.h File Reference . . . 144
7.76.1 Detailed Description . . . 145
7.76.2 Enumeration Type Documentation . . . 145
7.76.2.1 IXNMusePreset . . . 145
7.77 IXNMusePreset.h . . . 149
7.78 IXNMuseVersion.h File Reference . . . 150
7.79 IXNMuseVersion.h . . . 150
7.80 IXNNotchFrequency.h File Reference . . . 151
7.80.1 Detailed Description . . . 151
7.80.2 Enumeration Type Documentation . . . 151
7.80.2.1 IXNNotchFrequency . . . 151
7.81 IXNNotchFrequency.h . . . 151
7.82 IXNOptics.h File Reference . . . 152
7.82.1 Detailed Description . . . 152
7.82.2 Enumeration Type Documentation . . . 152
7.82.2.1 IXNOptics . . . 152
7.83 IXNOptics.h . . . 153
7.84 IXNPpg.h File Reference . . . 153
7.84.1 Detailed Description . . . 154

7.84.2 Enumeration Type Documentation . . . 154
7.84.2.1 IXNPpg . . . 154
7.85 IXNPpg.h . . . 154
7.86 IXNPressure.h File Reference . . . 155
7.86.1 Detailed Description . . . 155
7.86.2 Enumeration Type Documentation . . . 155
7.86.2.1 IXNPressure . . . 155
7.87 IXNPressure.h . . . 155
7.88 IXNReaderListener.h File Reference . . . 155
7.89 IXNReaderListener.h . . . 156
7.90 IXNReaderMuse.h File Reference . . . 156
7.91 IXNReaderMuse.h . . . 156
7.92 IXNReaderMuseBuilder.h File Reference . . . 157
7.93 IXNReaderMuseBuilder.h . . . 157
7.94 IXNReaderMusePlaybackSettings.h File Reference . . . 157
7.94.1 Detailed Description . . . 157
7.94.2 Enumeration Type Documentation . . . 157
7.94.2.1 IXNReaderMusePlaybackSettings . . . 157
7.95 IXNReaderMusePlaybackSettings.h . . . 158
7.96 IXNReaderPlaybackListener.h File Reference . . . 159
7.97 IXNReaderPlaybackListener.h . . . 159
7.98 IXNResult.h File Reference . . . 159
7.99 IXNResult.h . . . 159
7.100 IXNResultLevel.h File Reference . . . 160
7.100.1 Detailed Description . . . 160
7.100.2 Enumeration Type Documentation . . . 160
7.100.2.1 IXNResultLevel . . . 160
7.101 IXNResultLevel.h . . . 160
7.102 IXNSeverity.h File Reference . . . 161
7.102.1 Enumeration Type Documentation . . . 161
7.102.1.1 IXNSeverity . . . 161
7.103 IXNSeverity.h . . . 161
7.104 IXNStringify.h File Reference . . . 162
7.105 IXNStringify.h . . . 162
7.106 IXNTimestampMode.h File Reference . . . 162
7.106.1 Detailed Description . . . 162
7.106.2 Enumeration Type Documentation . . . 162
7.106.2.1 IXNTimestampMode . . . 162
7.107 IXNTimestampMode.h . . . 163
7.108 IXNUltraViolet.h File Reference . . . 163
7.108.1 Detailed Description . . . 163
7.108.2 Enumeration Type Documentation . . . 163
7.108.2.1 IXNUltraViolet . . . 163

7.109 IXNUltraViolet.h . . . 164
7.110 IXNComputingDeviceConfigurationFactory.h File Reference . . . 164
7.111 IXNComputingDeviceConfigurationFactory.h . . . 164
7.112 IXNMuseFileFactory.h File Reference . . . 164
7.113 IXNMuseFileFactory.h . . . 164
7.114 IXNMuseManagerIos.h File Reference . . . 165
7.115 IXNMuseManagerIos.h . . . 165
7.116 IXNRunEventLoop.h File Reference . . . 165
7.117 IXNRunEventLoop.h . . . 165

Chapter 1

Muse SDK Documentation

This library enables communication with and control of Muse headbands from client software.

1.0.1 Quick start

1.0.1.1 iOS

Every Objective-C class is prefixed with IXN. Start with IXNMuseManager and explore from there, or use the Classes tab.

1.0.2 Threading

Most LibMuse methods are thread safe, except IXNMuse::execute and IXNMuse::runAsynchronously . Each method describes its threading concerns if there are any.

1.0.3 Exceptions

The LibMuse native library catches and handles every exception that it can reasonably be expected to do so. There are, however, some situations in which native code can still throw an exception. Any API method that can throw an exception documents this behaviour.

Chapter 2

Deprecated List

Member [IXNMuseDataPacket values]

Use IXNMuseDataPacket::getAccelerometerValue , IXNMuseDataPacket::getBatteryValue , IXNMuseDataPacket::getDrlRefValue , IXNMuseDataPacket::getEegChannelValue , IXNMuseDataPacket::getPpgCha , IXNMuseDataPacket::getGyroValue instead.

Member [IXNMuseManagerIos showMusePickerWithCompletion:]

This will only display Muse 2014 (MU_01) headsets. Instead set a IXNMuseListener with setMuseListener: (IXNMuseManager-p) to receive callbacks when either model of headband is detected. See the sample application for an example of how to update the UI and connect to the headset.

Member [IXNReaderMuse run]

Use playback (IXNReaderMuse): with the playback setting IXNReaderMusePlaybackSettingsAsFastAsPossibleWithSa to replicate this behaviour.

Member [IXNReaderMuse runInRealTimespan]

Use playback (IXNReaderMuse): with the playback setting IXNReaderMusePlaybackSettingsSimulatedWithSavedTim to replicate this behaviour.

Member [IXNReaderMuseBuilder buildWithAsync:asyncLoop:]

Set the IXNEventLoop with IXNReaderMuseBuilder::withEventLoop and then call IXNReaderMuseBuilder::build instead.

Member IXNMuseDataPacketTypeDroppedAccelerometer

This is never emitted in an actual Muse session; instead, NaN-filled packets of the basic type (EEG or ACCELEROMETER) are emitted to stand in for dropped packets. This can only appear when reading Muse files written with older versions of the library.

Member IXNMuseDataPacketTypeDroppedEeg

This is never emitted in an actual Muse session; instead, NaN-filled packets of the basic type (EEG or ACCELEROMETER) are emitted to stand in for dropped packets. This can only appear when reading Muse files written with older versions of the library.

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

<IXNAction> . . . 11
<IXNEventLoop> . . . 26
IXNRunEventLoop . . . 100
<IXNLogListener> . . . 28
<IXNMuseConnectionListener> . . . 52
<IXNMuseDataListener> . . . 55
<IXNMuseErrorListener> . . . 64
<IXNMuseFile> . . . 65
<IXNMuseListener> . . . 81
<IXNMuseManager> . . . 82
IXNMuseManagerIos . . . 84
<IXNReaderListener> . . . 89
<IXNReaderPlaybackListener> . . . 97
NSObject
IXNAdvertisingStats . . . 12
IXNAnnotationData . . . 14
IXNApiVersion . . . 16
IXNComputingDeviceConfiguration . . . 18
IXNComputingDeviceConfigurationFactory . . . 21
IXNDspData . . . 22
IXNError . . . 24
IXNLibmuseVersion . . . 27
IXNLogManager . . . 29
IXNLogPacket . . . 33
IXNMuse . . . 35
IXNMuseArtifactPacket . . . 45
IXNMuseConfiguration . . . 47
IXNMuseConnectionPacket . . . 53
IXNMuseDataPacket . . . 56
IXNMuseFileFactory . . . 67
IXNMuseFileReader . . . 69
IXNMuseFileWriter . . . 74
IXNMuseManagerIos . . . 84
IXNMuseVersion . . . 86
IXNReaderMuse . . . 90

IXNReaderMuseBuilder 94
IXNResult 98
IXNRunEventLoop 100
IXNStringify 101

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

<IXNAction> . . . 11
IXNAdvertisingStats . . . 12
IXNAnnotationData . . . 14
IXNApiVersion . . . 16
IXNComputingDeviceConfiguration . . . 18
IXNComputingDeviceConfigurationFactory . . . 21
IXNDspData . . . 22
IXNError . . . 24
<IXNEventLoop> . . . 26
IXNLibmuseVersion . . . 27
<IXNLogListener> . . . 28
IXNLogManager . . . 29
IXNLogPacket . . . 33
IXNMuse . . . 35
IXNMuseArtifactPacket . . . 45
IXNMuseConfiguration . . . 47
<IXNMuseConnectionListener> . . . 52
IXNMuseConnectionPacket . . . 53
<IXNMuseDataListener> . . . 55
IXNMuseDataPacket . . . 56
<IXNMuseErrorListener> . . . 64
<IXNMuseFile> . . . 65
IXNMuseFileFactory . . . 67
IXNMuseFileReader . . . 69
IXNMuseFileWriter . . . 74
<IXNMuseListener> . . . 81
<IXNMuseManager> . . . 82
IXNMuseManagerIos . . . 84
IXNMuseVersion . . . 86
<IXNReaderListener> . . . 89
IXNReaderMuse . . . 90
IXNReaderMuseBuilder . . . 94
<IXNReaderPlaybackListener> . . . 97
IXNResult . . . 98
IXNRunEventLoop . . . 100
IXNStringify . . . 101

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

IXNAccelerometer.h . . . 103
IXNAction.h . . . 105
IXNAdvertisingStats.h . . . 106
IXNAnnotationData.h . . . 106
IXNAnnotationFormat.h . . . 107
IXNApiVersion.h . . . 108
IXNBattery.h . . . 108
IXNComputingDeviceConfiguration.h . . . 110
IXNConnectionState.h . . . 110
IXNDrlRef.h . . . 111
IXNDspData.h . . . 112
IXNEeg.h . . . 113
IXNError.h . . . 114
IXNErrorType.h . . . 115
IXNEventLoop.h . . . 116
IXNGyro.h . . . 116
IXNLibmuseVersion.h . . . 119
IXNLogListener.h . . . 119
IXNLogManager.h . . . 120
IXNLogPacket.h . . . 120
IXNMagnetometer.h . . . 121
IXNMessageType.h . . . 123
IXNMuse.h . . . 125
IXNMuseArtifactPacket.h . . . 126
IXNMuseConfiguration.h . . . 126
IXNMuseConnectionListener.h . . . 127
IXNMuseConnectionPacket.h . . . 128
IXNMuseDataListener.h . . . 128
IXNMuseDataPacket.h . . . 129
IXNMuseDataPacketType.h . . . 130
IXNMuseErrorListener.h . . . 139
IXNMuseFile.h . . . 139
IXNMuseFileReader.h . . . 140
IXNMuseFileWriter.h . . . 141
IXNMuseListener.h . . . 142

IXNMuseManager.h . . . 142
IXNMuseModel.h . . . 143
IXNMusePreset.h . . . 144
IXNMuseVersion.h . . . 150
IXNNotchFrequency.h . . . 151
IXNOptics.h . . . 152
IXNPpg.h . . . 153
IXNPressure.h . . . 155
IXNReaderListener.h . . . 155
IXNReaderMuse.h . . . 156
IXNReaderMuseBuilder.h . . . 157
IXNReaderMusePlaybackSettings.h . . . 157
IXNReaderPlaybackListener.h . . . 159
IXNResult.h . . . 159
IXNResultLevel.h . . . 160
IXNSeverity.h . . . 161
IXNStringify.h . . . 162
IXNTimestampMode.h . . . 162
IXNUltraViolet.h . . . 163
IXNComputingDeviceConfigurationFactory.h . . . 164
IXNMuseFileFactory.h . . . 164
IXNMuseManagerIos.h . . . 165
IXNRunEventLoop.h . . . 165

Chapter 6

Class Documentation

6.1 <IXNAction> Protocol Reference

#include <IXNAction.h>

Instance Methods

- (void) - run

6.1.1 Detailed Description

A wrapper that represents a block or runnable action.

See also

IXNEventLoop

6.1.2 Method Documentation

6.1.2.1 run

- (void) run

Performs the action.

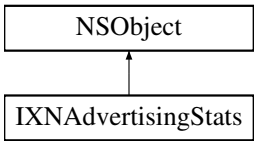
The documentation for this protocol was generated from the following file:

- IXNAction.h

6.2 IXNAdvertisingStats Class Reference

#include <IXNAdvertisingStats.h>

Inheritance diagram for IXNAdvertisingStats:



Instance Methods

- (nonnull instancetype) - initWithNumAdvertisingPackets:avgAdvertisingInterval:sigmaAdvertisingInterval:maxAdve

Class Methods

- (nonnull instancetype) + advertisingStatsWithNumAdvertisingPackets:avgAdvertisingInterval:sigmaAdvertisingInte

Properties

- int32_t numAdvertisingPackets
- double avgAdvertisingInterval
- double sigmaAdvertisingInterval
- double maxAdvertisingInterval
- BOOL isLost
- BOOL hasBadMac

6.2.1 Detailed Description

Statistics about the advertising packets that LibMuse sees from Muse 2016 (MU_02) or later headbands.

Warning

This is only available on Android devices.

6.2.2 Method Documentation

6.2.2.1 advertisingStatsWithNumAdvertisingPackets:avgAdvertisingInterval:sigmaAdvertisingInterval:maxAdvertisingI

+ (nonnull instancetype) advertisingStatsWithNumAdvertisingPackets:
(int32_t) numAdvertisingPackets
avgAdvertisingInterval:(double) avgAdvertisingInterval
sigmaAdvertisingInterval:(double) sigmaAdvertisingInterval
maxAdvertisingInterval:(double) maxAdvertisingInterval
isLost:(BOOL) isLost
hasBadMac:(BOOL) hasBadMac

6.2.2.2 initWithNumAdvertisingPackets:avgAdvertisingInterval:sigmaAdvertisingInterval:maxAdvertisingInterval:isLost

- (nonnull instancetype) initWithNumAdvertisingPackets:
(int32_t) numAdvertisingPackets
avgAdvertisingInterval:(double) avgAdvertisingInterval
sigmaAdvertisingInterval:(double) sigmaAdvertisingInterval
maxAdvertisingInterval:(double) maxAdvertisingInterval
isLost:(BOOL) isLost
hasBadMac:(BOOL) hasBadMac

6.2.3 Property Documentation

6.2.3.1 avgAdvertisingInterval

- (double) avgAdvertisingInterval [read], [nonatomic], [assign]

Returns the average interval in seconds between advertising packets. This is a running average since the time the statistics were created or reset.

Returns

the average interval between advertising packets.

6.2.3.2 hasBadMac

- (BOOL) hasBadMac [read], [nonatomic], [assign]

Indicates an issue with the MAC address of the headband.

Returns

true if for some reason the MAC address of a headband has changed since initial detection. false under normal circumstances.

6.2.3.3 isLost

- (BOOL) isLost [read], [nonatomic], [assign]

Indicates if the phone has lost contact with the Muse headband. This is a subjective measurement. Even if this is true, the phone may still be able to connect with the headband if a connection is requested.

Returns

true if the phone has lost contact with the headband. false otherwise.

6.2.3.4 maxAdvertisingInterval

- (double) maxAdvertisingInterval [read], [nonatomic], [assign]

Returns the maximum interval in seconds between receiving 2 advertising packets.

Returns

the maximum interval between advertising packets.

6.2.3.5 numAdvertisingPackets

- (int32_t) numAdvertisingPackets [read], [nonatomic], [assign]

Returns the number of advertising packets seen.

Returns

the number of advertising packets seen.

6.2.3.6 sigmaAdvertisingInterval

- (double) sigmaAdvertisingInterval [read], [nonatomic], [assign]

Returns the standard deviation of the average advertising interval.

Returns

the standard deviation of the average advertising interval.

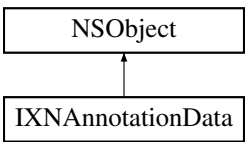
The documentation for this class was generated from the following file:

- IXNAdvertisingStats.h

6.3 IXNAnnotationData Class Reference

#include <IXNAnnotationData.h>

Inheritance diagram for IXNAnnotationData:



Instance Methods

- (nonnull instancetype) - initWithData:format:eventType:eventId:parentId:

Class Methods

- (nonnull instancetype) + annotationDataWithData:format:eventType:eventId:parentId:

Properties

- NSString * data
- IXNAnnotationFormat format
- NSString * eventType
- NSString * eventId
- NSString * parentId

6.3.1 Detailed Description

A data structure that corresponds to an Annotation message in the .muse file format.

6.3.2 Method Documentation

6.3.2.1 annotationDataWithData:format:eventType:eventId:parentId:

```
+ (nonnull instancetype) annotationDataWithData:
                    (nonnull NSString *) data
                    format:(IXNAnnotationFormat) format
                    eventType:(nonnull NSString *) eventType
                    eventId:(nonnull NSString *) eventId
                    parentId:(nonnull NSString *) parentId
```

6.3.2.2 initWithData:format:eventType:eventId:parentId:

```
- (nonnull instancetype) initWithData:
                    (nonnull NSString *) data
                    format:(IXNAnnotationFormat) format
                    eventType:(nonnull NSString *) eventType
                    eventId:(nonnull NSString *) eventId
                    parentId:(nonnull NSString *) parentId
```

6.3.3 Property Documentation

6.3.3.1 data

- (NSString*) data [read], [nonatomic], [assign]

This is additional data for the event, can be in any format specified by IXNAnnotationFormat

6.3.3.2 eventId

- (NSString∗) eventId [read], [nonatomic], [assign]

This can be used to pair events together or give them unique IDs. If you do not need this value, set it to the empty string "".

6.3.3.3 eventType

- (NSString∗) eventType [read], [nonatomic], [assign]

This can be used to classify events, for example "blink" or "error". If you do not need this value, set it to the empty string "".

6.3.3.4 format

- (IXNAnnotationFormat) format [read], [nonatomic], [assign]

This specifies the string format of the event data.

6.3.3.5 parentId

- (NSString∗) parentId [read], [nonatomic], [assign]

This can be used to pair events together or give them unique IDs. If you do not need this value, set it to the empty string "".

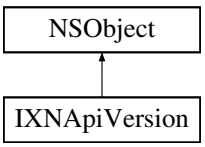
The documentation for this class was generated from the following file:

- IXNAnnotationData.h

6.4 IXNApiVersion Class Reference

#include <IXNApiVersion.h>

Inheritance diagram for IXNApiVersion:



Instance Methods

- (int64_t) - getMonotonic
- (int64_t) - getMajor
- (int64_t) - getMinor
- (int64_t) - getPatch
- (int64_t) - getApi
- (nonnull NSString ∗) - getString

6.4.1 Detailed Description

Library version interface.

Describes the version of LibMuse.

6.4.2 Method Documentation

6.4.2.1 getApi

- (int64_t) getApi

API

This number is incremented when changes are introduced to the API that are not backwards compatible.

Returns

The API version.

6.4.2.2 getMajor

- (int64_t) getMajor

Major release.

This number is incremented on a major release of new functionality.

Returns

The major version.

6.4.2.3 getMinor

- (int64_t) getMinor

Minor release.

This number is incremented when minor changes are made that are backwards compatible.

Returns

The minor version.

6.4.2.4 getMonotonic

- (int64_t) getMonotonic

Returns a monotonically increasing version number.

This number is guaranteed to increase from release to release. Currently it is equivalent to:

get_major() * 1000000 + get_minor() * 10000 + get_patch() * 100 + get_api()

So for major = 5, minor = 1, patch = 3, api = 2, it would be 5010302.

6.4.2.5 getPatch

- (int64_t) getPatch

Patch release.

This number is incremented when backwards-compatible bugfixes are made.

Returns

The patch version.

6.4.2.6 getString

- (nonnull NSString *) getString

String representation of the version.

This is a string like "x.y.z API Version j" where x is get_major(), y is get_minor(), z is get_patch() and j is get_api(). The format may change from release to release. In particular, don't make assumptions about it or try to parse it. Use the other API calls on this interface to get the numerical versions.

Returns

The version string.

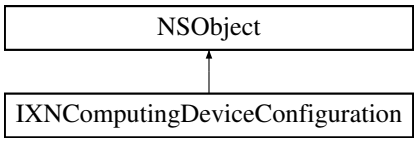
The documentation for this class was generated from the following file:

- IXNApiVersion.h

6.5 IXNComputingDeviceConfiguration Class Reference

#include <IXNComputingDeviceConfiguration.h>

Inheritance diagram for IXNComputingDeviceConfiguration:



Instance Methods

- (nonnull instancetype) - initWithOsType:osVersion:hardwareModelName:hardwareModelId:processorName:processo

Class Methods

- (nonnull instancetype) + computingDeviceConfigurationWithOsType:osVersion:hardwareModelName:hardwareMod

Properties

- NSString * osType
- NSString * osVersion
- NSString * hardwareModelName
- NSString * hardwareModelId
- NSString * processorName
- NSString * processorSpeed
- int32_t numberOfProcessors
- NSString * memorySize
- NSString * bluetoothVersion
- NSString * timeZone
- int32_t timeZoneOffsetSeconds

6.5.1 Detailed Description

This data structure contains fields, which describe the running machine. To prevent confusion, field names should correspond to names in the protobuf schema of the .muse file format. If the computing device configuration cannot be obtained for any field, they will be empty strings.

6.5.2 Method Documentation

6.5.2.1 computingDeviceConfigurationWithOsType:osVersion:hardwareModelName:hardwareModelId:processorName:p

+ (nonnull instancetype) computingDeviceConfigurationWithOsType:
(nonnull NSString *) osType
osVersion:(nonnull NSString *) osVersion
hardwareModelName:(nonnull NSString *) hardwareModelName
hardwareModelId:(nonnull NSString *) hardwareModelId
processorName:(nonnull NSString *) processorName
processorSpeed:(nonnull NSString *) processorSpeed
numberOfProcessors:(int32_t) numberOfProcessors
memorySize:(nonnull NSString *) memorySize
bluetoothVersion:(nonnull NSString *) bluetoothVersion
timeZone:(nonnull NSString *) timeZone
timeZoneOffsetSeconds:(int32_t) timeZoneOffsetSeconds

6.5.2.2 initWithOsType:osVersion:hardwareModelName:hardwareModelId:processorName:processorSpeed:numberOfPro

- (nonnull instancetype) initWithOsType:
(nonnull NSString ∗) osType
osVersion:(nonnull NSString ∗) osVersion
hardwareModelName:(nonnull NSString ∗) hardwareModelName
hardwareModelId:(nonnull NSString ∗) hardwareModelId
processorName:(nonnull NSString ∗) processorName
processorSpeed:(nonnull NSString ∗) processorSpeed
numberOfProcessors:(int32_t) numberOfProcessors
memorySize:(nonnull NSString ∗) memorySize
bluetoothVersion:(nonnull NSString ∗) bluetoothVersion
timeZone:(nonnull NSString ∗) timeZone
timeZoneOffsetSeconds:(int32_t) timeZoneOffsetSeconds

6.5.3 Property Documentation

6.5.3.1 bluetoothVersion

- (NSString∗) bluetoothVersion [read], [nonatomic], [assign]

Bluetooth version

6.5.3.2 hardwareModelId

- (NSString∗) hardwareModelId [read], [nonatomic], [assign]

Hardware model id. For ex.: ”SM-N900W8”, ”MacBookPro-101”

6.5.3.3 hardwareModelName

- (NSString∗) hardwareModelName [read], [nonatomic], [assign]

Hardware model. For ex.: ”Samsung Galaxy Note 3”, ”Macbook Pro”

6.5.3.4 memorySize

- (NSString∗) memorySize [read], [nonatomic], [assign]

Memory size. For ex.: ”500MB”, ”2000MB”

6.5.3.5 numberOfProcessors

- (int32_t) numberOfProcessors [read], [nonatomic], [assign]

Number of cores

6.5.3.6 osType

- (NSString∗) osType [read], [nonatomic], [assign]

Operation system type. For ex.: ”Android”, ”iOS”, ”Windows”

6.5.3.7 osVersion

- (NSString∗) osVersion [read], [nonatomic], [assign]

Operation system version. For ex.: ”10.1”, ”4.4.2”

6.5.3.8 processorName

- (NSString∗) processorName [read], [nonatomic], [assign]

Processor name. For ex.: ”Intel”, ”ARM”

6.5.3.9 processorSpeed

- (NSString∗) processorSpeed [read], [nonatomic], [assign]

Processor frequency in Hz

6.5.3.10 timeZone

- (NSString∗) timeZone [read], [nonatomic], [assign]

time zone indicator

6.5.3.11 timeZoneOffsetSeconds

- (int32_t) timeZoneOffsetSeconds [read], [nonatomic], [assign]

time zone offset in seconds

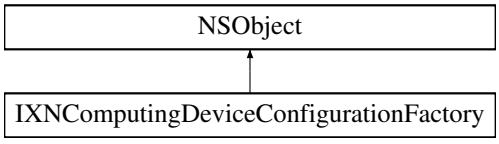
The documentation for this class was generated from the following file:

- IXNComputingDeviceConfiguration.h

6.6 IXNComputingDeviceConfigurationFactory Class Reference

#include <IXNComputingDeviceConfigurationFactory.h>

Inheritance diagram for IXNComputingDeviceConfigurationFactory:



Class Methods

- (nonnull IXNComputingDeviceConfigurationFactory *) + getInstance

Properties

- IXNComputingDeviceConfiguration * computingDeviceConfiguration

6.6.1 Detailed Description

Provides access to the IXNComputingDeviceConfiguration object containing information about the computing device.

6.6.2 Method Documentation

6.6.2.1 getInstance

+ (nonnull IXNComputingDeviceConfigurationFactory *) getInstance

Static constructor for the singleton object.

Returns

An instance of the IXNComputingDeviceConfigurationFactory object.

6.6.3 Property Documentation

6.6.3.1 computingDeviceConfiguration

- (IXNComputingDeviceConfiguration*) computingDeviceConfiguration [read], [write], [nonatomic], [assign]

The IXNComputingDeviceConfiguration structure for the current computing device.

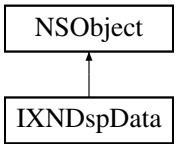
The documentation for this class was generated from the following file:

- IXNComputingDeviceConfigurationFactory.h

6.7 IXNDspData Class Reference

#include <IXNDspData.h>

Inheritance diagram for IXNDspData:



Instance Methods

- (nonnull instancetype) - initWithType:floatArray:intArray:version:

Class Methods

- (nonnull instancetype) + dspDataWithType:floatArray:intArray:version:

Properties

- NSString * type
- NSArray< NSNumber * > * floatArray
- NSArray< NSNumber * > * intArray
- NSString * version

6.7.1 Detailed Description

Data struct corresponds to a DSP message in the .muse file format. This message is very generic and allows you to save custom messages.

6.7.2 Method Documentation

6.7.2.1 dspDataWithType:floatArray:intArray:version:

```
+ (nonnull instancetype) dspDataWithType:
                (nonnull NSString *) type
                floatArray:(nonnull NSArray< NSNumber * > *) floatArray
                intArray:(nonnull NSArray< NSNumber * > *) intArray
                version:(nonnull NSString *) version
```

6.7.2.2 initWithType:floatArray:intArray:version:

```
- (nonnull instancetype) initWithType:
                (nonnull NSString *) type
                floatArray:(nonnull NSArray< NSNumber * > *) floatArray
                intArray:(nonnull NSArray< NSNumber * > *) intArray
                version:(nonnull NSString *) version
```

6.7.3 Property Documentation

6.7.3.1 floatArray

- (NSArray<NSNumber *>*) floatArray [read], [nonatomic], [assign]

array of floating point data

6.7.3.2 intArray

- (NSArray<NSNumber *>*) intArray [read], [nonatomic], [assign]

array of integer data

6.7.3.3 type

- (NSString*) type [read], [nonatomic], [assign]

Type of your message

6.7.3.4 version

- (NSString*) version [read], [nonatomic], [assign]

version of your message format

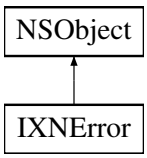
The documentation for this class was generated from the following file:

- IXNDspData.h

6.8 IXNError Class Reference

#include <IXNError.h>

Inheritance diagram for IXNError:



Instance Methods

- (nonnull instancetype) - initWithType:code:info:

Class Methods

- (nonnull instancetype) + errorWithType:code:info:

Properties

- IXNErrorType type
- int32_t code
- NSString * info

6.8.1 Detailed Description

Represents an error from an operation (synchronous or asynchronous). These will be forwarded to any ErrorListeners that are registered.

See also

IXNMuseErrorListener

IXNMuse::registerErrorListener

6.8.2 Method Documentation

6.8.2.1 errorWithType:code:info:

```
+ (nonnull instancetype) errorWithType:
                (IXNErrorType) type
                code:(int32_t) code
                info:(nonnull NSString *) info
```

6.8.2.2 initWithType:code:info:

```
- (nonnull instancetype) initWithType:
                (IXNErrorType) type
                code:(int32_t) code
                info:(nonnull NSString *) info
```

6.8.3 Property Documentation

6.8.3.1 code

- (int32_t) code [read], [nonatomic], [assign]

Machine-parseable error code

Returns

a machine parseable error code.

6.8.3.2 info

- (NSString*) info [read], [nonatomic], [assign]

Human-readable description.

Returns

the english error message.

6.8.3.3 type

- (IXNErrorType) type [read], [nonatomic], [assign]

Type of error

Returns

the type of error that has occurred.

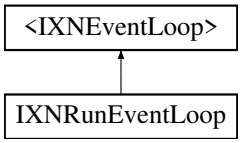
The documentation for this class was generated from the following file:

- IXNError.h

6.9 <IXNEventLoop> Protocol Reference

#include <IXNEventLoop.h>

Inheritance diagram for <IXNEventLoop>:



Instance Methods

- (void) - post:
- (void) - postDelayed:delayMilliseconds:
- (void) - cancel

6.9.1 Detailed Description

Wrapper around a platform-specific event loop.

6.9.2 Method Documentation

6.9.2.1 cancel

- (void) cancel

Cancel all pending actions.

6.9.2.2 post:

- (void) post:
(nullable id< IXNAction >) action

Posts an action to the event loop for processing on the next pass.

Parameters

action	The action to perform.

6.9.2.3 postDelayed:delayMilliseconds:

- (void) postDelayed:
(nullable id< IXNAction >) action
delayMilliseconds:(int64_t) delayMilliseconds

Posts an action to the event loop for processing after the specified delay in milliseconds has passed.

Parameters

action	The action to perform.
delayMilliseconds	The time to wait before posting the action to the event loop for processing.

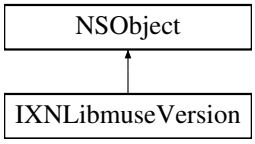
The documentation for this protocol was generated from the following file:

- IXNEventLoop.h

6.10 IXNLibmuseVersion Class Reference

#include <IXNLibmuseVersion.h>

Inheritance diagram for IXNLibmuseVersion:



Class Methods

- (nullable IXNApiVersion *) + instance

6.10.1 Detailed Description

The version of the LibMuse library.

6.10.2 Method Documentation

6.10.2.1 instance

+ (nullable IXNApiVersion *) instance

Returns the LibMuse API version.

Returns

the LibMuse API version.

The documentation for this class was generated from the following file:

- IXNLibmuseVersion.h

6.11 <IXNLogListener> Protocol Reference

#include <IXNLogListener.h>

Instance Methods

- (void) - receiveLog:

6.11.1 Detailed Description

User-supplied log message handler.

You can provide an instance of this class in order to integrate LibMuse logs into your application's logging system.

See also

IXNLogManager

6.11.2 Method Documentation

6.11.2.1 receiveLog:

- (void) receiveLog:
(nonnull IXNLogPacket *) log

Handler method for log messages. These are sent when interesting events happen. A simple handler may just print each message to stderr or stdout separated by newlines. It's also possible to implement arbitrary log policies within this handler.

Warning

receiveLog may be entered concurrently from multiple threads; if your listener implementation requires synchronization, it should provide its own.

Your listener should not make any LibMuse API calls; otherwise, unbounded recursion may occur, leading to resource exhaustion.

Parameters

log	The log message.

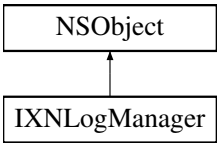
The documentation for this protocol was generated from the following file:

- IXNLogListener.h

6.12 IXNLogManager Class Reference

#include <IXNLogManager.h>

Inheritance diagram for IXNLogManager:



Instance Methods

- (nullable id< IXNLogListener >) - makeDefaultLogListener
- (void) - setLogListener:
- (void) - setMinimumSeverity:
- (void) - writeLog:raw:tag:message:
- (int64_t) - getTimestamp
- (double) - timeSince:

Class Methods

- (nullable IXNLogManager *) + instance

6.12.1 Detailed Description

Manages the listener for any logging events generated by LibMuse.

To disable logging:

setLogListener(null);

To install a custom log listener for your application:

setLogListener(your_log_listener);

To restore default logging to standard error (if you have previously changed the log listener):

setLogListener(makeDefaultLogListener());

An instance of the default listener is installed unless another listener is supplied. The default log listener prints all messages to standard error as they show up, without any buffering, using the following formatting rules:

- Raw logs are printed directly without any formatting or newline.
- Other messages are formatted to pretty-print with the tag, timestamp, severity, and log message on a single line (prepending a newline if the last message printed was raw.)

For example, suppose that these packets showed up in order: {info, false, ”A”, 4, ”thing A”}, {info, true, ”FOO”, 1, ”+”}, {err, true, ”BAR”, 3, ”-”}, {warn, false, ”B”, 2, ”thing B”} where the second parameter idicates if this is a raw log message. The log output would be formatted roughly as such:

```
A [I 4.000] thing A
+-
B [W 2.000] thing B
```

In order to provide this formatting without buffering or losing messages, the default log listener uses a mutex, effectively making the logging subsystem single-threaded and IO-bound.

See also

IXNLogListener

6.12.2 Method Documentation

6.12.2.1 getTimestamp

- (int64_t) getTimestamp

Return the current timestamp.

The returned value is in microseconds since some epoch – January 1 1970 on UNIX, but possibly other values on other systems.

Returns

The current timestamp.

6.12.2.2 instance

+ (nullable IXNLogManager *) instance

Returns the shared log manager instance.

Returns

The shared log manager instance.

6.12.2.3 makeDefaultLogListener

- (nullable id< IXNLogListener >) makeDefaultLogListener

Constructs and returns a default log listener.

Returns

an instance of the default log listener.

6.12.2.4 setLogListener:

- (void) setLogListener:
(nullable id< IXNLogListener >) listener

Sets the log listener.

There is only one log listener active at a time; hence this is just a plain setter rather than a register / unregister pair.

Parameters

listener	the LogListener to use.

6.12.2.5 setMinimumSeverity:

- (void) setMinimumSeverity:
(IXNSeverity) severity

Set the minimum log severity.

Messages at or above this severity will be passed to the configured log listener. Messages below this severity will be dropped.

Parameters

severity	the minimum log severity to log.

6.12.2.6 timeSince:

- (double) timeSince:
(int64_t) timestamp

Return the time elapsed in seconds since the passed timestamp.

Parameters

timestamp	The prior timestamp.

6.12.2.7 writeLog:raw:tag:message:

- (void) writeLog:
(IXNSeverity) severity
raw:(BOOL) raw
tag:(nonnull NSString *) tag
message:(nonnull NSString *) message

Write a log line.

This method sends messages to the configured log listener. It is exposed here primarily so that LibMuse can access it from its own platform-specific code, but there's nothing stopping you from using it and doing all your logging through LibMuse. Except please don't, because we may change the API at any time, and why would you?

The log's timestamp is the number of seconds since this log manager was created.

Parameters

severity	The severity of this log message.
raw	true if this is a raw log message (no formatting), false otherwise
tag	The tag for this message.
message	The log message.

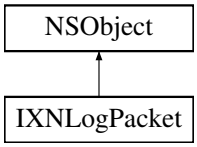
The documentation for this class was generated from the following file:

- IXNLogManager.h

6.13 IXNLogPacket Class Reference

#include <IXNLogPacket.h>

Inheritance diagram for IXNLogPacket:



Instance Methods

- (nonnull instancetype) - initWithSeverity:raw:tag:timestamp:message:

Class Methods

- (nonnull instancetype) + logPacketWithSeverity:raw:tag:timestamp:message:

Properties

- IXNSeverity severity
- BOOL raw
- NSString * tag
- double timestamp
- NSString * message

6.13.1 Detailed Description

A log message.

6.13.2 Method Documentation

6.13.2.1 initWithSeverity:raw:tag:timestamp:message:

```
- (nonnull instancetype) initWithSeverity:
                    (IXNSeverity) severity
                    raw:(BOOL) raw
                    tag:(nonnull NSString *) tag
                    timestamp:(double) timestamp
                    message:(nonnull NSString *) message
```

6.13.2.2 logPacketWithSeverity:raw:tag:timestamp:message:

```
+ (nonnull instancetype) logPacketWithSeverity:
                    (IXNSeverity) severity
                    raw:(BOOL) raw
                    tag:(nonnull NSString *) tag
                    timestamp:(double) timestamp
                    message:(nonnull NSString *) message
```

6.13.3 Property Documentation

6.13.3.1 message

- (NSString*) message [read], [nonatomic], [assign]

Content of this log message. Not newline-terminated; handlers should provide their own line-based formatting.

Returns

The log message content.

6.13.3.2 raw

- (BOOL) raw [read], [nonatomic], [assign]

Denotes raw messages.

Raw messages are generally short (one or two character) messages that can be passed straight to the screen without any formatting or added newlines. (We use this to reduce the noise level on some of our verbose log mesages.)

Returns

true if this is a raw log message, false otherwise.

6.13.3.3 severity

- (IXNSeverity) severity [read], [nonatomic], [assign]

Severity of this message.

Returns

The severity of this message.

6.13.3.4 tag

- (NSString*) tag [read], [nonatomic], [assign]

Human-readable, usually all-caps, tag for this message, e.g. "CONNECTOR" or "DEVICE".

Returns

The message tag.

6.13.3.5 timestamp

- (double) timestamp [read], [nonatomic], [assign]

Timestamp for this log, in seconds.

Returns

The timestamp of this log message.

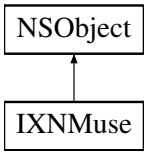
The documentation for this class was generated from the following file:

- IXNLogPacket.h

6.14 IXNMuse Class Reference

#include <IXNMuse.h>

Inheritance diagram for IXNMuse:



Instance Methods

- (void) - connect
- (void) - disconnect
- (void) - execute
- (void) - runAsynchronously
- (IXNConnectionState) - getConnectionState
- (nonnull NSString *) - getMacAddress
- (nonnull NSString *) - getName
- (double) - getRssi
- (IXNMuseModel) - getModel
- (double) - getLastDiscoveredTime
- (void) - setNumConnectTries:
- (nullable IXNMuseConfiguration *) - getMuseConfiguration
- (nullable IXNMuseVersion *) - getMuseVersion
- (void) - registerConnectionListener:
- (void) - unregisterConnectionListener:
- (void) - registerDataListener:type:
- (void) - unregisterDataListener:type:
- (void) - registerErrorListener:
- (void) - unregisterErrorListener:
- (void) - unregisterAllListeners
- (void) - setPreset:
- (void) - enableLedIndicator:
- (void) - enableDataTransmission:
- (void) - setNotchFrequency:
- (BOOL) - isLowEnergy
- (BOOL) - isPaired
- (BOOL) - isConnectable
- (void) - setLicenseData:
- (void) - enableException:
- (void) - setProperty:value:

6.14.1 Detailed Description

Provides the client interface to a specific Muse Headband.

This class maps to a single Muse Headband, providing an API for receiving connection events and data packets, as well as the ability to inspect various properties of the headband it corresponds to. Users can initiate and halt a connection to the device, enable data transmission, find out the human-readable name of the headband, etc.

This class should never be constructed directly by a user; rather, references to it can be retrieved from MuseManager.

There are two main ways of interacting with a Muse:

1. calling runAsynchronously (IXNMuse)

2. calling connect (IXNMuse) on your own and then repeatedly calling execute (IXNMuse)

If you call runAsynchronously (IXNMuse) , you do not need to call connect (IXNMuse) or execute (IXNMuse) on your own. Everything is taken care of by LibMuse for the duration of the connection (i.e. until you receive a disconnected event).

If you call connect (IXNMuse) on your own, you are responsible for calling execute (IXNMuse) on your own as well. Execute performs small, non-blocking steps of work such as initiating a connection, streaming data or disconnecting. If execute (IXNMuse) is not called, no work is performed. The Muse will not transition its connection state and you will not receive any data. If you are using this method of interacting with a Muse you need to continue calling execute (IXNMuse) until you receive a DISCONNECTED event through a registered connection listener. It may take multiple execute calls to reach the DISCONNECTED after issuing the disconnect request. Failure to wait for the DISCONNECTED event can result in unanticipated consequences.

6.14.2 Method Documentation

6.14.2.1 connect

- (void) connect

Initiate a connection to a Muse headband.

This call is non-blocking and returns immediately. Since this returns before the connection is established, you should instead use a connection listener to implement any logic, such as updating a UI element, on successful connection. When the connection is successful you will receive a CONNECTED event. If the connection is unsuccessful you will receive a DISCONNECTED event.

Threading: method is thread-safe.

6.14.2.2 disconnect

- (void) disconnect

Disconnects your mobile device from Muse Headband.

If you are calling execute on your own, remember to continue calling execute until you receive the DISCONNECTED event.

Threading: method is thread-safe.

If you don't want to receive disconnection event, unregister listeners manually first.

6.14.2.3 enableDataTransmission:

- (void) enableDataTransmission:
(BOOL) enable

Starts/stops data transmission (but keep-alive packets will be still sent). If you're using low-level interface (connect + execute), you should still call execute to continue sending keep-alive messages. If you're using runAsynchronously , then it will take care about sending keep-alive packets.
Threading: method is thread-safe. You can call it in the middle of an execute operation. If method is called before connection is established, the right setting will be passed to device during connection routine.

Parameters

enable	true to start streaming data. false to pause the data stream.

6.14.2.4 enableException:

- (void) enableException:
(BOOL) enable

Enable/disable libmuse to rethrow any exceptions caught in your app code from the various listeners. For example, if there is a bug in your data listener code that results in an exception. Libmuse will catch that and it can rethrow that to cause your app to crash if enabled. Or if disabled it will swallow that exception and prevent the app from crashing. It is useful to enable this in development to help track down any bugs in your code more easily. It is best to disable this in production code to prevent your app from crashing. If this method is not called, the default is disabled.

Parameters

enable	true to enable exception to be thrown. false to disable.

6.14.2.5 enableLedIndicator:

- (void) enableLedIndicator:
(BOOL) enable

Toggles the LED indicator state on Muse headbands supporting sleep.

Warning

> The headband must be in the CONNECTED state before calling this method. Threading: method is thread-safe.

Parameters

enable	true to turn indicator LEDs on. false to turn them off.

6.14.2.6 execute

- (void) execute

Runs a single, non-blocking step of processing.

Reads data from Bluetooth if there is any; updates the connection state; sends packets to listeners; etc. This should be called relatively frequently: max 250ms, ideally 20ms.

If you are listening for computed values (e.g. band powers) rather than just raw EEG samples, then be aware that this method may trigger some amount of processing in the foreground before it returns. If this has undesirable effects on app performance, then you may want to move it to a dedicated thread.

Also be aware that all packets will be received on the same thread as execute is run.

Threading: This method is NOT thread safe. You must synchronize the Muse object on your own if you wish to call it concurrently from more than one thread.

6.14.2.7 getConnectionState

- (IXNConnectionState) getConnectionState

Returns current connection state.

Threading: method is thread-safe.

Returns

The current connection state.

6.14.2.8 getLastDiscoveredTime

- (double) getLastDiscoveredTime

Returns the time at which this device was most recently discovered.

Muse devices broadcast service information every few seconds, and this is the last time LibMuse has heard this broadcast info.

Only implemented on low-energy Muses. Returns NaN if called on non-low-energy Muses.

The value is in microseconds since some common start time (e.g. epoch, device start time, or app start time) that is guaranteed not to change within a running app process. The value may be used e.g. to determine which of two devices was discovered more recently than the other. It should not be used as an absolute time, nor should the common start time be assumed to be the same between any two running app instances.

This value is computed via:

CACurrentMediaTime() * 1000.0 * 1000.0.

You may therefore use CACurrentMediaTime() to recover deltas from the current time to e.g. implement ”only muses discovered in the last 20 seconds.”

Returns

The time at which this device was most recently discovered.

6.14.2.9 getMacAddress

- (nonnull NSString *) getMacAddress

Returns Bluetooth MAC address of the Muse Headband.

Threading: method is thread-safe.

Returns

The MAC address of the Muse Headband.

6.14.2.10 getModel

- (IXNMuseModel) getModel

Returns the model of the connected Muse Headband.

For Muse2 2018 (MU_03) and later, the Muse must have reached the CONNECTED state before this method is called. Otherwise it will return MU_02.

This should return a value for every Muse currently available.

Threading: method is thread-safe.

Returns

The model of the Muse Headband.

6.14.2.11 getMuseConfiguration

- (nullable IXNMuseConfiguration *) getMuseConfiguration

Returns struct which contains all information about Muse configuration.

Threading: method is thread-safe.

Warning

The Muse Configuration object is only populated during connection routine or after headband settings (like preset or notch frequency) are changed. If this is called before the Muse is connected, the configuration will be null.

Returns

The configuration information of this Muse or null if the configuration is unknown.

6.14.2.12 getMuseVersion

- (nullable IXNMuseVersion *) getMuseVersion

Returns all information about the version of the Muse.

Threading: method is thread-safe.

Warning

The Muse Version is populated during connection routine only. If this is called before the Muse is connected, the version will be null.

Returns

The version of this Muse or null if the version is unknown.

6.14.2.13 getName

- (nonnull NSString *) getName

Returns Bluetooth name of the Muse Headband.

Threading: method is thread-safe.

Returns

The name of the Muse Headband.

6.14.2.14 getRssi

- (double) getRssi

Returns the RSSI of this device.

Only implemented on low-energy Muses. Returns NaN if called on non-low-energy Muses.

Threading: method is thread-safe.

Returns

The RSSI of a low-energy Muse or NaN if the Muse is not low-energy.

6.14.2.15 isConnectable

- (BOOL) isConnectable

Returns true if the Muse is connectable, false otherwise. This will always return true for Muse 2014 (MU_01). For Muse 2016 (MU_02) or later headbands this will return true on Android and iOS. For Muse 2016 (MU_02) or later headbands on Windows, this will return true if the headband is advertising it is connectable, false if it is not.

Returns

true if the Muse is connectable, false otherwise.

6.14.2.16 isLowEnergy

- (BOOL) isLowEnergy

True if this device supports Bluetooth Low-Energy.

Returns

true if this Muse supports Bluetooth Low-Energy. false if it does not.

6.14.2.17 isPaired

- (BOOL) isPaired

Returns true if the Muse is paired with the OS, false otherwise. This will always return true for Muse 2014 (MU_01). For Muse 2016 (MU_02) or later headbands this will return false on Android and iOS. For Muse 2016 (MU_02) or later headbands on Windows, this will return true if the headband is paired with the OS and false if it is not.

Returns

true if the Muse is paired with the OS, false otherwise.

6.14.2.18 registerConnectionListener:

- (void) registerConnectionListener:
(nullable id< IXNMuseConnectionListener >) listener

Registers a connection listener. The same listener cannot be registered twice. If the listener was already registered, then this method does nothing.

Threading: method is thread-safe.

Parameters

listener	The listener to register.

6.14.2.19 registerDataListener:type:

- (void) registerDataListener:
(nullable id< IXNMuseDataListener >) listener
type:(IXNMuseDataPacketType) type

Registers a data listener. You can register the same listener to listen for different packet types - just call this method again. It's your responsibility to make sure that the listener handles all packet types correctly.

If this listener was already registered for this specific type then this method does nothing.

Threading: method is thread-safe.

Parameters

listener	The listener to register.
type	The type of data packet the listener will receive.

6.14.2.20 registerErrorListener:

- (void) registerErrorListener:
(nullable id< IXNMuseErrorListener >) listener

Registers an error listener.

Threading: method is thread-safe.

Parameters

listener	The listener to register.

6.14.2.21 runAsynchronously

- (void) runAsynchronously

Connect to a headband out of the main logic flow.

This method handles all the connection logic for a single connection to a headband (i.e. starting with issuing a "connecting" event and ending with issuing a "disconnected" event). It may run in a separate thread or an event loop on the main thread; the details are platform-specific.

If this method is used, then all packets are delivered on the main thread.

The lifetime of effect of this method ends on disconnection; if you want to initiate a new connection to a headband that has disconnected, you should call runAsynchronously on it again.

If this method is called multiple times within the lifetime of a single connection, any subsequent calls will do nothing other than print a log message.

Note that there is currently a race condition if runAsynchronously is called

1. as a result of a DISCONNECTED packet, and
2. on a different thread from the one that delivered the notification.

This will be resolved in a future library release. For now, call runAsynchronously on the same thread that sent the DISCONNECTED packet, or after some modest delay (say 1 second.)

Threading: This method is thread safe.

6.14.2.22 setLicenseData:

- (void) setLicenseData:
(nonnull NSData *) data

Allows forwarding of license data from cloud.

Parameters

data	The encrypted license blob.

6.14.2.23 setNotchFrequency:

- (void) setNotchFrequency:
(IXNNotchFrequency) newFrequency

Changes notch frequency (power line frequency).

Threading: method is thread-safe. You can call it in the middle of execute operation, but in this case be aware that this operation will interrupt data streaming to set new notch frequency. Data streaming will be restored after that. If method is called before a connection is established, the right setting will be passed to device during connection routine.

When combining a call setNotchFrequency with IXNMuse::setPreset setNotchFrequency can be called before or after the call to IXNMuse::setPreset The notch frequency will be applied to the preset after the preset is set.

Warning

> This is only supported on Muse 2014 (MU_01) headbands and then only with presets: IXNMusePresetPreset10 , IXNMusePresetPreset12 and IXNMusePresetPreset14 Under those presets, the only valid frequencies are: IXNNotchFrequencyNotch50hz IXNNotchFrequencyNotch60hz
>
> Calling this with IXNNotchFrequencyNotchNone will do nothing.
>
> This does nothing on Muse 2016 (MU_02) or later headbands and on Muse 2014 (MU_01) headbands with presets IXNMusePresetPresetAb or IXNMusePresetPresetAd

Parameters

newFrequency	The new notch frequency.

6.14.2.24 setNumConnectTries:

- (void) setNumConnectTries:
(int32_t) numTries

When LibMuse tries connect to a Muse, it will by default try 3 times before giving up. This function allows you to change the number of tries from 3 to anything you want. numTries must be a positive integer. This function only works for model MU-02 Muses, calling it on a MU-01 will do nothing. This function was added for internal testing purposes, and we do not anticipate that 3rd party clients of the LibMuse library will need to modify this value.

Threading: This method is thread safe.

Parameters

numTries	The number of times to try to connect before giving up.

6.14.2.25 setPreset:

- (void) setPreset:
(IXNMusePreset) preset

Changes Muse Headband settings. Threading: method is thread-safe. You can call it in the middle of execute operation, but in this case be aware that this operation will interrupt data streaming to set new preset. Data streaming will be restored after that. If method is called before connection is established, the right setting will be passed to device during connection routine.

Parameters

preset	The new preset.

6.14.2.26 setProperty:value:

- (void) setProperty:
(nonnull NSString ∗) name
value:(nonnull NSString ∗) value

Factory use only

6.14.2.27 unregisterAllListeners

- (void) unregisterAllListeners

Unregisters all registered connection listeners and data listeners.

Threading: method is thread-safe.

6.14.2.28 unregisterConnectionListener:

- (void) unregisterConnectionListener:
(nullable id< IXNMuseConnectionListener >) listener

Unregisters connection listeners.

Threading: method is thread-safe.

Parameters

listener	The listener to unregister.

6.14.2.29 unregisterDataListener:type:

- (void) unregisterDataListener:
(nullable id< IXNMuseDataListener >) listener
type:(IXNMuseDataPacketType) type

Unregisters a data listener that was registered before.

If the listener was not registered before, then this method does nothing.

Threading: method is thread-safe.

Parameters

listener	The listener to unregister.
type	The type of data packet the listener will stop receiving.

6.14.2.30 unregisterErrorListener:

- (void) unregisterErrorListener:
(nullable id< IXNMuseErrorListener >) listener

Unregisters an info listener that was registered before.

If the listener was not registered before, then this method does nothing.

Threading: method is thread-safe.

Parameters

listener	The listener to unregister.

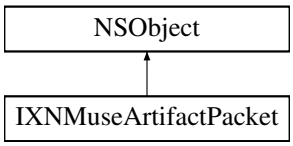
The documentation for this class was generated from the following file:

- IXNMuse.h

6.15 IXNMuseArtifactPacket Class Reference

#include <IXNMuseArtifactPacket.h>

Inheritance diagram for IXNMuseArtifactPacket:



Instance Methods

- (nonnull instancetype) - initWithHeadbandOn:blink:jawClench:timestamp:

Class Methods

- (nonnull instancetype) + museArtifactPacketWithHeadbandOn:blink:jawClench:timestamp:

Properties

- BOOL headbandOn
- BOOL blink
- BOOL jawClench
- int64_t timestamp

6.15.1 Detailed Description

Provides access to calculated artifacts.

6.15.2 Method Documentation

6.15.2.1 initWithHeadbandOn:blink:jawClench:timestamp:

- (nonnull instancetype) initWithHeadbandOn:
(BOOL) headbandOn
blink:(BOOL) blink
jawClench:(BOOL) jawClench
timestamp:(int64_t) timestamp

6.15.2.2 museArtifactPacketWithHeadbandOn:blink:jawClench:timestamp:

+ (nonnull instancetype) museArtifactPacketWithHeadbandOn:
(BOOL) headbandOn
blink:(BOOL) blink
jawClench:(BOOL) jawClench
timestamp:(int64_t) timestamp

6.15.3 Property Documentation

6.15.3.1 blink

- (BOOL) blink [read], [nonatomic], [assign]

Eye blinking flag.

Returns

true if an eye blink was detected. false otherwise.

6.15.3.2 headbandOn

- (BOOL) headbandOn [read], [nonatomic], [assign]

Flag which represents if the Muse Headband is on a person's head. Headband On detection is one of the first things muse elements algorithm does. If this flag is false you will not see other artifact or band power data.

Returns

true if the headband is on. false otherwise.

6.15.3.3 jawClench

- (BOOL) jawClench [read], [nonatomic], [assign]

Jaw clench flag.

Returns

true if a jaw clench was detected. false otherwise.

6.15.3.4 timestamp

- (int64_t) timestamp [read], [nonatomic], [assign]

Microseconds since epoch (usually Jan 1, 1970).

Returns

the timestamp of the packet expressed in microseconds since epoch.

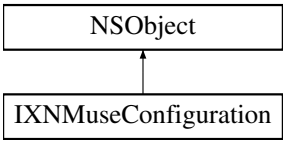
The documentation for this class was generated from the following file:

- IXNMuseArtifactPacket.h

6.16 IXNMuseConfiguration Class Reference

#include <IXNMuseConfiguration.h>

Inheritance diagram for IXNMuseConfiguration:



Instance Methods

- (IXNMusePreset) - getPreset
- (nonnull NSString *) - getHeadbandName
- (nonnull NSString *) - getMicrocontrollerId
- (int32_t) - getEegChannelCount
- (int32_t) - getAfeGain
- (int32_t) - getDownsampleRate
- (int32_t) - getSeroutMode
- (int32_t) - getOutputFrequency
- (int32_t) - getAdcFrequency
- (BOOL) - getNotchFilterEnabled
- (IXNNotchFrequency) - getNotchFilter
- (int32_t) - getAccelerometerSampleFrequency
- (BOOL) - getBatteryDataEnabled
- (BOOL) - getDrlRefEnabled
- (int32_t) - getDrlRefFrequency
- (double) - getBatteryPercentRemaining
- (nonnull NSString *) - getBluetoothMac
- (nonnull NSString *) - getSerialNumber
- (nonnull NSString *) - getHeadsetSerialNumber
- (IXNMuseModel) - getModel
- (nonnull NSString *) - getLicenseNonce
- (int32_t) - getSwitch

6.16.1 Detailed Description

Encapsulates the configuration information of the Muse headband.

You must connect to the headband at least once to before this information is available. Once you have connected once the information will remain available, even after you disconnect.

6.16.2 Method Documentation

6.16.2.1 getAccelerometerSampleFrequency

- (int32_t) getAccelerometerSampleFrequency

Returns accelerometer sample frequency in Hz.

Returns

The frequency of accelerometer samples.

6.16.2.2 getAdcFrequency

- (int32_t) getAdcFrequency

The sample frequency of EEG packets (ADC) in Hz. A value of -1 means "unknown".

Returns

The ADC frequency of the headband.

6.16.2.3 getAfeGain

- (int32_t) getAfeGain

Gain to apply to incoming EEG samples.

Returns

The gain that should be applied to incoming EEG samples.

6.16.2.4 getBatteryDataEnabled

- (BOOL) getBatteryDataEnabled

Provides information about battery data transmission state (on/off).

Returns

true if the battery data transmission is enabled. false if it is not.

6.16.2.5 getBatteryPercentRemaining

- (double) getBatteryPercentRemaining

Returns battery charge remaining in percent.

Returns

The percentage of battery remaining.

6.16.2.6 getBluetoothMac

- (nonnull NSString *) getBluetoothMac

Bluetooth MAC address reported by device.

Returns

The MAC address of the headband.

6.16.2.7 getDownsampleRate

- (int32_t) getDownsampleRate

EEG downsampling rate. A value of -1 means "unknown".

Returns

The EEG downsampling rate.

6.16.2.8 getDrlRefEnabled

- (BOOL) getDrlRefEnabled

Provides information about DRL/REF transmission state (on/off).

Returns

true if the DRL/REF data transmission is enabled. false if it is not.

6.16.2.9 getDrlRefFrequency

- (int32_t) getDrlRefFrequency

DRL/REF data output frequency in Hz. 0 if DRL/REF is not enabled.

Returns

The frequency at which DRL/REF packets are transmitted.

6.16.2.10 getEegChannelCount

- (int32_t) getEegChannelCount

Number of channels currently active.

Returns

The number of EEG channels currently active.

6.16.2.11 getHeadbandName

- (nonnull NSString *) getHeadbandName

The name of the headband. This is only available on Muse 2016 (MU_02) or later.

Returns

The name of the headband.

6.16.2.12 getHeadsetSerialNumber

- (nonnull NSString *) getHeadsetSerialNumber

Headset serial number reported by device. Only applicable to Aster.

Returns

The headset serial number.

6.16.2.13 getLicenseNonce

- (nonnull NSString *) getLicenseNonce

The licensing nonce

Returns

The licensing nonce.

6.16.2.14 getMicrocontrollerId

- (nonnull NSString *) getMicrocontrollerId

Unique 96 bit ID identifying the microcontroller on the headband. This is only available on Muse 2016 (MU_02) or later.

Returns

The id of the microcontroller.

6.16.2.15 getModel

- (IXNMuseModel) getModel

The headband model

Returns

The model of the headband.

6.16.2.16 getNotchFilter

- (IXNNotchFrequency) getNotchFilter

Provides information about notch filter frequency (50 Hz/60 Hz).

Returns

The frequency of the notch filter.

6.16.2.17 getNotchFilterEnabled

- (BOOL) getNotchFilterEnabled

Provides information about notch filter state (on/off).

Returns

true if the notch filter is enabled. false if it is not.

6.16.2.18 getOutputFrequency

- (int32_t) getOutputFrequency

The output frequency of EEG packets in Hz.

Returns

The output frequency of EEG packets.

6.16.2.19 getPreset

- (IXNMusePreset) getPreset

Provides access to current preset.

6.16.2.20 getSerialNumber

- (nonnull NSString *) getSerialNumber

Serial number reported by device.

Returns

The device serial number.

6.16.2.21 getSeroutMode

- (int32_t) getSeroutMode

Serial output mode. The relates to the size of the EEG packets transmitted from the headband. A value of 2 indicates that the packet data is compressed (10 bits). A value of 3 indicates that the packet data is uncompressed (16 bits). A value of -1 means "unknown".

Returns

2 if the data is compressed, 3 if the data is uncompressed or -1 if this value is unknown.

6.16.2.22 getSwitch

- (int32_t) getSwitch

Factory use only

The documentation for this class was generated from the following file:

- IXNMuseConfiguration.h

6.17 <IXNMuseConnectionListener> Protocol Reference

#include <IXNMuseConnectionListener.h>

Instance Methods

- (void) - receiveMuseConnectionPacket:muse:

6.17.1 Detailed Description

Implement this interface to listen for changes to the current connection status (like connect/disconnect).

Important: DO NOT CALL connect (IXNMuse) / disconnect (IXNMuse): directly from a connection listener handler. Either create a new thread and run it from there or use runAsynchronously (IXNMuse) which creates a new thread implicitly.

6.17.2 Method Documentation

6.17.2.1 receiveMuseConnectionPacket:muse:

- (void) receiveMuseConnectionPacket:
(nonnull IXNMuseConnectionPacket *) packet
muse:(nullable IXNMuse *) muse

Called from the Muse connection thread whenever there is a change in the current connection status of the IXNMuse

Warning

It is important that you do not perform any computation intensive tasks in this callback. This would result in significant delays in all the listener callbacks from being called. You should delegate any intensive tasks to another thread or schedule it to run with a delay through handler/scheduler for the platform.

However, you can register/unregister listeners in this callback. All previously registered listeners would still receive callbacks for this current event. On subsequent events, the newly registered listeners will be called. For example, if you had 2 listeners 'A' and 'B' for this event. If, on the callback for listener A, listener A unregisters all listeners and registers a new listener 'C' and then in the callback for listener 'B', you unregister all listeners again and register a new listener 'D'. Then on the subsequent event callback, only listener D's callback will be invoked.

Parameters

packet	The connection packet
muse	The IXNMuse that sent the connection packet.

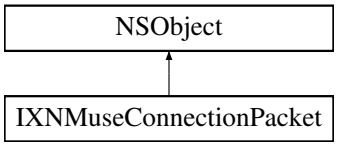
The documentation for this protocol was generated from the following file:

- IXNMuseConnectionListener.h

6.18 IXNMuseConnectionPacket Class Reference

#include <IXNMuseConnectionPacket.h>

Inheritance diagram for IXNMuseConnectionPacket:



Instance Methods

- (nonnull instancetype) - initWithPreviousConnectionState:currentConnectionState:

Class Methods

- (nonnull instancetype) + museConnectionPacketWithPreviousConnectionState:currentConnectionState:

Properties

- IXNConnectionState previousConnectionState
- IXNConnectionState currentConnectionState

6.18.1 Detailed Description

Packet provides information about the connection state.

6.18.2 Method Documentation

6.18.2.1 initWithPreviousConnectionState:currentConnectionState:

```
- (nonnull instancetype) initWithPreviousConnectionState:
                (IXNConnectionState) previousConnectionState
                currentConnectionState:(IXNConnectionState) currentConnectionState
```

6.18.2.2 museConnectionPacketWithPreviousConnectionState:currentConnectionState:

```
+ (nonnull instancetype) museConnectionPacketWithPreviousConnectionState:
                (IXNConnectionState) previousConnectionState
                currentConnectionState:(IXNConnectionState) currentConnectionState
```

6.18.3 Property Documentation

6.18.3.1 currentConnectionState

- (IXNConnectionState) currentConnectionState [read], [nonatomic], [assign]

Provides access to the current connection status.

Returns

The current (new) connection state.

6.18.3.2 previousConnectionState

- (IXNConnectionState) previousConnectionState [read], [nonatomic], [assign]

Provides access to the previous connection status.

Returns

The prior connection state.

The documentation for this class was generated from the following file:

- IXNMuseConnectionPacket.h

6.19 <IXNMuseDataListener> Protocol Reference

#include <IXNMuseDataListener.h>

Instance Methods

- (void) - receiveMuseDataPacket:muse:
- (void) - receiveMuseArtifactPacket:muse:

6.19.1 Detailed Description

Implement this interface to receive data from the headband. The data may be either data packets or artifact packets. Check out IXNMuseDataPacketType for information about all acceptable data packet types. All packets can be handled in one listener or you can specify a separate listener for every specific packet type.

A packet is only sent if a listener was registered to listen for this packet type.

6.19.2 Method Documentation

6.19.2.1 receiveMuseArtifactPacket:muse:

- (void) receiveMuseArtifactPacket:
(nonnull IXNMuseArtifactPacket *) packet
muse:(nullable IXNMuse *) muse

Handler method for Muse artifact packets. Packet is sent only if listener was registered to listen for such packets.

Warning

> It is important that you do not perform any computation intensive tasks in this callback. This would result in significant delays in all the listener callbacks from being called. You should delegate any intensive tasks to another thread or schedule it to run with a delay through handler/scheduler for the platform.

However, you can register/unregister listeners in this callback. All previously registered listeners would still receive callbacks for this current event. On subsequent events, the newly registered listeners will be called. For example, if you had 2 listeners 'A' and 'B' for this event. If, on the callback for listener A, listener A unregisters all listeners and registers a new listener 'C' and then in the callback for listener 'B', you unregister all listeners again and register a new listener 'D'. Then on the subsequent event callback, only listener D's callback will be invoked.

Parameters

packet	The artifact packet
muse	The IXNMuse that sent the artifact packet.

6.19.2.2 receiveMuseDataPacket:muse:

- (void) receiveMuseDataPacket:
(nullable IXNMuseDataPacket *) packet
muse:(nullable IXNMuse *) muse

Handler method for Muse data packets

Warning

It is important that you do not perform any computation intensive tasks in this callback. This would result in significant delays in all the listener callbacks from being called. You should delegate any intensive tasks to another thread or schedule it to run with a delay through handler/scheduler for the platform.

However, you can register/unregister listeners in this callback. All previously registered listeners would still receive callbacks for this current event. On subsequent events, the newly registered listeners will be called. For example, if you had 2 listeners 'A' and 'B' for this event. If, on the callback for listener A, listener A unregisters all listeners and registers a new listener 'C' and then in the callback for listener 'B', you unregister all listeners again and register a new listener 'D'. Then on the subsequent event callback, only listener D's callback will be invoked.

Parameters

packet	The data packet
muse	The IXNMuse that sent the data packet.

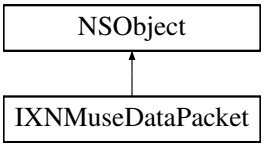
The documentation for this protocol was generated from the following file:

- IXNMuseDataListener.h

6.20 IXNMuseDataPacket Class Reference

#include <IXNMuseDataPacket.h>

Inheritance diagram for IXNMuseDataPacket:



Instance Methods

- (IXNMuseDataPacketType) - packetType
- (int64_t) - timestamp
- (nonnull NSArray< NSNumber * > *) - values
- (int64_t) - valuesSize
- (double) - getEegChannelValue:
- (double) - getPpgChannelValue:
- (double) - getPpgMicroamps:ppgValue:
- (double) - getOpticsChannelValue:
- (double) - getBatteryValue:
- (double) - getAccelerometerValue:
- (double) - getGyroValue:
- (double) - getMagnetometerValue:
- (double) - getDrlRefValue:
- (double) - getPressureValue:
- (double) - getTemperatureValue
- (double) - getUvValue:

Class Methods

- (nullable IXNMuseDataPacket *) + makeUninitializedPacket:
- (nullable IXNMuseDataPacket *) + makePacket:timestamp:values:

6.20.1 Detailed Description

Contains information received from the headband. Each packet can represent different data: eeg, accelerometer, quantization, etc. Take a look at IXNMuseDataPacketType enum for all possible types

6.20.2 Method Documentation

6.20.2.1 getAccelerometerValue:

- (double) getAccelerometerValue:
(IXNAccelerometer) a

Get the IXNAccelerometer value from the packet.

Calling this function does not perform additional allocations and is preferable to using values.

Parameters

a	the IXNAccelerometer value to retrieve (ie. IXNAccelerometerX)

Returns

the value requested.

Exceptions

SIGABRT	If this function is called on a packet type that is not type IXNMuseDataPacketTypeAccelerometer , LibMuse will throw an exception. Use packetType to check the type before calling this function.

6.20.2.2 getBatteryValue:

- (double) getBatteryValue:
(IXNBattery) b

Get the IXNBattery value from the packet.

Calling this function does not perform additional allocations and is preferable to using values.

Parameters

b	the IXNBattery value to retrieve (ie. IXNBatteryMillivolts)

Returns

the value requested.

Exceptions

SIGABRT	If this function is called on a packet type that is not type IXNMuseDataPacketTypeBattery , LibMuse will throw an exception. Use packetType to check the type before calling this function.

6.20.2.3 getDrlRefValue:

- (double) getDrlRefValue:
(IXNDrlRef) drl

Get the IXNDrlRef value from the packet.

Calling this function does not perform additional allocations and is preferable to using values.

Parameters

drl	the IXNDrlRef value to retrieve (ie. IXNDrlRefDrl)

Returns

the value requested.

Exceptions

SIGABRT	If this function is called on a packet type that is not type IXNMuseDataPacketTypeDrlRef , LibMuse will throw an exception. Use packetType to check the type before calling this function.

6.20.2.4 getEegChannelValue:

- (double) getEegChannelValue:
(IXNEeg) channelNum

Get the raw EEG or EEG derived value from the packet. EEG derived value are data that is calculated based on the raw EEG values. Take a look at enum in IXNMuseDataPacketType for values derived from EEG channel. Calling this function does not perform additional allocations and is preferable to using values.

Parameters

channelNum	the IXNEeg channel to retrieve (ie. IxnEegEEG1)

Returns

the value requested.

Exceptions

SIGABRT	If this function is called on a packet type that is not type IXNMuseDataPacketTypeEeg , LibMuse will throw an exception. Use packetType to check the type before calling this function.

6.20.2.5 getGyroValue:

- (double) getGyroValue:
(IXNGyro) g

Get the IXNGyro value from the packet.

Calling this function does not perform additional allocations and is preferable to using values.

Parameters

g	the IXNGyro value to retrieve (ie. IXNGyroX)

Returns

the value requested.

Exceptions

SIGABRT	If this function is called on a packet type that is not type IXNMuseDataPacketTypeGyro , LibMuse will throw an exception. Use packetType to check the type before calling this function.

6.20.2.6 getMagnetometerValue:

- (double) getMagnetometerValue:
(IXNMagnetometer) m

Get the IXNMagnetometer value from the packet.

Calling this function does not perform additional allocations and is preferable to using values.

Parameters

m	the IXNMagnetometer value to retrieve (ie. IXNMagnetometerX)

Returns

the value requested.

Exceptions

SIGABRT	If this function is called on a packet type that is not type IXNMuseDataPacketTypeMagnetometer , LibMuse will throw an exception. Use packetType to check the type before calling this function.

6.20.2.7 getOpticsChannelValue:

- (double) getOpticsChannelValue:
(IXNOptics) channelNum

Get the optics value from the packet. Calling this function does not perform additional allocations and is preferable to using values.

Parameters

channelNum	the IXNOptics channel to retrieve (ie. IXNOpticsOPTICS1)

Returns

the value requested.

Exceptions

SIGABRT	If this function is called on a packet type that is not type IXNMuseDataPacketTypeOptics , LibMuse will throw an exception. Use packetType to check the type before calling this function.

6.20.2.8 getPpgChannelValue:

- (double) getPpgChannelValue:
(IXNPpg) channelNum

Get the PPG value from the packet. Calling this function does not perform additional allocations and is preferable to using values.

Parameters

channelNum	the IXNPpg channel to retrieve (ie. IxnPpgIR)

Returns

the value requested.

Exceptions

SIGABRT	If this function is called on a packet type that is not type IXNMuseDataPacketTypePpg , LibMuse will throw an exception. Use packetType to check the type before calling this function.

6.20.2.9 getPpgMicroamps:ppgValue:

- (double) getPpgMicroamps:
(IXNMuseModel) model
ppgValue:(double) ppgValue

Get PPG microamps from the raw PPG value.

Parameters

model	the Muse model to which the conversion is applied.
ppgValue	the raw PPG channel value.

Returns

the PPG value in microamps.

Exceptions

SIGABRT	

6.20.2.10 getPressureValue:

- (double) getPressureValue:
(IXNPressure) pressure

Get the IXNPressure value from the packet.

Calling this function does not perform additional allocations and is preferable to using values.

Parameters

pressure	the IXNPressure value to retrieve (ie. IXNPressureAveraged)

Returns

the value requested.

Exceptions

SIGABRT	If this function is called on a packet type that is not type IXNMuseDataPacketTypePressure , LibMuse will throw an exception. Use packetType to check the type before calling this function.

6.20.2.11 getTemperatureValue

- (double) getTemperatureValue

Get the temperature value from the packet.

Calling this function does not perform additional allocations and is preferable to using values.

Returns

the value requested.

Exceptions

SIGABRT	If this function is called on a packet type that is not type IXNMuseDataPacketTypeTemperature , LibMuse will throw an exception. Use packetType to check the type before calling this function.

6.20.2.12 getUvValue:

- (double) getUvValue:
(IXNUltraViolet) v

Get the IXNUltraViolet value from the packet.

Calling this function does not perform additional allocations and is preferable to using values.

Parameters

v	the IXNUltraViolet value to retrieve (ie. IXNMagnetometerUVA)

Returns

the value requested.

Exceptions

SIGABRT	If this function is called on a packet type that is not type IXNMuseDataPacketTypeUltraViolet , LibMuse will throw an exception. Use packetType to check the type before calling this function.

6.20.2.13 makePacket:timestamp:values:

+ (nullable IXNMuseDataPacket *) makePacket:
(IXNMuseDataPacketType) type
timestamp:(int64_t) timestamp
values:(nonnull NSArray< NSNumber * > *) values

Create a new packet with the given contents.

Parameters

type	the type of packet to create
timestamp	the timestamp of the packet
values	the data the packet contains.

6.20.2.14 makeUninitializedPacket:

+ (nullable IXNMuseDataPacket *) makeUninitializedPacket:
(int64_t) capacity

Create a new packet with reserved capacity but unspecified contents.

Parameters

capacity	the number of data entries to reserve.

6.20.2.15 packetType

- (IXNMuseDataPacketType) packetType

Specifies what kind of values are stored in the packet.

Returns

the type of packet.

6.20.2.16 timestamp

- (int64_t) timestamp

Microseconds since epoch (usually Jan 1, 1970).

Returns

the timestamp of the packet expressed in microseconds since epoch.

6.20.2.17 values

- (nonnull NSArray< NSNumber * > *) values

Raw packet data as an array. The size of this array and the meaning of its elements depend on the packet type. See the documentation for IXNMuseDataPacketType for details.

Note that this method causes new memory to be allocated for an array of boxed double values. If this impacts your application's performance, you should use get_xxx_value() methods instead.

Deprecated Use IXNMuseDataPacket::getAccelerometerValue , IXNMuseDataPacket::getBatteryValue , IXNMuseDataPacket::getDrlRefValue , IXNMuseDataPacket::getEegChannelValue , IXNMuseDataPacket::getPpgChannelValue , IXNMuseDataPacket::getGyroValue instead.

6.20.2.18 valuesSize

- (int64_t) valuesSize

Get the number of values in this packet.

Returns

the number of data values in this packet.

The documentation for this class was generated from the following file:

- IXNMuseDataPacket.h

6.21 <IXNMuseErrorListener> Protocol Reference

#include <IXNMuseErrorListener.h>

Instance Methods

- (void) - receiveError:muse:

6.21.1 Detailed Description

Listens for Muse Error packets.

6.21.2 Method Documentation

6.21.2.1 receiveError:muse:

- (void) receiveError:
(nonnull IXNError *) error
muse:(nullable IXNMuse *) muse

Handler method for Muse error packets. Packet is sent iff muse receives an exception.

Generally if an error occurred, IXNLogListener::receiveLog will be called with a IXNLogPacket of "error" severity. It is therefore not usually necessary or advisable to print log messages for errors received by this method. Rather, this handler should take actions like alerting users or deciding to end a session.

Warning

> It is important that you do not perform any computation intensive tasks in this callback. This would result in significant delays in all the listener callbacks from being called. You should delegate any intensive tasks to another thread or schedule it to run with a delay through handler/scheduler for the platform.

However, you can register/unregister listeners in this callback. All previously registered listeners would still receive callbacks for this current event. On subsequent events, the newly registered listeners will be called. For example, if you had 2 listeners 'A' and 'B' for this event. If, on the callback for listener A, listener A unregisters all listeners and registers a new listener 'C' and then in the callback for listener 'B', you unregister all listeners again and register a new listener 'D'. Then on the subsequent event callback, only listener D's callback will be invoked.

Parameters

error	The error encountered.
muse	The IXNMuse that generated the error.

The documentation for this protocol was generated from the following file:

- IXNMuseErrorListener.h

6.22 <IXNMuseFile> Protocol Reference

#include <IXNMuseFile.h>

Instance Methods

- (BOOL) - open:
- (BOOL) - write:
- (nonnull NSData *) - read:
- (BOOL) - close:

6.22.1 Detailed Description

A File interface. Normally you will not need to use this class directly as you can get a IXNMuseFileReader or IXNMuseFileWriter using IXNMuseFileFactory .
If you provide your own implementation of this class, you should provide a way to specify the path to the file you want to access in a platform specific way.

Threading: IXNMuseFile is not thread safe by default, but IXNMuseFileWriter provides the necessary synchronization for its usage. If you are using IXNMuseFile outside of that context, make sure the calls to the IXNMuseFile::open , IXNMuseFile::write and IXNMuseFile::close functions are thread-safe.

6.22.2 Method Documentation

6.22.2.1 close:

- (BOOL) close:
(BOOL) forWriting

Closes opened file.
This method is called when IXNMuseFileWriter or IXNMuseFileReader is destroyed, or when close is called on either class.

Parameters

forWriting	true if the file should be closed for writing. false if for reading.

Returns

true if file is closed properly, false otherwise.

6.22.2.2 open:

- (BOOL) open:
(BOOL) forWriting

Opens/creates the file. You can open same file for both writing and reading if you really (!) have to do so. Provide synchronization in this case. This method is called automatically when IXNMuseFileWriter or IXNMuseFileReader is created.

Parameters

forWriting	true if the file should be opened for writing. false if for reading.

Returns

true when file is opened successfully, false otherwise.

6.22.2.3 read:

- (nonnull NSData ∗) read:
(int32_t) length

Reads ”length” bytes from file starting from current pointer position.

Parameters

length	The number of bytes to read.

Returns

A byte array containing the data read. An empty buffer is returned if the file is empty or if the end of file was reached.

6.22.2.4 write:

- (BOOL) write:
(nonnull NSData *) buffer

Writes buffer to file.
This method is called when you flush IXNMuseFileWriter

Parameters

buffer	The data to write.

Returns

true when buffer is written to successfully, false otherwise.

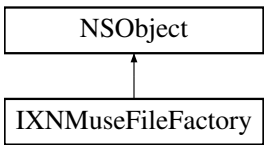
The documentation for this protocol was generated from the following file:

- IXNMuseFile.h

6.23 IXNMuseFileFactory Class Reference

#include <IXNMuseFileFactory.h>

Inheritance diagram for IXNMuseFileFactory:



Class Methods

- (IXNMuseFileWriter *) + museFileWriterWithPathString:
- (IXNMuseFileReader *) + museFileReaderWithPathString:
- (id< IXNMuseFile >) + museFileWithPathString:

6.23.1 Detailed Description

Creates IXNMuseFileWriter, IXNMuseFileReader and IXNMuseFile objects.

6.23.2 Method Documentation

6.23.2.1 museFileReaderWithPathString:

+ (IXNMuseFileReader *) museFileReaderWithPathString:
(NSString *) filePath

Creates and returns IXNMuseFileReader object based on provided path. Interaxon's IXNMuseFile implementation is used in this case.

Parameters

filePath	The path of the file to read.

Returns

IXNMuseFileReader

6.23.2.2 museFileWithPathString:

+ (id< IXNMuseFile >) museFileWithPathString:
(NSString *) filePath

Returns a IXNMuseFile object, which uses Interaxon's implementation

Parameters

filePath	The path of the file.

Returns

IXNMuseFile

6.23.2.3 museFileWriterWithPathString:

+ (IXNMuseFileWriter *) museFileWriterWithPathString:
(NSString *) filePath

Creates and returns IXNMuseFileWriter object based on provided path. Interaxon's IXNMuseFile implementation is used in this case.

Note that upon creation of IXNMuseFileWriter, an Annotation is automatically written out to the file. The annotation contains the app's name and version and libmuse version. If app's name and version can not be determined, they will be empty strings.

Parameters

filePath	The path of the file to write.

Returns

IXNMuseFileWriter

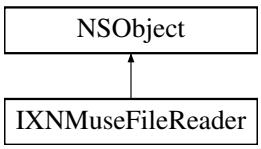
The documentation for this class was generated from the following file:

- IXNMuseFileFactory.h

6.24 IXNMuseFileReader Class Reference

#include <IXNMuseFileReader.h>

Inheritance diagram for IXNMuseFileReader:



Instance Methods

- (BOOL) - open
- (BOOL) - close
- (nonnull IXNResult *) - gotoNextMessage
- (IXNMessageType) - getMessageType
- (int32_t) - getMessageId
- (int64_t) - getMessageTimestamp
- (nonnull IXNAnnotationData *) - getAnnotation
- (nullable IXNMuseConfiguration *) - getConfiguration
- (nullable IXNMuseVersion *) - getVersion
- (nonnull IXNComputingDeviceConfiguration *) - getComputingDeviceConfiguration
- (nonnull IXNDspData *) - getDsp
- (nullable IXNMuseDataPacket *) - getDataPacket
- (nonnull IXNMuseArtifactPacket *) - getArtifactPacket

Class Methods

- (nullable IXNMuseFileReader *) + getFileReader:

6.24.1 Detailed Description

Reads a .muse file formatted according to the Muse protobuf schema.

IXNMuseFileReader is very similar to IXNReaderMuse as both classes allow you to parse a file and generate data packets from it. In most instances, IXNReaderMuse is the better choice at it abstracts the details of the parsing of the file and allows you to handle the data the same way that you would if you were connected to an actual IXNMuse . Use IXNMuseFileReader if you want finer control of the parsing of the file.

When the file reader is created it always starts reading from the beginning of the file to the end.

Threading: It is NOT thread safe, you have to provide your own synchronization mechanism if you plan to calls methods of this class from different threads.

6.24.2 Method Documentation

6.24.2.1 close

- (BOOL) close

Closes the file. Calls IXNMuseFile::close You don't have to call close explicitly unless you want to close file immediately. close is called automatically, when the file reader object is destroyed.

6.24.2.2 getAnnotation

- (nonnull IXNAnnotationData ∗) getAnnotation

Returns annotation data at the current position in the file.

Returns

The annotation data at the current position in the file.

Exceptions

IncorrectMessageType	If current message type is not IXNMessageTypeAnnotation

6.24.2.3 getArtifactPacket

- (nonnull IXNMuseArtifactPacket ∗) getArtifactPacket

Returns the muse artifact packet data at the current position in the file.

Returns

The muse artifact packet at the current position in the file.

Exceptions

IncorrectMessageType	If current message type is not IXNMessageTypeArtifact ,

6.24.2.4 getComputingDeviceConfiguration

- (nonnull IXNComputingDeviceConfiguration *) getComputingDeviceConfiguration

Returns computing device configuration data at the current position in the file.

Returns

The device configuration data at the current position in the file.

Exceptions

IncorrectMessageType	If current message type is not IXNMessageTypeComputingDevice

6.24.2.5 getConfiguration

- (nullable IXNMuseConfiguration *) getConfiguration

Returns muse configuration data at the current position in the file.

Returns

The configuration data at the current position in the file.

Exceptions

IncorrectMessageType	If current message type is not IXNMessageTypeConfiguration

6.24.2.6 getDataPacket

- (nullable IXNMuseDataPacket *) getDataPacket

Returns the muse data packet data at the current position in the file. Use this method to get EEG, Accelerometer, Battery and Quantization packets.

Returns

The muse data packet data at the current position in the file.

Exceptions

IncorrectMessageType	If current message type is not one of: IXNMessageTypeAccelerometer , IXNMessageTypeAccDropped , IXNMessageTypeBattery , IXNMessageTypeEeg , IXNMessageTypeEegDropped , IXNMessageTypeGyro , IXNMessageTypeMuseElements or IXNMessageTypeQuantization

6.24.2.7 getDsp

- (nonnull IXNDspData *) getDsp

Returns the dsp data at the current position in the file.

Returns

The dsp data at the current position in the file.

Exceptions

IncorrectMessageType	If current message type is not IXNMessageTypeDsp

6.24.2.8 getFileReader:

+ (nullable IXNMuseFileReader *) getFileReader:
(nullable id< IXNMuseFile >) file

Returns an instance of file reader. Automatically opens a file.

Returns

a IXNMuseFileReader instance.

6.24.2.9 getMessageId

- (int32_t) getMessageId

Returns the id of the message at the current position in the file or -1 if the id isn't found in the protobuf specification.

Returns

The id of the message at the current position in the file or -1 if the id isn't found in the protobuf specification.

6.24.2.10 getMessageTimestamp

- (int64_t) getMessageTimestamp

Returns the timestamp of the message at the current position in the file.

Returns

The timestamp of the message.

6.24.2.11 getMessageType

- (IXNMessageType) getMessageType

Returns the type of message at the current position in the file.

Returns

The type of message at the current position in the file.

6.24.2.12 getVersion

- (nullable IXNMuseVersion *) getVersion

Returns muse version data at the current position in the file.

Returns

The version data at the current position in the file.

Exceptions

IncorrectMessageType	If current message type is not IXNMessageTypeVersion

6.24.2.13 gotoNextMessage

- (nonnull IXNResult *) gotoNextMessage

Reads the next message in the protobuf stream.

Returns

The IXNResult object with the status of the read operation.

6.24.2.14 open

- (BOOL) open

Opens an existing file Use this method if you explicitly closed file and want to open it again. Calls IXNMuseFile::open

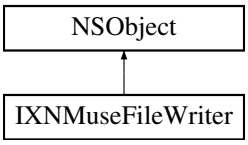
The documentation for this class was generated from the following file:

- IXNMuseFileReader.h

6.25 IXNMuseFileWriter Class Reference

#include <IXNMuseFileWriter.h>

Inheritance diagram for IXNMuseFileWriter:



Instance Methods

- (BOOL) - open
- (BOOL) - isOpen
- (BOOL) - close
- (void) - discardBufferedPackets
- (BOOL) - flush
- (int32_t) - getBufferredMessagesCount
- (int32_t) - getBufferedMessagesSize
- (int64_t) - getTotalBytesWritten
- (void) - addArtifactPacket:packet:
- (void) - addDataPacket:packet:
- (void) - addAnnotationString:annotation:
- (void) - addAnnotation:annotation:
- (void) - addConfiguration:configuration:
- (void) - addVersion:version:
- (void) - addComputingDeviceConfiguration:configuration:
- (void) - addDsp:dsp:
- (void) - setTimestampMode:
- (void) - setTimestamp:

Class Methods

- (nullable IXNMuseFileWriter *) + getFileWriter:

6.25.1 Detailed Description

This class manages saving Muse packets and other data into a file, which can later be read and replayed by MusePlayer. Google Protobuf is used to store the information. For better control, data is not written to the file immediately, but stored in a buffer instead. The buffer then can be flushed to a file or discarded.

Note that upon creation of a IXNMuseFileWriter , an Annotation and the ComputingDeviceConfiguration are automatically written out to the file. The annotation contains the app's name and version and libmuse version. If app's name and version can not be determined, they will be empty strings. The ComputingDeviceConfiguration contains hardware information for the computing device. If any information fields can not be determined, they will be empty strings.

Threading: It is thread safe, so you can call write()/get()/flush()/discard() operations from different threads.

Warning

> If you don't call flush (IXNMuseFileWriter) method, messages will be accumulated in memory and eventually you will get memory overflow error. It's library client responsibility to call flush (IXNMuseFileWriter) or discardBufferedPackets (IXNMuseFileWriter) and to clean the memory. You can use helper methods to get how many messages are currently stored in the buffer and how much memory is used.

6.25.2 Method Documentation

6.25.2.1 addAnnotation:annotation:

```
- (void) addAnnotation:
                (int32_t) id
                annotation:(nonnull IXNAnnotationData *) annotation
```

Adds annotation data structure to the buffer. Similar to IXNMuseFileWriter::addAnnotationString: , but this method allows you to extra fields. Empty fields in annotation struct won't be added to protobuf. If annotation.data field is empty, method returns immediately.

Parameters

id	The id of the device generating the annotation. This is arbitrary and is used to differentiate data from multiple devices in a single file.
annotation	The annotation data to add.

6.25.2.2 addAnnotationString:annotation:

```
- (void) addAnnotationString:
                (int32_t) id
                annotation:(nonnull NSString *) annotation
```

Adds an annotation string to the buffer. It may be useful if you want to log specific events. This is a simplified version of IXNMuseFileWriter::addAnnotation: Use this method if you want to add one string to protobuf. Use IXNMuseFileWriter::addAnnotation: for more advanced options. This function does nothing if annotation is empty.

Parameters

id	The id of the device generating the packet. This is arbitrary and is used to differentiate data from multiple devices in a single file.
annotation	The annotation string to add.

6.25.2.3 addArtifactPacket:packet:

- (void) addArtifactPacket:
(int32_t) id
packet:(nonnull IXNMuseArtifactPacket *) packet

Adds IXNMuseArtifactPacket to the buffer.

Parameters

id	The id of the device generating the packet. This is arbitrary and is used to differentiate data from multiple devices in a single file.
packet	The artifact packet to add.

6.25.2.4 addComputingDeviceConfiguration:configuration:

- (void) addComputingDeviceConfiguration:
(int32_t) id
configuration:(nonnull IXNComputingDeviceConfiguration *) configuration

Adds information about the running device to the buffer.

Parameters

id	The id of the device saving the configuration. This is arbitrary and is used to differentiate data from multiple devices in a single file.
configuration	The device configuration data to add.

6.25.2.5 addConfiguration:configuration:

- (void) addConfiguration:
(int32_t) id
configuration:(nullable IXNMuseConfiguration *) configuration

Adds IXNMuseConfiguration to the buffer.

Parameters

id	The id of the device saving the configuration. This is arbitrary and is used to differentiate data from multiple devices in a single file.
configuration	The configuration data to add.

6.25.2.6 addDataPacket:packet:

- (void) addDataPacket:
(int32_t) id
packet:(nullable IXNMuseDataPacket *) packet

Adds IXNMuseDataPacket to the buffer. All current packets are supported.

Parameters

id	The id of the device generating the packet. This is arbitrary and is used to differentiate data from multiple devices in a single file.
packet	The data packet to add.

6.25.2.7 addDsp:dsp:

- (void) addDsp:
(int32_t) id
dsp:(nonnull IXNDspData *) dsp

With this method you can save your custom data.

Parameters

id	The id of the device saving the data. This is arbitrary and is used to differentiate data from multiple devices in a single file.
dsp	The data to store.

6.25.2.8 addVersion:version:

- (void) addVersion:
(int32_t) id
version:(nullable IXNMuseVersion *) version

Adds IXNMuseVersion to the buffer.

Parameters

id	The id of the device saving the version. This is arbitrary and is used to differentiate data from multiple devices in a single file.
version	The version data to add.

6.25.2.9 close

- (BOOL) close

Closes the file. Calls IXNMuseFile::close

You don't have to call close explicitly unless you want to close file immediately. close is called automatically, when the file writer object is destroyed.

Returns

false if the file could not be closed for any reason. true otherwise.

6.25.2.10 discardBufferedPackets

- (void) discardBufferedPackets

Removes all saved messages from the memory.

6.25.2.11 flush

- (BOOL) flush

Flashes saved messages to the disk. Calls IXNMuseFile::write

Returns

false if can not write to file for any reason. true otherwise.

6.25.2.12 getBufferedMessagesSize

- (int32_t) getBufferedMessagesSize

Returns the size of saved messages in bytes.

Returns

the size of the saved messages in bytes.

6.25.2.13 getBufferredMessagesCount

- (int32_t) getBufferredMessagesCount

Returns number of saved messages

Returns

the number of saved messages.

6.25.2.14 getFileWriter:

+ (nullable IXNMuseFileWriter *) getFileWriter:
(nullable id< IXNMuseFile >) file

Returns an instance of file writer. Automatically opens a file.

Returns

an instance of the file writer.

6.25.2.15 getTotalBytesWritten

- (int64_t) getTotalBytesWritten

Returns the total number of bytes written to the file.

Returns

the total number of bytes written to the file.

6.25.2.16 isOpen

- (BOOL) isOpen

Checks if the file writer is open.

Returns

true when file writer is open, false otherwise.

6.25.2.17 open

- (BOOL) open

Opens a file if it exists or creates a new one. If file already exists, all new messages will be appended at the end. Use this method if you explicitly closed file and want to open it again. Calls IXNMuseFile::open

Returns

false if the file could not be opened for any reason. true otherwise.

6.25.2.18 setTimestamp:

- (void) setTimestamp:
(int64_t) timestamp

Set the timestamp for the next message to be written.

This only does anything in explicit timestamp mode. In explicit mode, the timestamp set by set_↩ timestamp applies to the next message and all subsequent messages. If set_timestamp is not called, then the timestamp is 0.

In all other modes, calling set_timestamp causes an assert failure – the app crashes in debug mode, and it is a noop in release mode.

The timestamp is in microseconds since some epoch (usually Jan 1, 1970).

Parameters

timestamp	The time to use for the timestamp in microseconds.

6.25.2.19 setTimestampMode:

- (void) setTimestampMode:
(IXNTimestampMode) mode

Set the timestamp mode.

By default, the mode is IXNTimestampModeLegacy

Parameters

mode	The timestamp mode to set.

The documentation for this class was generated from the following file:

- IXNMuseFileWriter.h

6.26 <IXNMuseListener> Protocol Reference

#include <IXNMuseListener.h>

Instance Methods

- (void) - museListChanged

6.26.1 Detailed Description

Implement this interface to receive callbacks whenever a Muse headband is discovered after calling startListening (IXNMuseManager-p):

6.26.2 Method Documentation

6.26.2.1 museListChanged

- (void) museListChanged

Called when the list of Muses detected by MuseManager has changed.

- You will receive 1 callback for each headband discovered and added to the list. For Muse 2014 (MU_01) headbands, there will be a callback for each headband paired with the device only when the headband is powered on and connected to the device.
- You will receive 1 callback for each removal operation. If more than 1 headband is removed as the result of a single removal operation, you will only receive 1 callback instead of 1 callback for each headband.

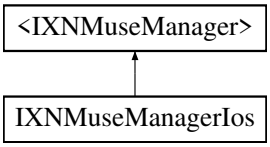
The documentation for this protocol was generated from the following file:

- IXNMuseListener.h

6.27 <IXNMuseManager> Protocol Reference

#include <IXNMuseManager.h>

Inheritance diagram for <IXNMuseManager>:



Instance Methods

- (nonnull NSArray< IXNMuse * > *) - getMuses
- (void) - startListening
- (void) - stopListening
- (void) - setMuseListener:
- (nonnull IXNAdvertisingStats *) - getAdvertisingStats:
- (void) - resetAdvertisingStats
- (void) - removeFromListAfter:

6.27.1 Detailed Description

Discovers the available Muse headbands that this device can connect to.

To connect to a headband, first call IXNMuseManager::setMuseListener to receive callbacks when a headband is discovered. Then call startListening (IXNMuseManager-p): When IXNMuseManager detects a headband you will receive a museListChanged (IXNMuseListener-p): callback. You can then call getMuses (IXNMuseManager-p): followed by the appropriate function on the IXNMuse to connect to the headband.

See also

IXNMuse

IXNMuseListener

6.27.2 Method Documentation

6.27.2.1 getAdvertisingStats:

- (nonnull IXNAdvertisingStats *) getAdvertisingStats:
(nullable IXNMuse *) m

Returns information about the advertising packets seen by LibMuse. This is intended for internal usage at Interaxon and is currently only implemented on Android for Muse 2016 (MU_02) or later. This function does nothing on iOS or Windows.

6.27.2.2 getMuses

- (nonnull NSArray< IXNMuse * > *) getMuses

Returns all currently available IXNMuse headbands.

The returned list is sorted lexicographically by Muse name.

Note that until you start listening, this list will be empty.

Returns

The list of available muses.

6.27.2.3 removeFromListAfter:

- (void) removeFromListAfter:
(int64_t) time

IXNMuseManager will automatically remove a Muse 2016 (MU_02) or later headband from the list of Muses if it has not received some type of communication from the headband within this time period.

By default this is 30 seconds To have Muse 2016 (MU_02) or later headbands remain in the list until the next call to startListening: set the time to 0.

Parameters

time	Specified in seconds, the time to wait before removing a headband from the list. If set to 0, headbands will not be removed automatically.

6.27.2.4 resetAdvertisingStats

- (void) resetAdvertisingStats

Erases all previously stored information about advertising packets. This is intended for internal usage at Interaxon and is currently only implemented Muse 2016 (MU_02) or later. This function does nothing on iOS.

6.27.2.5 setMuseListener:

- (void) setMuseListener:
(nullable id< IXNMuseListener >) listener

Set the listener to be notified on changes to the list of Muses.

Parameters

listener	The listener that will receive the callback when a Muse is discovered.

6.27.2.6 startListening

- (void) startListening

Start listening for any possible Muse devices.

As Muses are discovered, they are added to the list returned by getMuses:

Listening is a fairly heavyweight operation, so it should be disabled when it is not needed, e.g. after a device has been found. Specifically, you must call MuseManager::stopListening: before you try to connect to a Muse you have previously discovered.

MU-01 devices are discovered as they connect to the device, either by re-establishing a connection to the last iOS device they connected to, or by the user connecting to them from the OS's Bluetooth settings. If they go offline, they are removed from the list.

Any already-known Muse 2014 (MU_01) devices will be added to the list immediately; others will trickle in as events are received.

Muse 2016 (MU_02) and later devices are discovered as their discovery events are passed to the library from the OS.

6.27.2.7 stopListening

- (void) stopListening

Stop listening for Muse devices.

Stops receiving advertising packets or modifying the list of Muses returned by getMuses: If this is called while not scanning, it is a no-op.

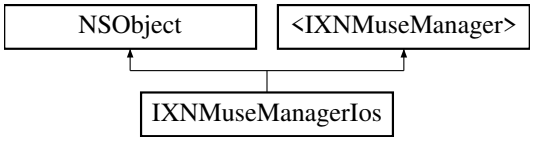
The documentation for this protocol was generated from the following file:

- IXNMuseManager.h

6.28 IXNMuseManagerIos Class Reference

#include <IXNMuseManagerIos.h>

Inheritance diagram for IXNMuseManagerIos:



Instance Methods

- (void) - showMusePickerWithCompletion:

Instance Methods inherited from <IXNMuseManager>

- (nonnull NSArray< IXNMuse * > *) - getMuses
- (void) - startListening
- (void) - stopListening
- (void) - setMuseListener:
- (nonnull IXNAdvertisingStats *) - getAdvertisingStats:
- (void) - resetAdvertisingStats
- (void) - removeFromListAfter:

Class Methods

- (IXNMuseManagerIos *) + sharedManager

Properties

- id< IXNMuseListener > museListener

6.28.1 Detailed Description

Provides access to all IXNMuse devices paired to this device.

6.28.2 Method Documentation

6.28.2.1 sharedManager

+ (IXNMuseManagerIos *) sharedManager

Returns the shared IXNMuseManager.

6.28.2.2 showMusePickerWithCompletion:

- (void) showMusePickerWithCompletion:
(EABluetoothAccessoryPickerCompletion) completion

Displays an alert that allows the user to pair the device with a Muse.

This just calls showBluetoothAccessoryPickerWithNameFilter:completion: with a filter that passes devices starting with "Muse".

Deprecated This will only display Muse 2014 (MU_01) headsets. Instead set a IXNMuseListener with setMuseListener: (IXNMuseManager-p) to receive callbacks when either model of headband is detected. See the sample application for an example of how to update the UI and connect to the headset.

6.28.3 Property Documentation

6.28.3.1 museListener

- (id<IXNMuseListener>) museListener [read], [write], [nonatomic], [assign]

Listener called on changes to the list of available muses.

Redefined readwrite here (the IXNMuseManager protocol only provides a setter.)

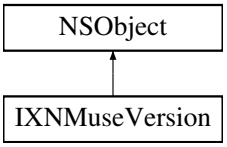
The documentation for this class was generated from the following file:

- IXNMuseManagerIos.h

6.29 IXNMuseVersion Class Reference

#include <IXNMuseVersion.h>

Inheritance diagram for IXNMuseVersion:



Instance Methods

- (nonnull NSString *) - getRunningState
- (nonnull NSString *) - getHardwareVersion
- (nonnull NSString *) - getBspVersion
- (nonnull NSString *) - getFirmwareVersion
- (nonnull NSString *) - getBootloaderVersion
- (nonnull NSString *) - getFirmwareBuildNumber
- (nonnull NSString *) - getFirmwareType
- (int32_t) - getProtocolVersion
- (nonnull NSString *) - getBleFirmwareVersion

Class Methods

- (nullable IXNMuseVersion *) + makeDefaultVersion
- (nullable IXNMuseVersion *) + makeVersion:

6.29.1 Detailed Description

Provides access to Muse firmware and hardware versions.

You must connect to the headband at least once to before this information is available. Once you have connected once the information will remain available, even after you disconnect.

6.29.2 Method Documentation

6.29.2.1 getBleFirmwareVersion

- (nonnull NSString *) getBleFirmwareVersion

Provides access to the Bluetooth firmware version.

Returns

The BLE firmware version.

6.29.2.2 getBootloaderVersion

- (nonnull NSString *) getBootloaderVersion

Provides access to Muse bootloader version.

Returns

The bootloader version.

6.29.2.3 getBspVersion

- (nonnull NSString *) getBspVersion

BSP (board support package) version. This is only available on Muse 2016 (MU_02) or later.

Returns

The BSP version.

6.29.2.4 getFirmwareBuildNumber

- (nonnull NSString *) getFirmwareBuildNumber

Provides access to Muse firmware build number. This is only available on Muse 2014 (MU_01).

Returns

The firmware build number.

6.29.2.5 getFirmwareType

- (nonnull NSString *) getFirmwareType

Type of firmware. One of consumer, research or test.

Returns

The type of firmware.

6.29.2.6 getFirmwareVersion

- (nonnull NSString *) getFirmwareVersion

Provides access to the firmware version.

Returns

The firmware version.

6.29.2.7 getHardwareVersion

- (nonnull NSString *) getHardwareVersion

Provides access to hardware version.

Returns

The hardware version.

6.29.2.8 getProtocolVersion

- (int32_t) getProtocolVersion

Provides access to Muse communication protocol version.

Returns

The communication protocol version.

6.29.2.9 getRunningState

- (nonnull NSString *) getRunningState

Provides access to the running state.

- For Muse 2014 (MU_01) this is one of app, bootloader or test.
- For Muse 2016 (MU_02) or later this is one of headband or bootloader.

  Returns

  The current running state of the headband.

6.29.2.10 makeDefaultVersion

+ (nullable IXNMuseVersion *) makeDefaultVersion

Internal use only. Create a default version.

6.29.2.11 makeVersion:

+ (nullable IXNMuseVersion *) makeVersion:
(nonnull NSString *) json

Internal use only. Create version from JSON string on Muse 2016 (MU_02) or later.

The documentation for this class was generated from the following file:

- IXNMuseVersion.h

6.30 <IXNReaderListener> Protocol Reference

#include <IXNReaderListener.h>

Instance Methods

- (void) - receiveAnnotation:
- (void) - receiveVersion:
- (void) - receiveConfiguration:
- (void) - receiveComputingDeviceConfiguration:

6.30.1 Detailed Description

Implement this interface to receive annotations and configuration/version info from a IXNReaderMuse

6.30.2 Method Documentation

6.30.2.1 receiveAnnotation:

- (void) receiveAnnotation:
(nonnull IXNAnnotationData *) annotation

Called once for each annotation in the file.

Parameters

annotation	The annotation data that was read.

6.30.2.2 receiveComputingDeviceConfiguration:

- (void) receiveComputingDeviceConfiguration:
(nonnull IXNComputingDeviceConfiguration *) computingDeviceConfiguration

Called each time a computing device packet is encountered.

Parameters

computingDeviceConfiguration	The device configuration that was read.

6.30.2.3 receiveConfiguration:

- (void) receiveConfiguration:
(nullable IXNMuseConfiguration *) configuration

Called each time a configuration packet is encountered.

Parameters

configuration	The configuration data that was read.

6.30.2.4 receiveVersion:

- (void) receiveVersion:
(nullable IXNMuseVersion *) version

Called each time a version packet is encountered.

Parameters

version	The version data that was read.

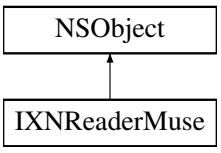
The documentation for this protocol was generated from the following file:

- IXNReaderListener.h

6.31 IXNReaderMuse Class Reference

#include <IXNReaderMuse.h>

Inheritance diagram for IXNReaderMuse:



Instance Methods

- (void) - run
- (void) - runInRealTimespan
- (int64_t) - currentTime
- (void) - playback
- (void) - stopPlayback
- (void) - setPlaybackSettings:
- (IXNReaderMusePlaybackSettings) - getPlaybackSettings
- (void) - setReaderListener:
- (void) - setPlaybackListener:
- (nullable IXNMuse ∗) - asMuse

6.31.1 Detailed Description

A IXNMuse associated with a IXNMuseFileReader

This interface can be used to play back packets from a .muse file so that they can be handled the same way that you would handle data from an actual IXNMuse . Details of the parsing are abstracted and you will simply receive data packets with the data that is read.

When you use this class, register IXNMuseDataListener for the data packets you would like to receive by calling asMuse (IXNReaderMuse): then registerDataListener:type: (IXNMuse)

See also

IXNReaderMuseBuilder

IXNReaderMusePlaybackSettings

IXNReaderPlaybackListener

6.31.2 Method Documentation

6.31.2.1 asMuse

- (nullable IXNMuse ∗) asMuse

Downcast to IXNMuse

Note that most of the API of the returned Muse is stubbed and will cause assert failures if called in debug mode. The methods that are implemented are:

- isLowEnergy / is_low_energy
- enableDataTransmission / enable_data_transmission
- register∗
- unregister∗

Note that the only listener that will ever receive packets from a IXNReaderMuse is IXNMuseDataListener

6.31.2.2 currentTime

- (int64_t) currentTime

Corresponds to the timestamp field for the last packet read.

This does not change within a given listener's body.

Returns

The timestamp of the last packet read.

6.31.2.3 getPlaybackSettings

- (IXNReaderMusePlaybackSettings) getPlaybackSettings

Returns the settings that will be used when playing back a file with this IXNReaderMuse

Returns

The settings that will be used to play back the file.

6.31.2.4 playback

- (void) playback

Play back the IXNMuseFile and call each listener for each packet as appropriate.

The speed of the playback is controlled by the playback settings. Simulated playback settings require the IXNReaderMuse to be constructed with an IXNEventLoop to simulate the time between packets. Calling playback without an IXNEventLoop will not playback the file and will log a warning.

This only works once; the file is not rewound on subsequent calls. If you want to play a file again, create another IXNReaderMuse

See also

IXNReaderMusePlaybackSettings

6.31.2.5 run

- (void) run

Run through all packets in the file, calling each listener for each packet as appropriate.

This only works once; the file is not rewound on subsequent calls. If you want to play a file again, create another IXNReaderMuse

Deprecated Use playback: with the playback setting IXNReaderMusePlaybackSettingsAsFastAsPossibleWithSavedTimes
to replicate this behaviour.

6.31.2.6 runInRealTimespan

- (void) runInRealTimespan

Run through all packets in the file, calling each listener for each packet as appropriate in 1:1 timespan

This only works once; the file is not rewound on subsequent calls. If you want to play a file again, create another IXNReaderMuse

Deprecated Use playback: with the playback setting IXNReaderMusePlaybackSettingsSimulatedWithSavedTimestamp to replicate this behaviour.

6.31.2.7 setPlaybackListener:

- (void) setPlaybackListener:
(nullable id< IXNReaderPlaybackListener >) listener

Set a listener to receive events related to playback. For example when playback has finished successfully or was interrupted due to error.

Parameters

listener	The playback listener to receive callbacks.

6.31.2.8 setPlaybackSettings:

- (void) setPlaybackSettings:
(IXNReaderMusePlaybackSettings) settings

Sets the settings to use when playing back a file with this IXNReaderMuse

Parameters

settings	The settings to use for playback.

6.31.2.9 setReaderListener:

- (void) setReaderListener:
(nullable id< IXNReaderListener >) listener

Set a listener to receive annotations, version, and configuration packets.

Parameters

listener	The listener for annotation, version and configuration packets.

6.31.2.10 stopPlayback

- (void) stopPlayback

Stop playback of the IXNMuseFile

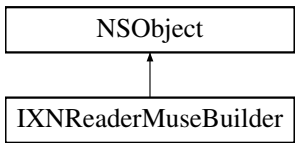
The documentation for this class was generated from the following file:

- IXNReaderMuse.h

6.32 IXNReaderMuseBuilder Class Reference

#include <IXNReaderMuseBuilder.h>

Inheritance diagram for IXNReaderMuseBuilder:



Instance Methods

- (nullable IXNReaderMuseBuilder *) - withPacketTypes:
- (nullable IXNReaderMuseBuilder *) - skipPacketTypes:
- (nullable IXNReaderMuseBuilder *) - withModel:
- (nullable IXNReaderMuseBuilder *) - withPlaybackSettings:
- (nullable IXNReaderMuseBuilder *) - withEventLoop:
- (nullable IXNReaderMuse *) - build:
- (nullable IXNReaderMuse *) - buildWithAsync:asyncLoop:

Class Methods

- (nullable IXNReaderMuseBuilder *) + get

6.32.1 Detailed Description

Builds a IXNReaderMuse that plays back the packets in the given IXNMuseFileReader It exposes a way to set the configuration with chainable method calls, e.g.:

```
IXNReaderMuse *muse = [[[ReaderMuseBuilder get]
    withPacketTypes: myPacketTypes]
    build: myMuseFileReader];
```

6.32.2 Method Documentation

6.32.2.1 build:

- (nullable IXNReaderMuse *) build:
(nullable IXNMuseFileReader *) reader

Construct a IXNReaderMuse

Parameters

reader	The IXNMuseFileReader to use to read the file.

Returns

A IXNReaderMuse configured with the parameters that were set.

6.32.2.2 buildWithAsync:asyncLoop:

- (nullable IXNReaderMuse *) buildWithAsync:
(nullable IXNMuseFileReader *) reader
asyncLoop:(nullable id< IXNEventLoop >) asyncLoop

Construct a IXNReaderMuse

Parameters

reader	The IXNMuseFileReader to use to read the file.
asyncLoop	The IXNEventLoop to use for the call to runAsynchronously (IXNMuse):

Returns

A IXNReaderMuse configured with the parameters that were set.

Deprecated Set the IXNEventLoop with IXNReaderMuseBuilder::withEventLoop and then call IXNReaderMuseBuilder::build instead.

6.32.2.3 get

+ (nullable IXNReaderMuseBuilder *) get

Returns a reference to a IXNReaderMuseBuilder

Returns

A reference to a IXNReaderMuseBuilder

6.32.2.4 skipPacketTypes:

- (nullable IXNReaderMuseBuilder *) skipPacketTypes:
(nonnull NSSet< NSNumber * > *) types

The set of packet types to explicitly ignore in the file.

The default is the empty set.

The eventual set used is computed by the last call to withPacketTypes: minus the last call to skipPacketTypes:

Parameters

types	The set of packet types to skip.

Returns

A reference to the same IXNReaderMuseBuilder

6.32.2.5 withEventLoop:

- (nullable IXNReaderMuseBuilder *) withEventLoop:
(nullable id< IXNEventLoop >) loop

The IXNEventLoop to use to handle simulated playback.

The default is a null pointer (no event loop).

Parameters

loop	The IXNEventLoop to use.

Returns

A reference to the same IXNReaderMuseBuilder

6.32.2.6 withModel:

- (nullable IXNReaderMuseBuilder *) withModel:
(IXNMuseModel) model

The model that this Muse should say it is.

The default is Muse 2014 (MU_01).

Parameters

model	The model to use.

Returns

A reference to the same IXNReaderMuseBuilder

6.32.2.7 withPacketTypes:

- (nullable IXNReaderMuseBuilder *) withPacketTypes:
(nonnull NSSet< NSNumber * > *) types

The set of packet types to pass through from the file.

The default set contains the following all packet types in IXNMuseDataPacketType

The eventual set used is computed by the last call to withPacketTypes: minus the last call to skipPacketTypes:

Parameters

types	The set of packet types to read.

Returns

A reference to the same IXNReaderMuseBuilder

6.32.2.8 withPlaybackSettings:

- (nullable IXNReaderMuseBuilder *) withPlaybackSettings:
(IXNReaderMusePlaybackSettings) settings

The playback settings to use with playing back the file.

The default is IXNReaderMusePlaybackSettingsAsFastAsPossibleWithSavedTimestamp

Parameters

settings	The playback settings to use.

Returns

A reference to the same IXNReaderMuseBuilder

The documentation for this class was generated from the following file:

- IXNReaderMuseBuilder.h

6.33 <IXNReaderPlaybackListener> Protocol Reference

#include <IXNReaderPlaybackListener.h>

Instance Methods

- (void) - receivePlaybackDone
- (void) - receivePlaybackInterrupted

6.33.1 Detailed Description

Listens for IXNReaderMuse events related to the playback of the file.

6.33.2 Method Documentation

6.33.2.1 receivePlaybackDone

- (void) receivePlaybackDone

Called when playback of muse file has finished (i.e., reach EOF).

6.33.2.2 receivePlaybackInterrupted

- (void) receivePlaybackInterrupted

Called when playback was interrupted/error occurred when reading the file.

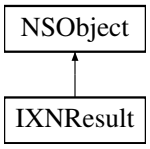
The documentation for this protocol was generated from the following file:

- IXNReaderPlaybackListener.h

6.34 IXNResult Class Reference

#include <IXNResult.h>

Inheritance diagram for IXNResult:



Instance Methods

- (nonnull instancetype) - initWithLevel:type:code:info:

Class Methods

- (nonnull instancetype) + resultWithLevel:type:code:info:

Properties

- IXNResultLevel level
- NSString * type
- int32_t code
- NSString * info

6.34.1 Detailed Description

Represents the result of a synchronous operation.

Results can be at different levels: eg, success, info, warning, error

6.34.2 Method Documentation

6.34.2.1 initWithLevel:type:code:info:

```
- (nonnull instancetype) initWithLevel:
                    (IXNResultLevel) level
                    type:(nonnull NSString *) type
                    code:(int32_t) code
                    info:(nonnull NSString *) info
```

6.34.2.2 resultWithLevel:type:code:info:

```
+ (nonnull instancetype) resultWithLevel:
                    (IXNResultLevel) level
                    type:(nonnull NSString *) type
                    code:(int32_t) code
                    info:(nonnull NSString *) info
```

6.34.3 Property Documentation

6.34.3.1 code

- (int32_t) code [read], [nonatomic], [assign]

A machine-parseable error code

Returns

0 if the result is a success, non-zero otherwise.

6.34.3.2 info

- (NSString*) info [read], [nonatomic], [assign]

Human-readable description

Returns

A verbose description of the result (error message or status)

6.34.3.3 level

- (IXNResultLevel) level [read], [nonatomic], [assign]

The level of the result.

Returns

The IXNResultLevel

6.34.3.4 type

- (NSString*) type [read], [nonatomic], [assign]

The type of operation that this result applies to as a string.

Returns

The type of this result.

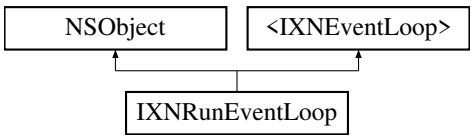
The documentation for this class was generated from the following file:

- IXNResult.h

6.35 IXNRunEventLoop Class Reference

#include <IXNRunEventLoop.h>

Inheritance diagram for IXNRunEventLoop:



Instance Methods

- (instancetype) - initWithRunLoop:

Instance Methods inherited from <IXNEventLoop>

- (void) - post:
- (void) - postDelayed:delayMilliseconds:
- (void) - cancel

6.35.1 Detailed Description

An IXNEventLoop implementation using NSRunLoop

6.35.2 Method Documentation

6.35.2.1 initWithRunLoop:

- (instancetype) initWithRunLoop:
(NSRunLoop *) runLoop

Initializes the object with the passed NSRunLoop.

Parameters

runLoop	The NSRunLoop to use for the IXNEventLoop.

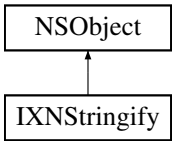
The documentation for this class was generated from the following file:

- IXNRunEventLoop.h

6.36 IXNStringify Class Reference

#include <IXNStringify.h>

Inheritance diagram for IXNStringify:



Instance Methods

- (nonnull NSString *) - packetType:
- (nonnull NSString *) - connectionState:

Class Methods

- (nullable IXNStringify *) + instance

6.36.1 Detailed Description

Converts some LibMuse constants to strings.

This is primarily used for LibMuse internal logging purposes. Don't use this to generate UI elements; the results are not guaranteed to be human-friendly, nor localized. You may, however, use this for your own logging or debugging purposes.

6.36.2 Method Documentation

6.36.2.1 connectionState:

- (nonnull NSString *) connectionState:
(IXNConnectionState) state

String connection state.

Parameters

state	The connection state.

Returns

The string representation of the connection state.

6.36.2.2 instance

+ (nullable IXNStringify *) instance

Returns the singleton instance.

Returns

the singleton instance.

6.36.2.3 packetType:

- (nonnull NSString *) packetType:
(IXNMuseDataPacketType) type

String packet type.

Only the primitive packet types (eeg, accelerometer, gyro, battery, drl_ref) are human-readable. The rest show up as "type_<n>" where n is the integer corresponding to the constant.

Parameters

type	The packet type.

Returns

The string representation of the packet type.

The documentation for this class was generated from the following file:

- IXNStringify.h

Chapter 7

File Documentation

7.1 mainpage.dox File Reference

7.2 IXNAccelerometer.h File Reference

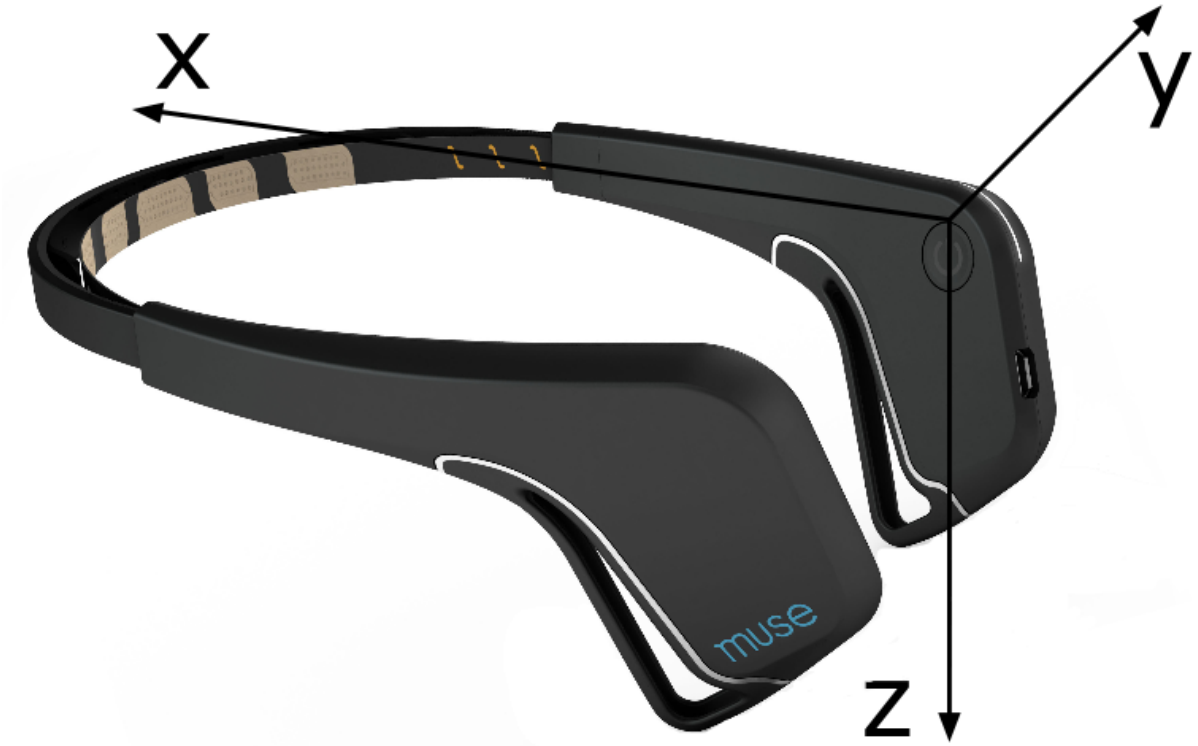
Enumerations

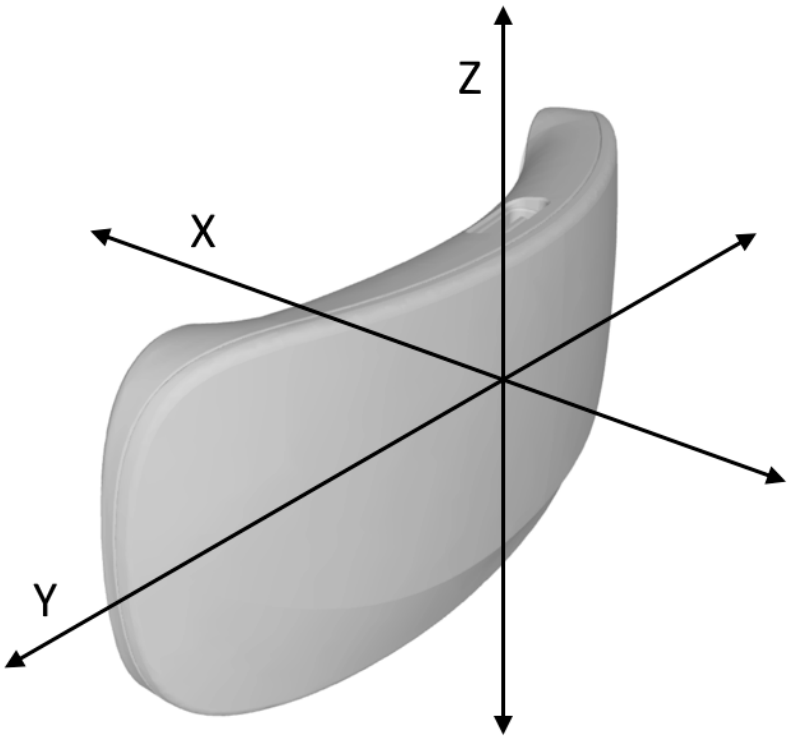
- enum IXNAccelerometer {
  IXNAccelerometerX ,
  IXNAccelerometerY ,
  IXNAccelerometerZ }

7.2.1 Detailed Description

Represents the data mapping in an Accelerometer data packet.

The accelerometer data is measured on 3 axes as shown in the pictures below:





The axes are oriented to a Right Hand Coordinate System along the headband axes. Values are given in g (9.81 m/s^2) and are negated to align with the orientation of the headband in Earth's gravitational

field. This convention is described in more detail in the following application note.
https://www.nxp.com/docs/en/application-note/AN3461.pdf
Values along the X axis increase as the head tilts down aligning the X axis with the downward force of gravity. Negative values indicate a tilt upwards.
Values along the Y axis increase as the head tilts to the right. Negative values indicate a tilt to the left.
When worn on a level head, or held in the level position shown in the figure above, the net acceleration of the device will only be caused from gravity. It will be in the direction of the ground aligned with the Z axis. This will give a_x =0, a_y = 0 and a_z = 1 g. As the headband tilts out of this orientation, the value of Z will decrease. -1 in Z represents a headband that is upside down.

See also

IXNMuseDataPacketTypeAccelerometer
IXNMuseDataPacket::getAccelerometerValue

7.2.2 Enumeration Type Documentation

7.2.2.1 IXNAccelerometer

enum IXNAccelerometer

Enumerator

IXNAccelerometerX	Orientation of the X axis relative to gravity in g. Values along the X axis increase as the head tilts down. Negative values indicate a tilt up.
IXNAccelerometerY	Orientation of the Y axis relative to gravity in g. Values along the Y axis increase as the head tilts to the right. Negative values indicate a tilt to the left.
IXNAccelerometerZ	Orientation of the Z axis relative to gravity in g.

7.3 IXNAccelerometer.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from packets.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00037 typedef NS_ENUM(NSInteger, IXNAccelerometer)
00038 {
00044     IXNAccelerometerX,
00050     IXNAccelerometerY,
00052     IXNAccelerometerZ,
00053 };
```

7.4 IXNAction.h File Reference

Classes

- protocol <IXNAction>

7.5 IXNAction.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from async.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00006
00011 @protocol IXNAction
00012
00014 - (void)run;
00015
00016 @end
```

7.6 IXNAdvertisingStats.h File Reference

Classes

- class IXNAdvertisingStats

7.7 IXNAdvertisingStats.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from sdk_bridge.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00012 @interface IXNAdvertisingStats : NSObject
00013 - (nonnull instancetype)initWithNumAdvertisingPackets:(int32_t)numAdvertisingPackets
00014                                 avgAdvertisingInterval:(double)avgAdvertisingInterval
00015                               sigmaAdvertisingInterval:(double)sigmaAdvertisingInterval
00016                                 maxAdvertisingInterval:(double)maxAdvertisingInterval
00017                                             isLost:(BOOL)isLost
00018                                          hasBadMac:(BOOL)hasBadMac;
00019 + (nonnull instancetype)advertisingStatsWithNumAdvertisingPackets:(int32_t)numAdvertisingPackets
00020                                           avgAdvertisingInterval:(double)avgAdvertisingInterval
00021                                         sigmaAdvertisingInterval:(double)sigmaAdvertisingInterval
00022                                           maxAdvertisingInterval:(double)maxAdvertisingInterval
00023                                                       isLost:(BOOL)isLost
00024                                                    hasBadMac:(BOOL)hasBadMac;
00025
00030 @property (nonatomic, readonly) int32_t numAdvertisingPackets;
00031
00038 @property (nonatomic, readonly) double avgAdvertisingInterval;
00039
00044 @property (nonatomic, readonly) double sigmaAdvertisingInterval;
00045
00050 @property (nonatomic, readonly) double maxAdvertisingInterval;
00051
00060 @property (nonatomic, readonly) BOOL isLost;
00061
00067 @property (nonatomic, readonly) BOOL hasBadMac;
00068
00069 @end
```

7.8 IXNAnnotationData.h File Reference

Classes

- class IXNAnnotationData

7.9 IXNAnnotationData.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from muse_file.djinni
00003
00004 #import "Muse/api/IXNAnnotationFormat.h"
00005 #import <Foundation/Foundation.h>
00006
00008 @interface IXNAnnotationData : NSObject
00009 - (nonnull instancetype)initWithData:(nonnull NSString *)data
00010                               format:(IXNAnnotationFormat)format
00011                            eventType:(nonnull NSString *)eventType
00012                              eventId:(nonnull NSString *)eventId
00013                             parentId:(nonnull NSString *)parentId;
00014 + (nonnull instancetype)annotationDataWithData:(nonnull NSString *)data
00015                                         format:(IXNAnnotationFormat)format
00016                                      eventType:(nonnull NSString *)eventType
00017                                        eventId:(nonnull NSString *)eventId
00018                                       parentId:(nonnull NSString *)parentId;
00019
00024 @property (nonatomic, readonly, nonnull) NSString * data;
00025
00027 @property (nonatomic, readonly) IXNAnnotationFormat format;
00028
00033 @property (nonatomic, readonly, nonnull) NSString * eventType;
00034
00039 @property (nonatomic, readonly, nonnull) NSString * eventId;
00040
00045 @property (nonatomic, readonly, nonnull) NSString * parentId;
00046
00047 @end
```

7.10 IXNAnnotationFormat.h File Reference

Enumerations

- enum IXNAnnotationFormat {
  IXNAnnotationFormatPlainString ,
  IXNAnnotationFormatJson ,
  IXNAnnotationFormatOsc }

7.10.1 Detailed Description

Represents all possible data types in the annotation data. These fields correspond to our Muse protobuf schema.

7.10.2 Enumeration Type Documentation

7.10.2.1 IXNAnnotationFormat

enum IXNAnnotationFormat

Enumerator

IXNAnnotationFormatPlainString	The data is a string with no inherrent format.
IXNAnnotationFormatJson	The data is a string of JSON name, value pairs.
IXNAnnotationFormatOsc	The data is formatted as OSC (open sound control) data.

7.11 IXNAnnotationFormat.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from muse_file.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00013 typedef NS_ENUM(NSInteger, IXNAnnotationFormat)
00014 {
00016     IXNAnnotationFormatPlainString,
00018     IXNAnnotationFormatJson,
00020     IXNAnnotationFormatOsc,
00021 };
```

7.12 IXNApiVersion.h File Reference

Classes

- class IXNApiVersion

7.13 IXNApiVersion.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from libmuse_version.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00006
00012 @interface IXNApiVersion : NSObject
00013
00024 - (int64_t)getMonotonic;
00025
00032 - (int64_t)getMajor;
00033
00041 - (int64_t)getMinor;
00042
00049 - (int64_t)getPatch;
00050
00058 - (int64_t)getApi;
00059
00070 - (nonnull NSString *)getString;
00071
00072 @end
```

7.14 IXNBattery.h File Reference

Enumerations

- enum IXNBattery {
  IXNBatteryChargePercentageRemaining ,
  IXNBatteryMillivolts ,
  IXNBatteryTemperatureCelsius ,
  IXNBatteryAverageCurrent ,
  IXNBatteryTimeToEmpty ,
  IXNBatteryTimeToFull ,
  IXNBatteryBatteryCapacity ,
  IXNBatteryRemainingCapacity ,
  IXNBatteryBatteryAge ,
  IXNBatteryTotalCycles }

7.14.1 Detailed Description

Represents the data mapping in a Battery packet.

See also

IXNMuseDataPacketTypeBattery

IXNMuseDataPacket::getBatteryValue

7.14.2 Enumeration Type Documentation

7.14.2.1 IXNBattery

enum IXNBattery

Enumerator

IXNBatteryChargePercentageRemaining	Charge percentage remaining of battery.
IXNBatteryMillivolts	Millivolts of battery from the view of the fuel gauge.
IXNBatteryTemperatureCelsius	Temperature in degrees Celsius.
IXNBatteryAverageCurrent	Average current in microamps.
IXNBatteryTimeToEmpty	Remaining seconds until charge depleted (while discharging).
IXNBatteryTimeToFull	Remaining seconds until fully charged (while charging).
IXNBatteryBatteryCapacity	Full capacity of battery in mAh.
IXNBatteryRemainingCapacity	Remaining capacity of battery in mAh.
IXNBatteryBatteryAge	Percentage battery age calculated from capacity.
IXNBatteryTotalCycles	Total number of charge/discharge cycles.

7.15 IXNBattery.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from packets.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00014 typedef NS_ENUM(NSInteger, IXNBattery)
00015 {
00017     IXNBatteryChargePercentageRemaining,
00019     IXNBatteryMillivolts,
00021     IXNBatteryTemperatureCelsius,
00023     IXNBatteryAverageCurrent,
00025     IXNBatteryTimeToEmpty,
00027     IXNBatteryTimeToFull,
00029     IXNBatteryBatteryCapacity,
00031     IXNBatteryRemainingCapacity,
00033     IXNBatteryBatteryAge,
00035     IXNBatteryTotalCycles,
00036 };
```

7.16 IXNComputingDeviceConfiguration.h File Reference

Classes

- class IXNComputingDeviceConfiguration

7.17 IXNComputingDeviceConfiguration.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from muse_file.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00013 @interface IXNComputingDeviceConfiguration : NSObject
00014 - (nonnull instancetype)initWithOsType:(nonnull NSString *)osType
00015                                osVersion:(nonnull NSString *)osVersion
00016                        hardwareModelName:(nonnull NSString *)hardwareModelName
00017                          hardwareModelId:(nonnull NSString *)hardwareModelId
00018                            processorName:(nonnull NSString *)processorName
00019                           processorSpeed:(nonnull NSString *)processorSpeed
00020                        numberOfProcessors:(int32_t)numberOfProcessors
00021                                memorySize:(nonnull NSString *)memorySize
00022                          bluetoothVersion:(nonnull NSString *)bluetoothVersion
00023                                  timeZone:(nonnull NSString *)timeZone
00024                     timeZoneOffsetSeconds:(int32_t)timeZoneOffsetSeconds;
00025 + (nonnull instancetype)computingDeviceConfigurationWithOsType:(nonnull NSString *)osType
00026                                                        osVersion:(nonnull NSString *)osVersion
00027                                                hardwareModelName:(nonnull NSString *)hardwareModelName
00028                                                  hardwareModelId:(nonnull NSString *)hardwareModelId
00029                                                    processorName:(nonnull NSString *)processorName
00030                                                   processorSpeed:(nonnull NSString *)processorSpeed
00031                                               numberOfProcessors:(int32_t)numberOfProcessors
00032                                                       memorySize:(nonnull NSString *)memorySize
00033                                                 bluetoothVersion:(nonnull NSString *)bluetoothVersion
00034                                                         timeZone:(nonnull NSString *)timeZone
00035                                            timeZoneOffsetSeconds:(int32_t)timeZoneOffsetSeconds;
00036
00038 @property (nonatomic, readonly, nonnull) NSString * osType;
00039
00041 @property (nonatomic, readonly, nonnull) NSString * osVersion;
00042
00044 @property (nonatomic, readonly, nonnull) NSString * hardwareModelName;
00045
00047 @property (nonatomic, readonly, nonnull) NSString * hardwareModelId;
00048
00050 @property (nonatomic, readonly, nonnull) NSString * processorName;
00051
00053 @property (nonatomic, readonly, nonnull) NSString * processorSpeed;
00054
00056 @property (nonatomic, readonly) int32_t numberOfProcessors;
00057
00059 @property (nonatomic, readonly, nonnull) NSString * memorySize;
00060
00062 @property (nonatomic, readonly, nonnull) NSString * bluetoothVersion;
00063
00065 @property (nonatomic, readonly, nonnull) NSString * timeZone;
00066
00068 @property (nonatomic, readonly) int32_t timeZoneOffsetSeconds;
00069
00070 @end
```

7.18 IXNConnectionState.h File Reference

Enumerations

- enum IXNConnectionState {
  IXNConnectionStateUnknown ,
  IXNConnectionStateConnected ,
  IXNConnectionStateConnecting ,
  IXNConnectionStateDisconnected ,
  IXNConnectionStateNeedsUpdate ,
  IXNConnectionStateNeedsLicense }

7.18.1 Detailed Description

Lists all possible connection states

7.18.2 Enumeration Type Documentation

7.18.2.1 IXNConnectionState

enum IXNConnectionState

Enumerator

IXNConnectionStateUnknown	Initial state
IXNConnectionStateConnected	This state is set if the connection was correctly established.
IXNConnectionStateConnecting	This state is set while trying to establish a connection.
IXNConnectionStateDisconnected	This state is set in case of an unsuccessful connection operation or after execution of disconnect method.
IXNConnectionStateNeedsUpdate	This state is set when the connection succeeded but the headband's firmware is out of date – if this occurs, you should instruct your users to use the Muse app to upgrade their firmware.
IXNConnectionStateNeedsLicense	This state is set when the connection succeeded but the headband's license is invalid.

7.19 IXNConnectionState.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from sdk_bridge.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00012 typedef NS_ENUM(NSInteger, IXNConnectionState)
00013 {
00015     IXNConnectionStateUnknown,
00017     IXNConnectionStateConnected,
00019     IXNConnectionStateConnecting,
00024     IXNConnectionStateDisconnected,
00030     IXNConnectionStateNeedsUpdate,
00035     IXNConnectionStateNeedsLicense,
00036 };
```

7.20 IXNDrlRef.h File Reference

Enumerations

- enum IXNDrlRef {
  IXNDrlRefDrl ,
  IXNDrlRefRef }

7.20.1 Detailed Description

Represents the data mapping in a DRL_REF packet.

See also

IXNMuseDataPacketTypeDrlRef

IXNMuseDataPacket::getDrlRefValue

7.20.2 Enumeration Type Documentation

7.20.2.1 IXNDrlRef

enum IXNDrlRef

Enumerator

IXNDrlRefDrl	DRL sensor
IXNDrlRefRef	REF sensor

7.21 IXNDrlRef.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from packets.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00014 typedef NS_ENUM(NSInteger, IXNDrlRef)
00015 {
00017     IXNDrlRefDrl,
00019     IXNDrlRefRef,
00020 };
```

7.22 IXNDspData.h File Reference

Classes

- class IXNDspData

7.23 IXNDspData.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from muse_file.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00010 @interface IXNDspData : NSObject
00011 - (nonnull instancetype)initWithType:(nonnull NSString *)type
00012                            floatArray:(nonnull NSArray<NSNumber *> *)floatArray
00013                              intArray:(nonnull NSArray<NSNumber *> *)intArray
00014                               version:(nonnull NSString *)version;
00015 + (nonnull instancetype)dspDataWithType:(nonnull NSString *)type
00016                               floatArray:(nonnull NSArray<NSNumber *> *)floatArray
00017                                 intArray:(nonnull NSArray<NSNumber *> *)intArray
00018                                  version:(nonnull NSString *)version;
00019
00021 @property (nonatomic, readonly, nonnull) NSString * type;
00022
00024 @property (nonatomic, readonly, nonnull) NSArray<NSNumber *> * floatArray;
00025
00027 @property (nonatomic, readonly, nonnull) NSArray<NSNumber *> * intArray;
00028
00030 @property (nonatomic, readonly, nonnull) NSString * version;
00031
00032 @end
```

7.24 IXNEeg.h File Reference

Enumerations

- enum IXNEeg {
  IXNEegEEG1 ,
  IXNEegEEG2 ,
  IXNEegEEG3 ,
  IXNEegEEG4 ,
  IXNEegAUXLEFT ,
  IXNEegAUXRIGHT ,
  IXNEegAUX1 ,
  IXNEegAUX2 ,
  IXNEegAUX3 ,
  IXNEegAUX4 }

7.24.1 Detailed Description

Represents the data mapping in an EEG packet. Enum values correspond to head locations.

Raw EEG values are given in microvolts. EEG derived values may have different units as defined in IXNMuseDataPacketType

See also

IXNMuseDataPacketTypeEeg

IXNMuseDataPacket::getEegChannelValue

7.24.2 Enumeration Type Documentation

7.24.2.1 IXNEeg

enum IXNEeg

Enumerator

IXNEegEEG1	Left ear
IXNEegEEG2	Left forehead
IXNEegEEG3	Right forehead
IXNEegEEG4	Right ear
IXNEegAUXLEFT	Left auxiliary (maps to AUX1 on MuseS 2025 (MS_03)).
IXNEegAUXRIGHT	Right auxiliary (maps to AUX2 on MuseS 2025 (MS_03)).
IXNEegAUX1	Auxiliary input 1 on MuseS 2025 (MS_03).
IXNEegAUX2	Auxiliary input 2 on MuseS 2025 (MS_03).
IXNEegAUX3	Auxiliary input 3 on MuseS 2025 (MS_03)
IXNEegAUX4	Auxiliary input 4 on MuseS 2025 (MS_03)

7.25 IXNEeg.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from packets.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00019 typedef NS_ENUM(NSInteger, IXNEeg)
00020 {
00022     IXNEegEEG1,
00024     IXNEegEEG2,
00026     IXNEegEEG3,
00028     IXNEegEEG4,
00030     IXNEegAUXLEFT,
00032     IXNEegAUXRIGHT,
00034     IXNEegAUX1,
00036     IXNEegAUX2,
00038     IXNEegAUX3,
00040     IXNEegAUX4,
00041 };
```

7.26 IXNError.h File Reference

Classes

- class IXNError

7.27 IXNError.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from validation.djinni
00003
00004 #import "Muse/api/IXNErrorType.h"
00005 #import <Foundation/Foundation.h>
00006
00013 @interface IXNError : NSObject
00014 - (nonnull instancetype)initWithType:(IXNErrorType)type
00015                                 code:(int32_t)code
00016                                 info:(nonnull NSString *)info;
00017 + (nonnull instancetype)errorWithType:(IXNErrorType)type
00018                                  code:(int32_t)code
```

```
00019                                info:(nonnull NSString *)info;
00020
00025 @property (nonatomic, readonly) IXNErrorType type;
00026
00031 @property (nonatomic, readonly) int32_t code;
00032
00037 @property (nonatomic, readonly, nonnull) NSString * info;
00038
00039 @end
```

7.28 IXNErrorType.h File Reference

Enumerations

- enum IXNErrorType {
  IXNErrorTypeFailure ,
  IXNErrorTypeTimeout ,
  IXNErrorTypeOverloaded ,
  IXNErrorTypeUnimplemented ,
  IXNErrorTypeHardwareEvent }

7.28.1 Detailed Description

Error classifications.

These are designed to provide users with courses of action based on the code that's received: e.g. retry on timeout, try a different way of doing the same thing on unimplemented, show the user an error message on failure.

7.28.2 Enumeration Type Documentation

7.28.2.1 IXNErrorType

enum IXNErrorType

Enumerator

IXNErrorTypeFailure	A generic failure occurred without any further information. Retrying the operation is unlikely to result in success.
IXNErrorTypeTimeout	Some timeout was exceeded; the operation might succeed if retried.
IXNErrorTypeOverloaded	Some resource (queue space, memory, bandwidth) was exhausted. Retry with backoff.
IXNErrorTypeUnimplemented	Something was tried that isn't implemented.
IXNErrorTypeHardwareEvent	A hardware event occurred.

7.29 IXNErrorType.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from validation.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00016 typedef NS_ENUM(NSInteger, IXNErrorType)
00017 {
00022     IXNErrorTypeFailure,
00024     IXNErrorTypeTimeout,
00029     IXNErrorTypeOverloaded,
00031     IXNErrorTypeUnimplemented,
00033     IXNErrorTypeHardwareEvent,
00034 };
```

7.30 IXNEventLoop.h File Reference

Classes

- protocol <IXNEventLoop>

7.31 IXNEventLoop.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from async.djinni
00003
00004 #import <Foundation/Foundation.h>
00005 @protocol IXNAction;
00006
00007
00009 @protocol IXNEventLoop
00010
00015 - (void)post:(nullable id<IXNAction>)action;
00016
00023 - (void)postDelayed:(nullable id<IXNAction>)action
00024   delayMilliseconds:(int64_t)delayMilliseconds;
00025
00027 - (void)cancel;
00028
00029 @end
```

7.32 IXNGyro.h File Reference

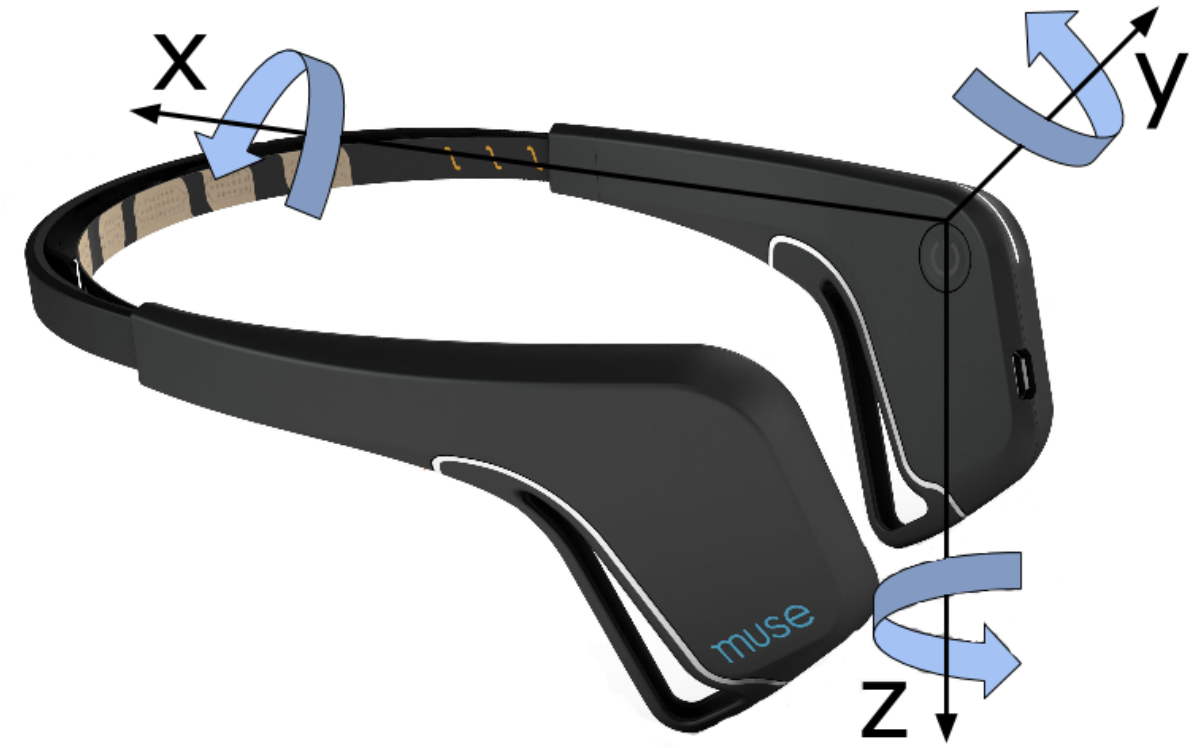
Enumerations

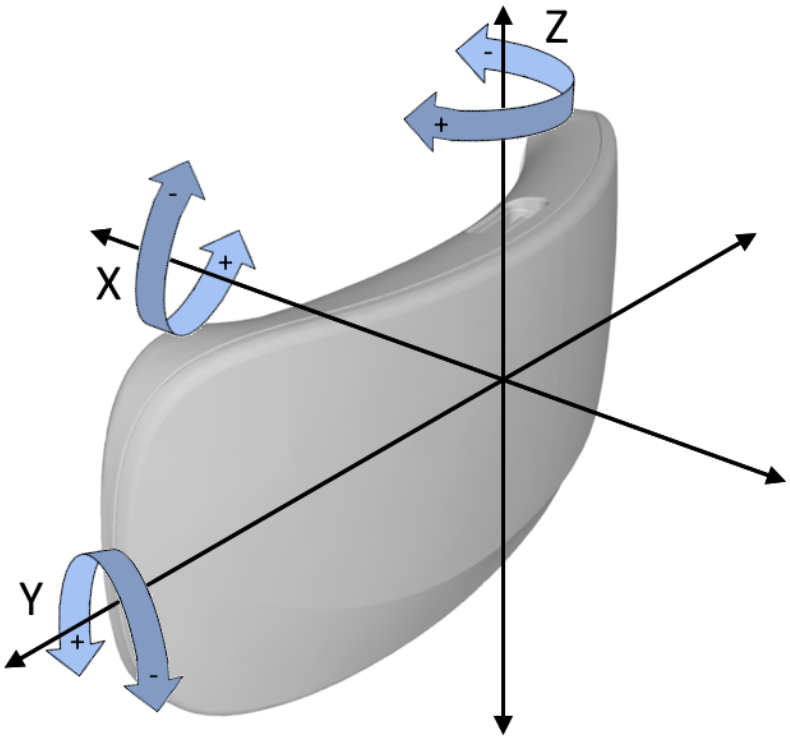
- enum IXNGyro {
  IXNGyroX ,
  IXNGyroY ,
  IXNGyroZ }

7.32.1 Detailed Description

Represents the data mapping in a Gyro data packet

The gyro data is measured as the rotation about 3 axes which is shown in the pictures below:





The axes are oriented to a Right Hand Coordinate System.
Rotation about the X axis corresponds to tilting the head side to side. Positive values increase when tilting to the right. This is also known as roll.
Rotation about the Y axis corresponds to tilting the head up and down. Positive values increase when looking up. This is also known as pitch.
Rotation about the Z axis corresponds to looking left and right. Positive values increase when looking to the right. This is also known as yaw.

See also

IXNMuseDataPacketTypeGyro

IXNMuseDataPacket::getGyroValue

7.32.2 Enumeration Type Documentation

7.32.2.1 IXNGyro

enum IXNGyro

Enumerator

IXNGyroX	Rotation about the X axis in degrees per second. Rotation about the X axis corresponds to tilting the head side to side. Positive values increase when tilting to the right. This is also known as roll.
IXNGyroY	Rotation about the Y axis in degrees per second. Rotation about the Y axis corresponds to tilting the head up and down. Positive values increase when looking up. This is also known as pitch.
IXNGyroZ	Rotation about the Z axis in degrees per second. Rotation about the Z axis corresponds to looking left and right. Positive values increase when looking to the right. This is also known as yaw.

7.33 IXNGyro.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from packets.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00028 typedef NS_ENUM(NSInteger, IXNGyro)
00029 {
00035     IXNGyroX,
00041     IXNGyroY,
00047     IXNGyroZ,
00048 };
```

7.34 IXNLibmuseVersion.h File Reference

Classes

- class IXNLibmuseVersion

7.35 IXNLibmuseVersion.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from libmuse_version.djinni
00003
00004 #import <Foundation/Foundation.h>
00005 @class IXNApiVersion;
00006
00007
00009 @interface IXNLibmuseVersion : NSObject
00010
00015 + (nullable IXNApiVersion *)instance;
00016
00017 @end
```

7.36 IXNLogListener.h File Reference

Classes

- protocol <IXNLogListener>

7.37 IXNLogListener.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from logging.djinni
00003
00004 #import "Muse/api/IXNLogPacket.h"
00005 #import <Foundation/Foundation.h>
00006
00007
00016 @protocol IXNLogListener
00017
00035 - (void)receiveLog:(nonnull IXNLogPacket *)log;
00036
00037 @end
```

7.38 IXNLogManager.h File Reference

Classes

- class IXNLogManager

7.39 IXNLogManager.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from logging.djinni
00003
00004 #import "Muse/api/IXNSeverity.h"
00005 #import <Foundation/Foundation.h>
00006 @class IXNLogManager;
00007 @protocol IXNLogListener;
00008
00009
00067 @interface IXNLogManager : NSObject
00068
00073 + (nullable IXNLogManager *)instance;
00074
00080 - (nullable id<IXNLogListener>)makeDefaultLogListener;
00081
00090 - (void)setLogListener:(nullable id<IXNLogListener>)listener;
00091
00099 - (void)setMinimumSeverity:(IXNSeverity)severity;
00100
00117 - (void)writeLog:(IXNSeverity)severity
00118              raw:(BOOL)raw
00119              tag:(nonnull NSString *)tag
00120          message:(nonnull NSString *)message;
00121
00130 - (int64_t)getTimestamp;
00131
00137 - (double)timeSince:(int64_t)timestamp;
00138
00139 @end
```

7.40 IXNLogPacket.h File Reference

Classes

- class IXNLogPacket

7.41 IXNLogPacket.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from logging.djinni
00003
00004 #import "Muse/api/IXNSeverity.h"
00005 #import <Foundation/Foundation.h>
00006
00008 @interface IXNLogPacket : NSObject
00009 - (nonnull instancetype)initWithSeverity:(IXNSeverity)severity
00010                                      raw:(BOOL)raw
00011                                      tag:(nonnull NSString *)tag
00012                                timestamp:(double)timestamp
00013                                  message:(nonnull NSString *)message;
00014 + (nonnull instancetype)logPacketWithSeverity:(IXNSeverity)severity
00015                                          raw:(BOOL)raw
00016                                          tag:(nonnull NSString *)tag
00017                                    timestamp:(double)timestamp
00018                                      message:(nonnull NSString *)message;
00019
00024 @property (nonatomic, readonly) IXNSeverity severity;
00025
00035 @property (nonatomic, readonly) BOOL raw;
00036
00042 @property (nonatomic, readonly, nonnull) NSString * tag;
00043
00048 @property (nonatomic, readonly) double timestamp;
00049
00055 @property (nonatomic, readonly, nonnull) NSString * message;
00056
00057 @end
```

7.42 IXNMagnetometer.h File Reference

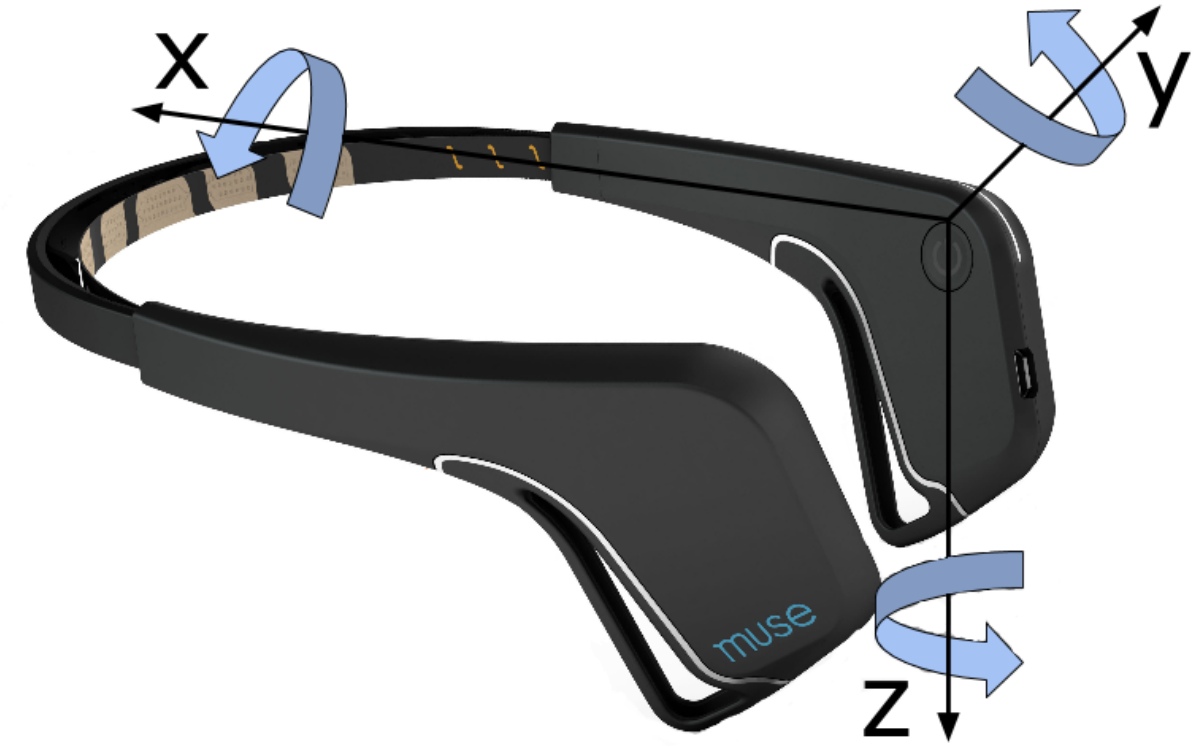
Enumerations

- enum IXNMagnetometer {
  IXNMagnetometerX ,
  IXNMagnetometerY ,
  IXNMagnetometerZ }

7.42.1 Detailed Description

Represents the data mapping in a Magnetometer data packet

The magnetometer data is measured on 3 axes as shown in the picture below:



The axes are oriented to a Right Hand Coordinate System.

See also

IXNMuseDataPacketTypeMagnetometer
IXNMuseDataPacket::getMagnetometerValue

7.42.2 Enumeration Type Documentation

7.42.2.1 IXNMagnetometer

enum IXNMagnetometer

Enumerator

IXNMagnetometerX	
IXNMagnetometerY	
IXNMagnetometerZ	

7.43 IXNMagnetometer.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from packets.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00020 typedef NS_ENUM(NSInteger, IXNMagnetometer)
00021 {
00022     IXNMagnetometerX,
00023     IXNMagnetometerY,
00024     IXNMagnetometerZ,
00025 };
```

7.44 IXNMessageType.h File Reference

Enumerations

- enum IXNMessageType {
  IXNMessageTypeEeg ,
  IXNMessageTypeQuantization ,
  IXNMessageTypeAccelerometer ,
  IXNMessageTypeBattery ,
  IXNMessageTypeVersion ,
  IXNMessageTypeConfiguration ,
  IXNMessageTypeAnnotation ,
  IXNMessageTypeHistogram ,
  IXNMessageTypeAlgValue ,
  IXNMessageTypeDsp ,
  IXNMessageTypeComputingDevice ,
  IXNMessageTypeEegDropped ,
  IXNMessageTypeAccDropped ,
  IXNMessageTypeCalmApp ,
  IXNMessageTypeCalmAlg ,
  IXNMessageTypeMuseElements ,
  IXNMessageTypeGyro ,
  IXNMessageTypeArtifact ,
  IXNMessageTypePressure ,
  IXNMessageTypeTemperature ,
  IXNMessageTypeUltraViolet ,
  IXNMessageTypeMagnetometer ,
  IXNMessageTypePpg ,
  IXNMessageTypeThermistor ,
  IXNMessageTypeOptics ,
  IXNMessageTypeAlgorithm ,
  IXNMessageTypeDeepSleepStim }

7.44.1 Detailed Description

Represents all possible MuseData message data types in a .muse file. This enum corresponds to the DataType enum in the protobuf schema.

7.44.2 Enumeration Type Documentation

7.44.2.1 IXNMessageType

enum IXNMessageType

Enumerator

IXNMessageTypeEeg	A message containing eeg data.
IXNMessageTypeQuantization	A message containing quantization data.
IXNMessageTypeAccelerometer	A message containing accelerometer data.
IXNMessageTypeBattery	A message containing battery data.
IXNMessageTypeVersion	A message containing version data.
IXNMessageTypeConfiguration	A message containing configuration data.

Enumerator

IXNMessageTypeAnnotation	A message containing annotation data.
IXNMessageTypeHistogram	A message containing histogram data.
IXNMessageTypeAlgValue	A message containing algorithm data.
IXNMessageTypeDsp	A message containing dsp data.
IXNMessageTypeComputingDevice	A message containing device data.
IXNMessageTypeEegDropped	A message containing dropped eeg data.
IXNMessageTypeAccDropped	A message containing dropped acc data.
IXNMessageTypeCalmApp	A message containing data from the calm application.
IXNMessageTypeCalmAlg	A message containing data from the calm algorithm.
IXNMessageTypeMuseElements	A message containing muse element data.
IXNMessageTypeGyro	A message containing gyro data.
IXNMessageTypeArtifact	A message containing artifact packet.
IXNMessageTypePressure	A message containing pressure data.
IXNMessageTypeTemperature	A message containing temperature data.
IXNMessageTypeUltraViolet	A message containing ultra violet data.
IXNMessageTypeMagnetometer	A message containing magnetometer data.
IXNMessageTypePpg	A message containing ppg data.
IXNMessageTypeThermistor	A message containing thermistor data.
IXNMessageTypeOptics	A message containing optics data.
IXNMessageTypeAlgorithm	A message containing algorithm data.
IXNMessageTypeDeepSleepStim	A message containing deep sleep stimulation data.

7.45 IXNMessageType.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from muse_file.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00013 typedef NS_ENUM(NSInteger, IXNMessageType)
00014 {
00016     IXNMessageTypeEeg,
00018     IXNMessageTypeQuantization,
00020     IXNMessageTypeAccelerometer,
00022     IXNMessageTypeBattery,
00024     IXNMessageTypeVersion,
00026     IXNMessageTypeConfiguration,
00028     IXNMessageTypeAnnotation,
00030     IXNMessageTypeHistogram,
00032     IXNMessageTypeAlgValue,
00034     IXNMessageTypeDsp,
00036     IXNMessageTypeComputingDevice,
00038     IXNMessageTypeEegDropped,
00040     IXNMessageTypeAccDropped,
00042     IXNMessageTypeCalmApp,
00044     IXNMessageTypeCalmAlg,
00046     IXNMessageTypeMuseElements,
00048     IXNMessageTypeGyro,
00050     IXNMessageTypeArtifact,
00052     IXNMessageTypePressure,
00054     IXNMessageTypeTemperature,
00056     IXNMessageTypeUltraViolet,
00058     IXNMessageTypeMagnetometer,
00060     IXNMessageTypePpg,
00062     IXNMessageTypeThermistor,
00064     IXNMessageTypeOptics,
00066     IXNMessageTypeAlgorithm,
00068     IXNMessageTypeDeepSleepStim,
00069 };
```

7.46 IXNMuse.h File Reference

Classes

- class IXNMuse

7.47 IXNMuse.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from sdk_bridge.djinni
00003
00004 #import "Muse/api/IXNConnectionState.h"
00005 #import "Muse/api/IXNMuseDataPacketType.h"
00006 #import "Muse/api/IXNMuseModel.h"
00007 #import "Muse/api/IXNMusePreset.h"
00008 #import "Muse/api/IXNNotchFrequency.h"
00009 #import <Foundation/Foundation.h>
00010 @class IXNMuseConfiguration;
00011 @class IXNMuseVersion;
00012 @protocol IXNMuseConnectionListener;
00013 @protocol IXNMuseDataListener;
00014 @protocol IXNMuseErrorListener;
00015
00016
00058 @interface IXNMuse : NSObject
00059
00072 - (void)connect;
00073
00087 - (void)disconnect;
00088
00110 - (void)execute;
00111
00149 - (void)runAsynchronously;
00150
00156 - (IXNConnectionState)getConnectionState;
00157
00163 - (nonnull NSString *)getMacAddress;
00164
00170 - (nonnull NSString *)getName;
00171
00182 - (double)getRssi;
00183
00195 - (IXNMuseModel)getModel;
00196
00224 - (double)getLastDiscoveredTime;
00225
00246 - (void)setNumConnectTries:(int32_t)numTries;
00247
00260 - (nullable IXNMuseConfiguration *)getMuseConfiguration;
00261
00272 - (nullable IXNMuseVersion *)getMuseVersion;
00273
00281 - (void)registerConnectionListener:(nullable id<IXNMuseConnectionListener>)listener;
00282
00288 - (void)unregisterConnectionListener:(nullable id<IXNMuseConnectionListener>)listener;
00289
00303 - (void)registerDataListener:(nullable id<IXNMuseDataListener>)listener
00304                         type:(IXNMuseDataPacketType)type;
00305
00315 - (void)unregisterDataListener:(nullable id<IXNMuseDataListener>)listener
00316                           type:(IXNMuseDataPacketType)type;
00317
00324 - (void)registerErrorListener:(nullable id<IXNMuseErrorListener>)listener;
00325
00334 - (void)unregisterErrorListener:(nullable id<IXNMuseErrorListener>)listener;
00335
00342 - (void)unregisterAllListeners;
00343
00354 - (void)setPreset:(IXNMusePreset)preset;
00355
00362 - (void)enableLedIndicator:(BOOL)enable;
00363
00377 - (void)enableDataTransmission:(BOOL)enable;
00378
00417 - (void)setNotchFrequency:(IXNNotchFrequency)newFrequency;
00418
00424 - (BOOL)isLowEnergy;
```

```
00425
00435 - (BOOL)isPaired;
00436
00446 - (BOOL)isConnectable;
00447
00453 - (void)setLicenseData:(nonnull NSData *)data;
00454
00466 - (void)enableException:(BOOL)enable;
00467
00469 - (void)setProperty:(nonnull NSString *)name
00470              value:(nonnull NSString *)value;
00471
00472 @end
```

7.48 IXNMuseArtifactPacket.h File Reference

Classes

- class IXNMuseArtifactPacket

7.49 IXNMuseArtifactPacket.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from packets.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00007 @interface IXNMuseArtifactPacket : NSObject
00008 - (nonnull instancetype)initWithHeadbandOn:(BOOL)headbandOn
00009                                      blink:(BOOL)blink
00010                                  jawClench:(BOOL)jawClench
00011                                  timestamp:(int64_t)timestamp;
00012 + (nonnull instancetype)museArtifactPacketWithHeadbandOn:(BOOL)headbandOn
00013                                                    blink:(BOOL)blink
00014                                                jawClench:(BOOL)jawClench
00015                                                timestamp:(int64_t)timestamp;
00016
00024 @property (nonatomic, readonly) BOOL headbandOn;
00025
00030 @property (nonatomic, readonly) BOOL blink;
00031
00036 @property (nonatomic, readonly) BOOL jawClench;
00037
00042 @property (nonatomic, readonly) int64_t timestamp;
00043
00044 @end
```

7.50 IXNMuseConfiguration.h File Reference

Classes

- class IXNMuseConfiguration

7.51 IXNMuseConfiguration.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from museinfo.djinni
00003
00004 #import "Muse/api/IXNMuseModel.h"
00005 #import "Muse/api/IXNMusePreset.h"
00006 #import "Muse/api/IXNNotchFrequency.h"
00007 #import <Foundation/Foundation.h>
00008
00009
00017 @interface IXNMuseConfiguration : NSObject
00018
00020 - (IXNMusePreset)getPreset;
00021
00026 - (nonnull NSString *)getHeadbandName;
00027
00033 - (nonnull NSString *)getMicrocontrollerId;
00034
00039 - (int32_t)getEegChannelCount;
00040
00045 - (int32_t)getAfeGain;
00046
00051 - (int32_t)getDownsampleRate;
00052
00061 - (int32_t)getSeroutMode;
00062
00067 - (int32_t)getOutputFrequency;
00068
00073 - (int32_t)getAdcFrequency;
00074
00079 - (BOOL)getNotchFilterEnabled;
00080
00085 - (IXNNotchFrequency)getNotchFilter;
00086
00091 - (int32_t)getAccelerometerSampleFrequency;
00092
00098 - (BOOL)getBatteryDataEnabled;
00099
00105 - (BOOL)getDrlRefEnabled;
00106
00112 - (int32_t)getDrlRefFrequency;
00113
00118 - (double)getBatteryPercentRemaining;
00119
00124 - (nonnull NSString *)getBluetoothMac;
00125
00130 - (nonnull NSString *)getSerialNumber;
00131
00137 - (nonnull NSString *)getHeadsetSerialNumber;
00138
00143 - (IXNMuseModel)getModel;
00144
00149 - (nonnull NSString *)getLicenseNonce;
00150
00152 - (int32_t)getSwitch;
00153
00154 @end
```

7.52 IXNMuseConnectionListener.h File Reference

Classes

- protocol <IXNMuseConnectionListener>

7.53 IXNMuseConnectionListener.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from listeners.djinni
00003
```

```
00004 #import "Muse/api/IXNMuseConnectionPacket.h"
00005 #import <Foundation/Foundation.h>
00006 @class IXNMuse;
00007
00008
00020 @protocol IXNMuseConnectionListener
00021
00045 - (void)receiveMuseConnectionPacket:(nonnull IXNMuseConnectionPacket *)packet
00046                                muse:(nullable IXNMuse *)muse;
00047
00048 @end
```

7.54 IXNMuseConnectionPacket.h File Reference

Classes

- class IXNMuseConnectionPacket

7.55 IXNMuseConnectionPacket.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from packets.djinni
00003
00004 #import "Muse/api/IXNConnectionState.h"
00005 #import <Foundation/Foundation.h>
00006
00008 @interface IXNMuseConnectionPacket : NSObject
00009 - (nonnull instancetype)initWithPreviousConnectionState:(IXNConnectionState)previousConnectionState
00010                                    currentConnectionState:(IXNConnectionState)currentConnectionState;
00011 + (nonnull
      instancetype)museConnectionPacketWithPreviousConnectionState:(IXNConnectionState)previousConnectionState
00012                                                  currentConnectionState:(IXNConnectionState)currentConnectionState;
00013
00018 @property (nonatomic, readonly) IXNConnectionState previousConnectionState;
00019
00024 @property (nonatomic, readonly) IXNConnectionState currentConnectionState;
00025
00026 @end
```

7.56 IXNMuseDataListener.h File Reference

Classes

- protocol <IXNMuseDataListener>

7.57 IXNMuseDataListener.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from listeners.djinni
00003
00004 #import "Muse/api/IXNMuseArtifactPacket.h"
00005 #import <Foundation/Foundation.h>
00006 @class IXNMuse;
00007 @class IXNMuseDataPacket;
00008
00009
00020 @protocol IXNMuseDataListener
00021
00044 - (void)receiveMuseDataPacket:(nullable IXNMuseDataPacket *)packet
00045                          muse:(nullable IXNMuse *)muse;
00046
00070 - (void)receiveMuseArtifactPacket:(nonnull IXNMuseArtifactPacket *)packet
00071                              muse:(nullable IXNMuse *)muse;
00072
00073 @end
```

7.58 IXNMuseDataPacket.h File Reference

Classes

- class IXNMuseDataPacket

7.59 IXNMuseDataPacket.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from packets.djinni
00003
00004 #import "Muse/api/IXNAccelerometer.h"
00005 #import "Muse/api/IXNBattery.h"
00006 #import "Muse/api/IXNDrlRef.h"
00007 #import "Muse/api/IXNEeg.h"
00008 #import "Muse/api/IXNGyro.h"
00009 #import "Muse/api/IXNMagnetometer.h"
00010 #import "Muse/api/IXNMuseDataPacketType.h"
00011 #import "Muse/api/IXNMuseModel.h"
00012 #import "Muse/api/IXNOptics.h"
00013 #import "Muse/api/IXNPpg.h"
00014 #import "Muse/api/IXNPressure.h"
00015 #import "Muse/api/IXNUltraViolet.h"
00016 #import <Foundation/Foundation.h>
00017 @class IXNMuseDataPacket;
00018
00019
00027 @interface IXNMuseDataPacket : NSObject
00028
00033 + (nullable IXNMuseDataPacket *)makeUninitializedPacket:(int64_t)capacity;
00034
00041 + (nullable IXNMuseDataPacket *)makePacket:(IXNMuseDataPacketType)type
00042                                  timestamp:(int64_t)timestamp
00043                                     values:(nonnull NSArray<NSNumber *> *)values;
00044
00049 - (IXNMuseDataPacketType)packetType;
00050
00055 - (int64_t)timestamp;
00056
00075 - (nonnull NSArray<NSNumber *> *)values;
00076
00081 - (int64_t)valuesSize;
00082
00108 - (double)getEegChannelValue:(IXNEeg)channelNum;
00109
00133 - (double)getPpgChannelValue:(IXNPpg)channelNum;
00134
00142 - (double)getPpgMicroamps:(IXNMuseModel)model
00143                   ppgValue:(double)ppgValue;
00144
00168 - (double)getOpticsChannelValue:(IXNOptics)channelNum;
00169
00184 - (double)getBatteryValue:(IXNBattery)b;
00185
00200 - (double)getAccelerometerValue:(IXNAccelerometer)a;
00201
00218 - (double)getGyroValue:(IXNGyro)g;
00219
00236 - (double)getMagnetometerValue:(IXNMagnetometer)m;
00237
00253 - (double)getDrlRefValue:(IXNDrlRef)drl;
00254
00271 - (double)getPressureValue:(IXNPressure)pressure;
00272
00288 - (double)getTemperatureValue;
00289
00306 - (double)getUvValue:(IXNUltraViolet)v;
00307
00308 @end
```

7.60 IXNMuseDataPacketType.h File Reference

Enumerations

- enum IXNMuseDataPacketType {
  IXNMuseDataPacketTypeAccelerometer ,
  IXNMuseDataPacketTypeGyro ,
  IXNMuseDataPacketTypeEeg ,
  IXNMuseDataPacketTypeDroppedAccelerometer ,
  IXNMuseDataPacketTypeDroppedEeg ,
  IXNMuseDataPacketTypeQuantization ,
  IXNMuseDataPacketTypeBattery ,
  IXNMuseDataPacketTypeDrlRef ,
  IXNMuseDataPacketTypeAlphaAbsolute ,
  IXNMuseDataPacketTypeBetaAbsolute ,
  IXNMuseDataPacketTypeDeltaAbsolute ,
  IXNMuseDataPacketTypeThetaAbsolute ,
  IXNMuseDataPacketTypeGammaAbsolute ,
  IXNMuseDataPacketTypeAlphaRelative ,
  IXNMuseDataPacketTypeBetaRelative ,
  IXNMuseDataPacketTypeDeltaRelative ,
  IXNMuseDataPacketTypeThetaRelative ,
  IXNMuseDataPacketTypeGammaRelative ,
  IXNMuseDataPacketTypeAlphaScore ,
  IXNMuseDataPacketTypeBetaScore ,
  IXNMuseDataPacketTypeDeltaScore ,
  IXNMuseDataPacketTypeThetaScore ,
  IXNMuseDataPacketTypeGammaScore ,
  IXNMuseDataPacketTypeIsGood ,
  IXNMuseDataPacketTypeHsi ,
  IXNMuseDataPacketTypeHsiPrecision ,
  IXNMuseDataPacketTypeArtifacts ,
  IXNMuseDataPacketTypeMagnetometer ,
  IXNMuseDataPacketTypePressure ,
  IXNMuseDataPacketTypeTemperature ,
  IXNMuseDataPacketTypeUltraViolet ,
  IXNMuseDataPacketTypeNotchFilteredEeg ,
  IXNMuseDataPacketTypeVarianceEeg ,
  IXNMuseDataPacketTypeVarianceNotchFilteredEeg ,
  IXNMuseDataPacketTypePpg ,
  IXNMuseDataPacketTypeIsPpgGood ,
  IXNMuseDataPacketTypeIsHeartGood ,
  IXNMuseDataPacketTypeThermistor ,
  IXNMuseDataPacketTypeIsThermistorGood ,
  IXNMuseDataPacketTypeAvgBodyTemperature ,
  IXNMuseDataPacketTypeCloudComputed ,
  IXNMuseDataPacketTypeOptics ,
  IXNMuseDataPacketTypeTotal }

7.60.1 Detailed Description

This Enum represents all possible packet types. The type of the packet tells you information about what data it contains. When you know the packet type, look at the corresponding enum for information about data mapping (e.g.: Accelerometer enum, EEG enum, etc).

EEG derived signals

Absolute band powers

The absolute band power for a given frequency range (for instance, alpha, i.e. 7.5-13Hz) is the logarithm of the sum of the Power Spectral Density of the EEG data over that frequency range. They are provided for each of the channels/electrode sites on Muse. Since it is a logarithm, some of the values will be negative (i.e. when the absolute power is less than 1). They are given on a log scale, units are Bels.

Relative band powers

The relative band powers are calculated by dividing the absolute linear-scale power in one band over the sum of the absolute linear-scale powers in all bands. The linear-scale band power can be calculated from the log-scale band power.

linear-scale band power = 10^ (log-scale band power)

Therefore, the relative band powers can be calculated as percentages of linear-scale band powers in each band. For example:

alpha_relative = (10^alpha_absolute / (10^alpha_absolute + 10^beta_absolute + 10^delta_absolute + 10^gamma_absolute + 10^theta_absolute))

Band power scores

The band session score is computed by comparing the current value of a band power to its history. This current value is mapped to a score between 0 and 1 using a linear function that returns 0 if the current value is equal to or below the 10th percentile of the distribution of band powers, and returns 1 if it's equal to or above the 90th percentile. Linear scoring between 0 and 1 is done for any value between these two percentiles.

Be advised that these scores are based on recent history and it will take a few seconds before having a stable distribution to score the power against. The estimated distribution is continuously updated as long as the headband is on the head. However, every time it's updated, the newest values are weighted to have more importance than the historical values. This means that eventually old values will not be present anymore in the estimated distribution. The half-life of the estimated distribution at any given point is around 10 seconds.

The score will start being calculated as soon as the headband has established a good connection with the skin. Whenever the headset loses connection with the head (as determined by the DRL/REF contact quality) the estimated distributions are reset. This means that when the headband is removed, the session data from any previous user will be cleared.

7.60.2 Enumeration Type Documentation

7.60.2.1 IXNMuseDataPacketType

enum IXNMuseDataPacketType

Enumerator

IXNMuseDataPacketTypeAccelerometer	3-axis accelerometer data packet An Accelerometer packet provides 3 pieces of data. See also • IXNAccelerometer for mapping details.
IXNMuseDataPacketTypeGyro	3-axis gyro data packet A Gyro packet provides 3 pieces of data. See also • IXNGyro for mapping details.
IXNMuseDataPacketTypeEeg	Specifies raw EEG samples. Values in this packet correspond to EEG data read from the different sensor locations on the headband. The accessors in the Eeg enum define the mapping from packet values to sensor locations. The units of EEG values are microvolts. The size of the data is unspecified, but it is large enough to hold all the EEG channels emitted by the current preset. In the future new Muse Presets may be added, which will have extra values. See also • IXNEeg for mapping details.
IXNMuseDataPacketTypeDroppedAccelerometer	Packet stands in for n dropped samples of the accelerometer type. Size of the values array for this packet is always 1. Deprecated This is never emitted in an actual Muse session; instead, NaN-filled packets of the basic type (EEG or ACCELEROMETER) are emitted to stand in for dropped packets. This can only appear when reading Muse files written with older versions of the library.
IXNMuseDataPacketTypeDroppedEeg	Packet stands in for n dropped samples of the eeg type. Size of the values array for this packet is always 1. Deprecated This is never emitted in an actual Muse session; instead, NaN-filled packets of the basic type (EEG or ACCELEROMETER) are emitted to stand in for dropped packets. This can only appear when reading Muse files written with older versions of the library.

Enumerator

IXNMuseDataPacketTypeQuantization	Packet contains information about signal quantization. This packet contains the same amount of data as an EEG packet and has the same channel mapping. Each index in this packet corresponds to the same index in an EEG packet. Quantization occurs when there is a particularly noisy signal, which generally happens when there is not a good contact between the headband and the skin. Higher numbers are worse; 1 is no quantization, and 16 is maximum quantization. These values are used under the hood by the library and by Muse Elements in reconstructing the EEG signal and contributing to an overall measure of noise; it is extremely unlikely that you will be interested in them. For measuring noise, it is recommended to instead use the more useful computed values like 'headband_on' or 'hsi'. Each quantization packet applies to the next 16 EEG packets. See also • IXNEeg for mapping details.
IXNMuseDataPacketTypeBattery	Muse headband battery data packet. This packet provides 3 pieces of data. See also • IXNBattery for mapping details.
IXNMuseDataPacketTypeDrlRef	Packet contains raw data from Muse DRL and REF sensors. This packet provides 2 pieces of data. The units of both values are in microvolts. See also • IXNDrlRef for mapping details.
IXNMuseDataPacketTypeAlphaAbsolute	EEG derived value. Absolute alpha band powers for each channel. This packet contains the same amount of data as an EEG packet and has the same channel mapping. See also • IXNEeg for mapping details.
IXNMuseDataPacketTypeBetaAbsolute	EEG derived value. Absolute beta band powers for each channel. This packet contains the same amount of data as an EEG packet and has the same channel mapping. See also • IXNEeg for mapping details.

Enumerator

IXNMuseDataPacketTypeDeltaAbsolute	EEG derived value. Absolute delta band powers for each channel. This packet contains the same amount of data as an EEG packet and has the same channel mapping. See also • IXNEeg for mapping details.
IXNMuseDataPacketTypeThetaAbsolute	EEG derived value. Absolute theta band powers for each channel. This packet contains the same amount of data as an EEG packet and has the same channel mapping. See also • IXNEeg for mapping details.
IXNMuseDataPacketTypeGammaAbsolute	EEG derived value. Absolute gamma band powers for each channel. This packet contains the same amount of data as an EEG packet and has the same channel mapping. See also • IXNEeg for mapping details.
IXNMuseDataPacketTypeAlphaRelative	EEG derived value. Relative alpha band powers for each channel. Values in this packet are in range [0; 1]. This packet contains the same amount of data as an EEG packet and has the same channel mapping. See also • IXNEeg for mapping details.
IXNMuseDataPacketTypeBetaRelative	EEG derived value. Relative beta band powers for each channel. Values in this packet are in range [0; 1]. This packet contains the same amount of data as an EEG packet and has the same channel mapping. See also • IXNEeg for mapping details.
IXNMuseDataPacketTypeDeltaRelative	EEG derived value. Relative delta band powers for each channel. Values in this packet are in range [0; 1]. This packet contains the same amount of data as an EEG packet and has the same channel mapping. See also • IXNEeg for mapping details.

Enumerator

IXNMuseDataPacketTypeThetaRelative	EEG derived value. Relative band powers for each channel. Values in this packet are in range [0; 1]. This packet contains the same amount of data as an EEG packet and has the same channel mapping. See also • IXNEeg for mapping details.
IXNMuseDataPacketTypeGammaRelative	EEG derived value. Relative band powers for each channel. Values in this packet are in range [0; 1]. This packet contains the same amount of data as an EEG packet and has the same channel mapping. See also • IXNEeg for mapping details.
IXNMuseDataPacketTypeAlphaScore	EEG derived value. Alpha band power scores for each channel. This packet contains the same amount of data as an EEG packet and has the same channel mapping. See also • IXNEeg for mapping details.
IXNMuseDataPacketTypeBetaScore	EEG derived value. Beta band power scores for each channel. This packet contains the same amount of data as an EEG packet and has the same channel mapping. See also • IXNEeg for mapping details.
IXNMuseDataPacketTypeDeltaScore	EEG derived value. Delta band power scores for each channel. This packet contains the same amount of data as an EEG packet and has the same channel mapping. See also • IXNEeg for mapping details.
IXNMuseDataPacketTypeThetaScore	EEG derived value. Theta band power scores for each channel. This packet contains the same amount of data as an EEG packet and has the same channel mapping. See also • IXNEeg for mapping details.

Enumerator

IXNMuseDataPacketTypeGammaScore	EEG derived value. Gamma band power scores for each channel. This packet contains the same amount of data as an EEG packet and has the same channel mapping. See also • IXNEeg for mapping details.
IXNMuseDataPacketTypeIsGood	EEG derived value. Is Good indicates whether or not the last 1 second of raw EEG data on each channel was good or not. Eye blinks or muscle movement can interfere with EEG data and cause Is Good to report that the data is not good. This is emitted every 1/10 of a second to represent the rolling window of the last second of EEG data. This is only useful for real time EEG analysis. This packet only contains 4 values for the 4 sensors on the headband, there is no support for the auxillary channels. See also • IXNEeg for mapping details.
IXNMuseDataPacketTypeHsi	EEG derived value. HSI values represent the fit of the headband. (known as headband status indicator). This value is not emitted by the LibMuse SDK.
IXNMuseDataPacketTypeHsiPrecision	EEG derived value. HSI precision values represent the fit of the headband. This packet contains 4 values corresponding to IXNEegEEG1 , IXNEegEEG2 , IXNEegEEG3 and IXNEegEEG4 There are no IXNMuseDataPacketTypeHsiPrecision values for the auxillary channels. Each channel represents the fit at that location. A value of 1 represents a good fit, 2 represents a mediocre fit, and a value or 4 represents a poor fit. See also • IXNEeg for mapping details.
IXNMuseDataPacketTypeArtifacts	Artifacts packet type will be sent. Note that this will result in your listener receiving IXNMuseArtifactPacket We never emit a IXNMuseDataPacket with IXNMuseDataPacketTypeArtifacts ; this is only here for use in register / unregister methods.
IXNMuseDataPacketTypeMagnetometer	3-axis magnetometer data packet A Magnetometer packet provides 3 pieces of data. See also • IXNMagnetometer for mapping details.

Enumerator

IXNMuseDataPacketTypePressure	Pressure packet provides both a raw and averaged ambient pressure value. See also • IXNPressure for mapping details.
IXNMuseDataPacketTypeTemperature	Temperature packet provides ambient temperature value.
IXNMuseDataPacketTypeUltraViolet	UltraViolet packet provides both UVA and UVB index value and the average of the 2. See also • IXNUltraViolet for mapping details.
IXNMuseDataPacketTypeNotchFilteredEeg	EEG derived value. Notch filtered EEG is the raw EEG passed through a band stop filter to remove frequencies between 45 and 65 Hz inclusive. This packet contains 4 values corresponding to IXNEegEEG1 , IXNEegEEG2 , IXNEegEEG3 and IXNEegEEG4 See also • IXNEeg for mapping details.
IXNMuseDataPacketTypeVarianceEeg	EEG derived value. Signal variance for raw EEG values. Variance is the numerical value that measures how widely a set of numbers within the interval are spread out from the average value. This packet contains the variance value of raw EEG over the last second. See also • IXNEeg for mapping details. • https://en.wikipedia.org/wiki/Variance#Discrete_random_variable
IXNMuseDataPacketTypeVarianceNotchFilteredEeg	EEG derived value. Signal variance for notch filtered EEG. Variance is the numerical value that measures how widely a set of numbers within the interval are spread out from the average value. This packet contains the variance value of notch filtered EEG over the last second. See also • IXNEeg for mapping details. • https://en.wikipedia.org/wiki/Variance#Discrete_random_variable

Enumerator

IXNMuseDataPacketTypePpg	Specifies PPG samples. Values in this packet correspond to PPG data read from supported hardware. The accessors in the Ppg enum define the mapping from packet data to 3 different values Ambient, IR, and RED. The units of PPG values are arbitrary. See also • IXNPpg for mapping details.
IXNMuseDataPacketTypeIsPpgGood	PPG derived value. See also • IXNPpg for mapping details.
IXNMuseDataPacketTypeIsHeartGood	PPG derived value. See also • IXNPpg for mapping details.
IXNMuseDataPacketTypeThermistor	Provides temperature values from the thermistor that is in contact with the user's skin.
IXNMuseDataPacketTypeIsThermistorGood	Thermistor derived value.
IXNMuseDataPacketTypeAvgBodyTemperature	Thermistor derived value.
IXNMuseDataPacketTypeCloudComputed	Specifies a cloud computed value. Values in this packet have been computed on the cloud remotely. This packet type allows the libmuse library to expose new packet type that has been generated remotely on the cloud.
IXNMuseDataPacketTypeOptics	Specifies OPTICS samples. Values in this packet correspond to optics data read from supported hardware. The accessors in the Optics enum define the mapping from packet data to 16 different values. The units of OPTICS values are microamps. See also • IXNOptics for mapping details.
IXNMuseDataPacketTypeTotal	The total number of possible data packet types

7.61 IXNMuseDataPacketType.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from packets.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00049 typedef NS_ENUM(NSInteger, IXNMuseDataPacketType)
00050 {
00057     IXNMuseDataPacketTypeAccelerometer,
00064     IXNMuseDataPacketTypeGyro,
00081     IXNMuseDataPacketTypeEeg,
00091     IXNMuseDataPacketTypeDroppedAccelerometer,
```

```
00101     IXNMuseDataPacketTypeDroppedEeg,
00125     IXNMuseDataPacketTypeQuantization,
00132     IXNMuseDataPacketTypeBattery,
00139     IXNMuseDataPacketTypeDrlRef,
00148     IXNMuseDataPacketTypeAlphaAbsolute,
00157     IXNMuseDataPacketTypeBetaAbsolute,
00166     IXNMuseDataPacketTypeDeltaAbsolute,
00175     IXNMuseDataPacketTypeThetaAbsolute,
00184     IXNMuseDataPacketTypeGammaAbsolute,
00194     IXNMuseDataPacketTypeAlphaRelative,
00204     IXNMuseDataPacketTypeBetaRelative,
00214     IXNMuseDataPacketTypeDeltaRelative,
00224     IXNMuseDataPacketTypeThetaRelative,
00234     IXNMuseDataPacketTypeGammaRelative,
00243     IXNMuseDataPacketTypeAlphaScore,
00252     IXNMuseDataPacketTypeBetaScore,
00261     IXNMuseDataPacketTypeDeltaScore,
00270     IXNMuseDataPacketTypeThetaScore,
00279     IXNMuseDataPacketTypeGammaScore,
00293     IXNMuseDataPacketTypeIsGood,
00299     IXNMuseDataPacketTypeHsi,
00319     IXNMuseDataPacketTypeHsiPrecision,
00329     IXNMuseDataPacketTypeArtifacts,
00336     IXNMuseDataPacketTypeMagnetometer,
00342     IXNMuseDataPacketTypePressure,
00344     IXNMuseDataPacketTypeTemperature,
00351     IXNMuseDataPacketTypeUltraViolet,
00364     IXNMuseDataPacketTypeNotchFilteredEeg,
00375     IXNMuseDataPacketTypeVarianceEeg,
00386     IXNMuseDataPacketTypeVarianceNotchFilteredEeg,
00398     IXNMuseDataPacketTypePpg,
00405     IXNMuseDataPacketTypeIsPpgGood,
00412     IXNMuseDataPacketTypeIsHeartGood,
00417     IXNMuseDataPacketTypeThermistor,
00419     IXNMuseDataPacketTypeIsThermistorGood,
00421     IXNMuseDataPacketTypeAvgBodyTemperature,
00430     IXNMuseDataPacketTypeCloudComputed,
00442     IXNMuseDataPacketTypeOptics,
00444     IXNMuseDataPacketTypeTotal,
00445 };
```

7.62 IXNMuseErrorListener.h File Reference

Classes

- protocol <IXNMuseErrorListener>

7.63 IXNMuseErrorListener.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from listeners.djinni
00003
00004 #import "Muse/api/IXNError.h"
00005 #import <Foundation/Foundation.h>
00006 @class IXNMuse;
00007
00008
00010 @protocol IXNMuseErrorListener
00011
00042 - (void)receiveError:(nonnull IXNError *)error
00043                 muse:(nullable IXNMuse *)muse;
00044
00045 @end
```

7.64 IXNMuseFile.h File Reference

Classes

- protocol <IXNMuseFile>

7.65 IXNMuseFile.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from muse_file.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00006
00028 @protocol IXNMuseFile
00029
00039 - (BOOL)open:(BOOL)forWriting;
00040
00048 - (BOOL)write:(nonnull NSData *)buffer;
00049
00056 - (nonnull NSData *)read:(int32_t)length;
00057
00065 - (BOOL)close:(BOOL)forWriting;
00066
00067 @end
```

7.66 IXNMuseFileReader.h File Reference

Classes

- class IXNMuseFileReader

7.67 IXNMuseFileReader.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from muse_file.djinni
00003
00004 #import "Muse/api/IXNAnnotationData.h"
00005 #import "Muse/api/IXNComputingDeviceConfiguration.h"
00006 #import "Muse/api/IXNDspData.h"
00007 #import "Muse/api/IXNMessageType.h"
00008 #import "Muse/api/IXNMuseArtifactPacket.h"
00009 #import "Muse/api/IXNResult.h"
00010 #import <Foundation/Foundation.h>
00011 @class IXNMuseConfiguration;
00012 @class IXNMuseDataPacket;
00013 @class IXNMuseFileReader;
00014 @class IXNMuseVersion;
00015 @protocol IXNMuseFile;
00016
00017
00038 @interface IXNMuseFileReader : NSObject
00039
00044 + (nullable IXNMuseFileReader *)getFileReader:(nullable id<IXNMuseFile>)file;
00045
00051 - (BOOL)open;
00052
00059 - (BOOL)close;
00060
00065 - (nonnull IXNResult *)gotoNextMessage;
00066
00071 - (IXNMessageType)getMessageType;
00072
00079 - (int32_t)getMessageId;
00080
00085 - (int64_t)getMessageTimestamp;
00086
00093 - (nonnull IXNAnnotationData *)getAnnotation;
00094
00101 - (nullable IXNMuseConfiguration *)getConfiguration;
00102
00109 - (nullable IXNMuseVersion *)getVersion;
00110
00117 - (nonnull IXNComputingDeviceConfiguration *)getComputingDeviceConfiguration;
00118
00125 - (nonnull IXNDspData *)getDsp;
```

```
00126
00141 - (nullable IXNMuseDataPacket *)getDataPacket;
00142
00149 - (nonnull IXNMuseArtifactPacket *)getArtifactPacket;
00150
00151 @end
```

7.68 IXNMuseFileWriter.h File Reference

Classes

- class IXNMuseFileWriter

7.69 IXNMuseFileWriter.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from muse_file.djinni
00003
00004 #import "Muse/api/IXNAnnotationData.h"
00005 #import "Muse/api/IXNComputingDeviceConfiguration.h"
00006 #import "Muse/api/IXNDspData.h"
00007 #import "Muse/api/IXNMuseArtifactPacket.h"
00008 #import "Muse/api/IXNTimestampMode.h"
00009 #import <Foundation/Foundation.h>
00010 @class IXNMuseConfiguration;
00011 @class IXNMuseDataPacket;
00012 @class IXNMuseFileWriter;
00013 @class IXNMuseVersion;
00014 @protocol IXNMuseFile;
00015
00016
00043 @interface IXNMuseFileWriter : NSObject
00044
00049 + (nullable IXNMuseFileWriter *)getFileWriter:(nullable id<IXNMuseFile>)file;
00050
00058 - (BOOL)open;
00059
00064 - (BOOL)isOpen;
00065
00074 - (BOOL)close;
00075
00077 - (void)discardBufferedPackets;
00078
00083 - (BOOL)flush;
00084
00089 - (int32_t)getBufferredMessagesCount;
00090
00095 - (int32_t)getBufferedMessagesSize;
00096
00101 - (int64_t)getTotalBytesWritten;
00102
00110 - (void)addArtifactPacket:(int32_t)id
00111                    packet:(nonnull IXNMuseArtifactPacket *)packet;
00112
00121 - (void)addDataPacket:(int32_t)id
00122                 packet:(nullable IXNMuseDataPacket *)packet;
00123
00137 - (void)addAnnotationString:(int32_t)id
00138                   annotation:(nonnull NSString *)annotation;
00139
00151 - (void)addAnnotation:(int32_t)id
00152            annotation:(nonnull IXNAnnotationData *)annotation;
00153
00161 - (void)addConfiguration:(int32_t)id
00162            configuration:(nullable IXNMuseConfiguration *)configuration;
00163
00171 - (void)addVersion:(int32_t)id
00172            version:(nullable IXNMuseVersion *)version;
00173
00181 - (void)addComputingDeviceConfiguration:(int32_t)id
00182                              configuration:(nonnull IXNComputingDeviceConfiguration *)configuration;
00183
00191 - (void)addDsp:(int32_t)id
```

```
00192           dsp:(nonnull IXNDspData *)dsp;
00193
00200 - (void)setTimestampMode:(IXNTimestampMode)mode;
00201
00216 - (void)setTimestamp:(int64_t)timestamp;
00217
00218 @end
```

7.70 IXNMuseListener.h File Reference

Classes

- protocol <IXNMuseListener>

7.71 IXNMuseListener.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from sdk_bridge.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00006
00012 @protocol IXNMuseListener
00013
00038 - (void)museListChanged;
00039
00040 @end
```

7.72 IXNMuseManager.h File Reference

Classes

- protocol <IXNMuseManager>

Variables

- int64_t const IXNMuseManagerDEFAULTREMOVEFROMLISTAFTER

7.72.1 Variable Documentation

7.72.1.1 IXNMuseManagerDEFAULTREMOVEFROMLISTAFTER

int64_t const IXNMuseManagerDEFAULTREMOVEFROMLISTAFTER [extern]

The default time in seconds after which a headband will be removed from the list of muses if IXNMuseManager has had no contact with it.

This has a value of 30 seconds.

7.73 IXNMuseManager.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from sdk_bridge.djinni
00003
00004 #import "Muse/api/IXNAdvertisingStats.h"
00005 #import <Foundation/Foundation.h>
00006 @class IXNMuse;
00007 @protocol IXNMuseListener;
00008
00015 extern int64_t const IXNMuseManagerDEFAULTREMOVEFROMLISTAFTER;
00016
00033 @protocol IXNMuseManager
00034
00043 - (nonnull NSArray<IXNMuse *> *)getMuses;
00044
00079 - (void)startListening;
00080
00095 - (void)stopListening;
00096
00102 - (void)setMuseListener:(nullable id<IXNMuseListener>)listener;
00103
00109 - (nonnull IXNAdvertisingStats *)getAdvertisingStats:(nullable IXNMuse *)m;
00110
00116 - (void)resetAdvertisingStats;
00117
00133 - (void)removeFromListAfter:(int64_t)time;
00134
00135 @end
```

7.74 IXNMuseModel.h File Reference

Enumerations

- enum IXNMuseModel {
  IXNMuseModelMu01 ,
  IXNMuseModelMu02 ,
  IXNMuseModelMu03 ,
  IXNMuseModelMu04 ,
  IXNMuseModelMu05 ,
  IXNMuseModelMu06 ,
  IXNMuseModelMs03 }

7.74.1 Detailed Description

The model identifier of the headbands. The model is laser printed on the inside of the left pod of the headband as either "MU-01", "MU-02", "MU-03" or "MU-06". MuseS 2019 (MU_04) MuseS 2021 (MU_05) and MuseS 2025 (MS_03) models are marked on the underside as "MS-01", "MS-02" or "MS-03".

7.74.2 Enumeration Type Documentation

7.74.2.1 IXNMuseModel

enum IXNMuseModel

Enumerator

IXNMuseModelMu01	First model of Muse, Muse 2014
IXNMuseModelMu02	Muse 2016 with Bluetooth Low Energy support.
IXNMuseModelMu03	Muse2 2018 adding PPG sensor, Accelerometer and Gyroscope.
IXNMuseModelMu04	MuseS 2019 softband (MS-01).
IXNMuseModelMu05	MuseS 2021 softband refresh (MS-02).
IXNMuseModelMu06	Muse2 2024 with USB-C connector.
IXNMuseModelMs03	MuseS 2025 softband with USB-C, Bluetooth 5.3, improved EEG and Optics (MS-03).

7.75 IXNMuseModel.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from museinfo.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00014 typedef NS_ENUM(NSInteger, IXNMuseModel)
00015 {
00017     IXNMuseModelMu01,
00019     IXNMuseModelMu02,
00021     IXNMuseModelMu03,
00023     IXNMuseModelMu04,
00025     IXNMuseModelMu05,
00027     IXNMuseModelMu06,
00029     IXNMuseModelMs03,
00030 };
```

7.76 IXNMusePreset.h File Reference

Enumerations

- enum IXNMusePreset {
  IXNMusePresetPreset10 ,
  IXNMusePresetPreset12 ,
  IXNMusePresetPreset14 ,
  IXNMusePresetPreset20 ,
  IXNMusePresetPreset21 ,
  IXNMusePresetPreset22 ,
  IXNMusePresetPreset23 ,
  IXNMusePresetPresetAb ,
  IXNMusePresetPresetAd ,
  IXNMusePresetPreset31 ,
  IXNMusePresetPreset32 ,
  IXNMusePresetPreset50 ,
  IXNMusePresetPreset51 ,
  IXNMusePresetPreset52 ,
  IXNMusePresetPreset53 ,
  IXNMusePresetPreset54 ,
  IXNMusePresetPreset55 ,
  IXNMusePresetPreset60 ,
  IXNMusePresetPreset61 ,
  IXNMusePresetPreset63 ,

IXNMusePresetPreset1021 ,
IXNMusePresetPreset1022 ,
IXNMusePresetPreset1023 ,
IXNMusePresetPreset1024 ,
IXNMusePresetPreset1025 ,
IXNMusePresetPreset1026 ,
IXNMusePresetPreset1027 ,
IXNMusePresetPreset1028 ,
IXNMusePresetPreset1029 ,
IXNMusePresetPreset102A ,
IXNMusePresetPreset1031 ,
IXNMusePresetPreset1032 ,
IXNMusePresetPreset1033 ,
IXNMusePresetPreset1034 ,
IXNMusePresetPreset1035 ,
IXNMusePresetPreset1036 ,
IXNMusePresetPreset1041 ,
IXNMusePresetPreset1042 ,
IXNMusePresetPreset1043 ,
IXNMusePresetPreset1044 ,
IXNMusePresetPreset1045 ,
IXNMusePresetPreset1046 }

7.76.1 Detailed Description

Defines all possible Muse presets.

Presets are specific to the model of headband. For example, setting a preset that is intended for a Muse 2014 (MU_01) headband on a Muse 2016 (MU_02) headband will not work. Attempting to set an incorrect preset on a headband will result in a warning in the log and the incorrect preset will be ignored. To avoid this, you should first connect to the headband without setting a preset and check the IXNMuseConfiguration to get the model of the headband and then set the appropriate preset. You can also obtain the current preset of the headband from the IXNMuseConfiguration

If you change the preset while the headband is connected, the headband will disconnect. If the preset is valid for that headband model, then the headband will automatically reconnect. If the preset is invalid for that headband model, then the headband will remain disconnected.

See also

IXNMuseConfiguration

7.76.2 Enumeration Type Documentation

7.76.2.1 IXNMusePreset

enum IXNMusePreset

Enumerator

IXNMusePresetPreset10	4 CH EEG, 10 bit @ 220 Hz, compression ON, no accelerometer, no battery data, no error data, no DRL/REF data. Availability: Muse 2014 (MU_01) only
IXNMusePresetPreset12	4 CH EEG, 10 bit @ 220 Hz, compression ON, 50 Hz accelerometer data, 0.1 Hz battery data, no error data, no DRL/REF data Availability: Muse 2014 (MU_01) only
IXNMusePresetPreset14	4 CH EEG, 10 bit @ 220 Hz, compression ON, 50 Hz accelerometer data, 0.1 Hz battery data, real-time error data, 10 bit @ 10 Hz DRL/REF data This is the default for Muse 2014 (MU_01). Availability: Muse 2014 (MU_01) only
IXNMusePresetPreset20	5 CH EEG, 12 bit @ 256 Hz, 52 Hz accelerometer/gyro, 0.1 Hz battery, 32 Hz DRL/REF. Availability: Muse 2016 (MU_02), Muse2 2018 (MU_03), MuseS 2019 (MU_04), MuseS 2021 (MU_05), Muse2 2024 (MU_06)
IXNMusePresetPreset21	4 CH EEG, 12 bit @ 256 Hz, 52 Hz accelerometer/gyro, 0.1 Hz battery, 32 Hz DRL/REF. This is the default for Muse 2016 (MU_02), Muse2 2018 (MU_03), MuseS 2019 (MU_04), MuseS 2021 (MU_05), Muse2 2024 (MU_06). Availability: Muse 2016 (MU_02), Muse2 2018 (MU_03), MuseS 2019 (MU_04), MuseS 2021 (MU_05), Muse2 2024 (MU_06)
IXNMusePresetPreset22	4 CH EEG, 12 bit @ 256 Hz, 0.1 Hz battery, 32 Hz DRL/REF. Availability: Muse 2016 (MU_02), Muse2 2018 (MU_03), MuseS 2019 (MU_04), MuseS 2021 (MU_05), Muse2 2024 (MU_06)
IXNMusePresetPreset23	5 CH EEG, 12 bit @ 256 Hz, 0.1 Hz battery, 32 Hz DRL/REF. Availability: Muse2 2018 (MU_03), MuseS 2019 (MU_04), MuseS 2021 (MU_05), Muse2 2024 (MU_06)
IXNMusePresetPresetAb	6 CH EEG, 16 bit @ 500 Hz, 50 Hz accelerometer, 0.1 Hz battery, compression OFF, notch filter OFF, no error data, no DRL/REF data Research preset: Only the following data packets are generated with this preset: IXNMuseDataPacketTypeEeg , IXNMuseDataPacketTypeAccelerometer , IXNMuseDataPacketTypeBattery Artifacts are not generated with this preset. Availability: Muse 2014 (MU_01) only

Enumerator

IXNMusePresetPresetAd	4 CH EEG, 16 bit @ 500 Hz, 50 Hz accelerometer, 0.1 Hz battery, compression OFF, notch filter OFF, no error data, no DRL/REF data Research preset: Only the following data packets are generated with this preset: IXNMuseDataPacketTypeEeg , IXNMuseDataPacketTypeAccelerometer , IXNMuseDataPacketTypeBattery Artifacts are not generated with this preset. Availability: Muse 2014 (MU_01) only
IXNMusePresetPreset31	4 CH EEG, 12 bit @ 256 Hz, 0.1 Hz battery, 32 Hz DRL/REF, 52 Hz Acc, Gyro and Magnetometer, 0.1 Hz for UV and Pressure. Availability: glasses only
IXNMusePresetPreset32	No EEG data, only other sensors. 32 Hz DRL/REF, 52 Hz Acc, Gyro and Magnetometer, 0.1 Hz for battery, UV and Pressure. Availability: glasses only
IXNMusePresetPreset50	5 CH EEG, 12 bit @ 256 Hz, 52 Hz accelerometer/gyro, 0.1 Hz battery, 32 Hz DRL/REF, PPG @ 64 Hz Availability: Muse2 2018 (MU_03), MuseS 2019 (MU_04), MuseS 2021 (MU_05), Muse2 2024 (MU_06)
IXNMusePresetPreset51	4 CH EEG, 12 bit @ 256 Hz, 52 Hz accelerometer/gyro, 0.1 Hz battery, 32 Hz DRL/REF, PPG @ 64 Hz Availability: Muse2 2018 (MU_03), MuseS 2019 (MU_04), MuseS 2021 (MU_05), Muse2 2024 (MU_06)
IXNMusePresetPreset52	4 CH EEG, 12 bit @ 256 Hz, 0.1 Hz battery, 32 Hz DRL/REF, PPG @ 64 Hz Availability: Muse2 2018 (MU_03), MuseS 2019 (MU_04), MuseS 2021 (MU_05), Muse2 2024 (MU_06)
IXNMusePresetPreset53	6 CH EEG, 12 bit @ 256 Hz, 52 Hz accelerometer/gyro, 0.1 Hz battery, 32 Hz DRL/REF, PPG @ 64 Hz Availability: MuseS 2019 (MU_04), MuseS 2021 (MU_05), Muse2 2024 (MU_06)
IXNMusePresetPreset54	6 CH EEG, 12 bit @ 128 Hz, 52 Hz accelerometer/gyro, 0.1 Hz battery, 16 Hz DRL/REF Availability: MuseS 2021 (MU_05)
IXNMusePresetPreset55	4 CH EEG, 12 bit @ 128 Hz, 0.1 Hz battery, 16 Hz DRL/REF, PPG @ 64 Hz Availability: MuseS 2021 (MU_05)
IXNMusePresetPreset60	5 CH EEG, 12 bit @ 256 Hz, 52 Hz accelerometer/gyro, 0.1 Hz battery, 32 Hz DRL/REF, PPG @ 64 Hz, THERMISTOR @ 16 Hz Availability: MuseS 2019 (MU_04), MuseS 2021 (MU_05)

Enumerator

IXNMusePresetPreset61	4 CH EEG, 12 bit @ 256 Hz, 52 Hz accelerometer/gyro, 0.1 Hz battery, 32 Hz DRL/REF, PPG @ 64 Hz, THERMISTOR @ 16 Hz Availability: MuseS 2019 (MU_04), MuseS 2021 (MU_05)
IXNMusePresetPreset63	6 CH EEG, 12 bit @ 256 Hz, 52 Hz accelerometer/gyro, 0.1 Hz battery, 32 Hz DRL/REF, PPG @ 64 Hz, THERMISTOR @ 16 Hz Availability: MuseS 2019 (MU_04), MuseS 2021 (MU_05)
IXNMusePresetPreset1021	4 CH EEG, 14 bit @ 256 Hz, 52 Hz accelerometer/gyro, 1 Hz battery, 32 Hz DRL/REF Availability: MuseS 2025 (MS_03)
IXNMusePresetPreset1022	8 CH EEG, 14 bit @ 256 Hz, 1 Hz battery, 32 Hz DRL/REF Availability: MuseS 2025 (MS_03)
IXNMusePresetPreset1023	Battery only @ 50 Hz Availability: MuseS 2025 (MS_03)
IXNMusePresetPreset1024	52 Hz accelerometer/gyro Availability: MuseS 2025 (MS_03)
IXNMusePresetPreset1025	16 CH Optics @ 64 Hz, low power Availability: MuseS 2025 (MS_03)
IXNMusePresetPreset1026	16 CH Optics @ 64 Hz, high power Availability: MuseS 2025 (MS_03)
IXNMusePresetPreset1027	8 CH Optics @ 64 Hz, low power Availability: MuseS 2025 (MS_03)
IXNMusePresetPreset1028	8 CH Optics @ 64 Hz, high power Availability: MuseS 2025 (MS_03)
IXNMusePresetPreset1029	4 CH Optics @ 64 Hz, low power Availability: MuseS 2025 (MS_03)
IXNMusePresetPreset102A	4 CH Optics @ 64 Hz, high power Availability: MuseS 2025 (MS_03)
IXNMusePresetPreset1031	4 CH EEG, 14 bit @ 256 Hz, 52 Hz accelerometer/gyro, 1 Hz battery, 32 Hz DRL/REF, 16 CH Optics @ 64 Hz, low power Availability: MuseS 2025 (MS_03)
IXNMusePresetPreset1032	4 CH EEG, 14 bit @ 256 Hz, 52 Hz accelerometer/gyro, 1 Hz battery, 32 Hz DRL/REF, 16 CH Optics @ 64 Hz, high power Availability: MuseS 2025 (MS_03)
IXNMusePresetPreset1033	4 CH EEG, 14 bit @ 256 Hz, 52 Hz accelerometer/gyro, 1 Hz battery, 32 Hz DRL/REF, 8 CH Optics @ 64 Hz, low power Availability: MuseS 2025 (MS_03)

Enumerator

IXNMusePresetPreset1034	4 CH EEG, 14 bit @ 256 Hz, 52 Hz accelerometer/gyro, 1 Hz battery, 32 Hz DRL/REF, 8 CH Optics @ 64 Hz, high power Availability: MuseS 2025 (MS_03)
IXNMusePresetPreset1035	4 CH EEG, 14 bit @ 256 Hz, 52 Hz accelerometer/gyro, 1 Hz battery, 32 Hz DRL/REF, 4 CH Optics @ 64 Hz, low power Availability: MuseS 2025 (MS_03)
IXNMusePresetPreset1036	4 CH EEG, 14 bit @ 256 Hz, 52 Hz accelerometer/gyro, 1 Hz battery, 32 Hz DRL/REF, 4 CH Optics @ 64 Hz, high power Availability: MuseS 2025 (MS_03)
IXNMusePresetPreset1041	8 CH EEG, 14 bit @ 256 Hz, 52 Hz accelerometer/gyro, 1 Hz battery, 32 Hz DRL/REF, 16 CH Optics @ 64 Hz, low power Availability: MuseS 2025 (MS_03)
IXNMusePresetPreset1042	8 CH EEG, 14 bit @ 256 Hz, 52 Hz accelerometer/gyro, 1 Hz battery, 32 Hz DRL/REF, 16 CH Optics @ 64 Hz, high power Availability: MuseS 2025 (MS_03)
IXNMusePresetPreset1043	8 CH EEG, 14 bit @ 256 Hz, 52 Hz accelerometer/gyro, 1 Hz battery, 32 Hz DRL/REF, 8 CH Optics @ 64 Hz, low power Availability: MuseS 2025 (MS_03)
IXNMusePresetPreset1044	8 CH EEG, 14 bit @ 256 Hz, 52 Hz accelerometer/gyro, 1 Hz battery, 32 Hz DRL/REF, 8 CH Optics @ 64 Hz, high power Availability: MuseS 2025 (MS_03)
IXNMusePresetPreset1045	8 CH EEG, 14 bit @ 256 Hz, 52 Hz accelerometer/gyro, 1 Hz battery, 32 Hz DRL/REF, 4 CH Optics @ 64 Hz, low power Availability: MuseS 2025 (MS_03)
IXNMusePresetPreset1046	8 CH EEG, 14 bit @ 256 Hz, 52 Hz accelerometer/gyro, 1 Hz battery, 32 Hz DRL/REF, 4 CH Optics @ 64 Hz, high power Availability: MuseS 2025 (MS_03)

7.77 IXNMusePreset.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from museinfo.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00029 typedef NS_ENUM(NSInteger, IXNMusePreset)
00030 {
00038     IXNMusePresetPreset10,
00046     IXNMusePresetPreset12,
00057     IXNMusePresetPreset14,
00065     IXNMusePresetPreset20,
00076     IXNMusePresetPreset21,
00084     IXNMusePresetPreset22,
00092     IXNMusePresetPreset23,
```

```
00108     IXNMusePresetPresetAb,
00124     IXNMusePresetPresetAd,
00133     IXNMusePresetPreset31,
00142     IXNMusePresetPreset32,
00150     IXNMusePresetPreset50,
00158     IXNMusePresetPreset51,
00166     IXNMusePresetPreset52,
00174     IXNMusePresetPreset53,
00182     IXNMusePresetPreset54,
00190     IXNMusePresetPreset55,
00198     IXNMusePresetPreset60,
00206     IXNMusePresetPreset61,
00214     IXNMusePresetPreset63,
00222     IXNMusePresetPreset1021,
00230     IXNMusePresetPreset1022,
00237     IXNMusePresetPreset1023,
00244     IXNMusePresetPreset1024,
00251     IXNMusePresetPreset1025,
00258     IXNMusePresetPreset1026,
00265     IXNMusePresetPreset1027,
00272     IXNMusePresetPreset1028,
00279     IXNMusePresetPreset1029,
00286     IXNMusePresetPreset102A,
00294     IXNMusePresetPreset1031,
00302     IXNMusePresetPreset1032,
00310     IXNMusePresetPreset1033,
00318     IXNMusePresetPreset1034,
00326     IXNMusePresetPreset1035,
00334     IXNMusePresetPreset1036,
00342     IXNMusePresetPreset1041,
00350     IXNMusePresetPreset1042,
00358     IXNMusePresetPreset1043,
00366     IXNMusePresetPreset1044,
00374     IXNMusePresetPreset1045,
00382     IXNMusePresetPreset1046,
00383 };
```

7.78 IXNMuseVersion.h File Reference

Classes

- class IXNMuseVersion

7.79 IXNMuseVersion.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from museinfo.djinni
00003
00004 #import <Foundation/Foundation.h>
00005 @class IXNMuseVersion;
00006
00007
00015 @interface IXNMuseVersion : NSObject
00016
00018 + (nullable IXNMuseVersion *)makeDefaultVersion;
00019
00021 + (nullable IXNMuseVersion *)makeVersion:(nonnull NSString *)json;
00022
00029 - (nonnull NSString *)getRunningState;
00030
00035 - (nonnull NSString *)getHardwareVersion;
00036
00042 - (nonnull NSString *)getBspVersion;
00043
00048 - (nonnull NSString *)getFirmwareVersion;
00049
00054 - (nonnull NSString *)getBootloaderVersion;
00055
00061 - (nonnull NSString *)getFirmwareBuildNumber;
00062
00067 - (nonnull NSString *)getFirmwareType;
00068
00073 - (int32_t)getProtocolVersion;
00074
00079 - (nonnull NSString *)getBleFirmwareVersion;
00080
00081 @end
```

7.80 IXNNotchFrequency.h File Reference

Enumerations

- enum IXNNotchFrequency {
  IXNNotchFrequencyNotchNone ,
  IXNNotchFrequencyNotch50hz ,
  IXNNotchFrequencyNotch60hz }

7.80.1 Detailed Description

Notch Frequencies refer to the possible power line frequencies in a geographic area that may create noise in raw EEG data.

Muse 2014 (MU_01) headbands are equipped with a hardware filter to remove this noise in certain preset configurations.

For Muse 2016 (MU_02) or later headbands or Muse 2014 (MU_01) headbands using a research preset there is no hardware filtering.

See also

- IXNMusePreset
- http://en.wikipedia.org/wiki/Utility_frequency

7.80.2 Enumeration Type Documentation

7.80.2.1 IXNNotchFrequency

enum IXNNotchFrequency

Enumerator

IXNNotchFrequencyNotchNone	The notch filter is not available on some hardware versions.
IXNNotchFrequencyNotch50hz	50 Hz frequency is used in most parts of the world: Europe, Russia, Africa
IXNNotchFrequencyNotch60hz	60 Hz frequency is used in North America and Asia. This is the default setting.

7.81 IXNNotchFrequency.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from museinfo.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00022 typedef NS_ENUM(NSInteger, IXNNotchFrequency)
00023 {
00025     IXNNotchFrequencyNotchNone,
00030     IXNNotchFrequencyNotch50hz,
00035     IXNNotchFrequencyNotch60hz,
00036 };
```

7.82 IXNOptics.h File Reference

Enumerations

- enum IXNOptics {
  IXNOpticsOPTICS1 ,
  IXNOpticsOPTICS2 ,
  IXNOpticsOPTICS3 ,
  IXNOpticsOPTICS4 ,
  IXNOpticsOPTICS5 ,
  IXNOpticsOPTICS6 ,
  IXNOpticsOPTICS7 ,
  IXNOpticsOPTICS8 ,
  IXNOpticsOPTICS9 ,
  IXNOpticsOPTICS10 ,
  IXNOpticsOPTICS11 ,
  IXNOpticsOPTICS12 ,
  IXNOpticsOPTICS13 ,
  IXNOpticsOPTICS14 ,
  IXNOpticsOPTICS15 ,
  IXNOpticsOPTICS16 }

7.82.1 Detailed Description

Represents the data mapping in an Optics packet.

Functional near-infrared spectroscopy (fNIRS) is a non-invasive imaging technique that measures brain activity by detecting changes in blood oxygenation.

The light absorbtion of the blood varies by wavelength. The 750nm wavelength is sensitive to deoxygenated hemoglobin (HbR), while the 850nm wavelength is sensitive to oxygenated hemoglobin (HbO). By analyzing the absorption of these wavelengths, we can determine changes in blood oxygenation levels.

For more information on fNIRS, see the following Wikipedia article:
https://en.wikipedia.org/wiki/Functional_near-infrared_spectroscopy

The Optics data is also used to measure PPG.

Optics data values are given in microamps (uA).

See also

IXNMuseDataPacketTypeOptics

IXNMuseDataPacket::getOpticsChannelValue

7.82.2 Enumeration Type Documentation

7.82.2.1 IXNOptics

enum IXNOptics

Enumerator

IXNOpticsOPTICS1	730nm left outer (8 and 16 channel modes) or 730nm left inner (4 channel mode)
IXNOpticsOPTICS2	730nm right outer (8 and 16 channel modes) or 730nm right inner (4 channel mode)
IXNOpticsOPTICS3	850nm left outer (8 and 16 channel modes) or 850nm left inner (4 channel mode)
IXNOpticsOPTICS4	850nm right outer (8 and 16 channel modes) or 850nm right inner (4 channel mode)
IXNOpticsOPTICS5	730nm left inner (8 and 16 channel modes)
IXNOpticsOPTICS6	730nm right inner (8 and 16 channel modes)
IXNOpticsOPTICS7	850nm left inner (8 and 16 channel modes)
IXNOpticsOPTICS8	850nm right inner (8 and 16 channel modes)
IXNOpticsOPTICS9	Red left outer (16 channel mode)
IXNOpticsOPTICS10	Red right outer (16 channel mode)
IXNOpticsOPTICS11	Ambient left outer (16 channel mode)
IXNOpticsOPTICS12	Ambient right outer (16 channel mode)
IXNOpticsOPTICS13	Red left inner (16 channel mode)
IXNOpticsOPTICS14	Red right inner (16 channel mode)
IXNOpticsOPTICS15	Ambient left inner (16 channel mode)
IXNOpticsOPTICS16	Ambient right inner (16 channel mode)

7.83 IXNOptics.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from packets.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00030 typedef NS_ENUM(NSInteger, IXNOptics)
00031 {
00033     IXNOpticsOPTICS1,
00035     IXNOpticsOPTICS2,
00037     IXNOpticsOPTICS3,
00039     IXNOpticsOPTICS4,
00041     IXNOpticsOPTICS5,
00043     IXNOpticsOPTICS6,
00045     IXNOpticsOPTICS7,
00047     IXNOpticsOPTICS8,
00049     IXNOpticsOPTICS9,
00051     IXNOpticsOPTICS10,
00053     IXNOpticsOPTICS11,
00055     IXNOpticsOPTICS12,
00057     IXNOpticsOPTICS13,
00059     IXNOpticsOPTICS14,
00061     IXNOpticsOPTICS15,
00063     IXNOpticsOPTICS16,
00064 };
```

7.84 IXNPpg.h File Reference

Enumerations

- enum IXNPpg {
  IXNPpgAMBIENT ,

IXNPpgIR ,
IXNPpgRED }

7.84.1 Detailed Description

Represents the data mapping in a PPG packet. Enum values for IR and RED correspond to the Infrared and Red values. The AMBIENT enum value is repurposed on models MuseS 2019 (MU_04) and MuseS 2021 (MU_05).

Muse2 2018 (MU_03) AMBIENT represents the Ambient value.
MuseS 2019 (MU_04) AMBIENT represents the Green value. There is no Ambient available and Red is not used.
MuseS 2021 (MU_05) AMBIENT represents the IR-H16 value. There is no Ambient available.

MuseS 2025 (MS_03) uses IXNOptics data for PPG.

Raw PPG values are given in arbitrary units.

See also

IXNMuseDataPacketTypePpg

IXNMuseDataPacket::getPpgChannelValue

7.84.2 Enumeration Type Documentation

7.84.2.1 IXNPpg

enum IXNPpg

Enumerator

IXNPpgAMBIENT	Ambient, Green or IR-H16.
IXNPpgIR	IR.
IXNPpgRED	Red.

7.85 IXNPpg.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from packets.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00025 typedef NS_ENUM(NSInteger, IXNPpg)
00026 {
00028     IXNPpgAMBIENT,
00030     IXNPpgIR,
00032     IXNPpgRED,
00033 };
```

7.86 IXNPressure.h File Reference

Enumerations

- enum IXNPressure {
  IXNPressureRaw ,
  IXNPressureAveraged }

7.86.1 Detailed Description

Represents the data mapping in a Pressure packet.

See also

IXNMuseDataPacketTypePressure

IXNMuseDataPacket::getPressureValue

7.86.2 Enumeration Type Documentation

7.86.2.1 IXNPressure

enum IXNPressure

Enumerator

IXNPressureRaw	The raw pressure value returned by the pressure sensor in mBar
IXNPressureAveraged	The averaged pressure value in mBar based on the last 10 readings. This provides a smoother curve for the pressure values.

7.87 IXNPressure.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from packets.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00014 typedef NS_ENUM(NSInteger, IXNPressure)
00015 {
00017     IXNPressureRaw,
00022     IXNPressureAveraged,
00023 };
```

7.88 IXNReaderListener.h File Reference

Classes

- protocol <IXNReaderListener>

7.89 IXNReaderListener.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from listeners.djinni
00003
00004 #import "Muse/api/IXNAnnotationData.h"
00005 #import "Muse/api/IXNComputingDeviceConfiguration.h"
00006 #import <Foundation/Foundation.h>
00007 @class IXNMuseConfiguration;
00008 @class IXNMuseVersion;
00009
00010
00015 @protocol IXNReaderListener
00016
00021 - (void)receiveAnnotation:(nonnull IXNAnnotationData *)annotation;
00022
00027 - (void)receiveVersion:(nullable IXNMuseVersion *)version;
00028
00033 - (void)receiveConfiguration:(nullable IXNMuseConfiguration *)configuration;
00034
00039 - (void)receiveComputingDeviceConfiguration:(nonnull IXNComputingDeviceConfiguration
      *)computingDeviceConfiguration;
00040
00041 @end
```

7.90 IXNReaderMuse.h File Reference

Classes

- class IXNReaderMuse

7.91 IXNReaderMuse.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from sdk_bridge.djinni
00003
00004 #import "Muse/api/IXNReaderMusePlaybackSettings.h"
00005 #import <Foundation/Foundation.h>
00006 @class IXNMuse;
00007 @protocol IXNReaderListener;
00008 @protocol IXNReaderPlaybackListener;
00009
00010
00028 @interface IXNReaderMuse : NSObject
00029
00042 - (void)run;
00043
00056 - (void)runInRealTimespan;
00057
00065 - (int64_t)currentTime;
00066
00085 - (void)playback;
00086
00088 - (void)stopPlayback;
00089
00096 - (void)setPlaybackSettings:(IXNReaderMusePlaybackSettings)settings;
00097
00104 - (IXNReaderMusePlaybackSettings)getPlaybackSettings;
00105
00112 - (void)setReaderListener:(nullable id<IXNReaderListener>)listener;
00113
00119 - (void)setPlaybackListener:(nullable id<IXNReaderPlaybackListener>)listener;
00120
00135 - (nullable IXNMuse *)asMuse;
00136
00137 @end
```

7.92 IXNReaderMuseBuilder.h File Reference

Classes

- class IXNReaderMuseBuilder

7.93 IXNReaderMuseBuilder.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from sdk_bridge.djinni
00003
00004 #import "Muse/api/IXNMuseDataPacketType.h"
00005 #import "Muse/api/IXNMuseModel.h"
00006 #import "Muse/api/IXNReaderMusePlaybackSettings.h"
00007 #import <Foundation/Foundation.h>
00008 @class IXNMuseFileReader;
00009 @class IXNReaderMuse;
00010 @class IXNReaderMuseBuilder;
00011 @protocol IXNEventLoop;
00012
00013
00042 @interface IXNReaderMuseBuilder : NSObject
00043
00049 + (nullable IXNReaderMuseBuilder *)get;
00050
00065 - (nullable IXNReaderMuseBuilder *)withPacketTypes:(nonnull NSSet<NSNumber *> *)types;
00066
00080 - (nullable IXNReaderMuseBuilder *)skipPacketTypes:(nonnull NSSet<NSNumber *> *)types;
00081
00089 - (nullable IXNReaderMuseBuilder *)withModel:(IXNMuseModel)model;
00090
00101 - (nullable IXNReaderMuseBuilder *)withPlaybackSettings:(IXNReaderMusePlaybackSettings)settings;
00102
00112 - (nullable IXNReaderMuseBuilder *)withEventLoop:(nullable id<IXNEventLoop>)loop;
00113
00120 - (nullable IXNReaderMuse *)build:(nullable IXNMuseFileReader *)reader;
00121
00142 - (nullable IXNReaderMuse *)buildWithAsync:(nullable IXNMuseFileReader *)reader
00143                                 asyncLoop:(nullable id<IXNEventLoop>)asyncLoop;
00144
00145 @end
```

7.94 IXNReaderMusePlaybackSettings.h File Reference

Enumerations

- enum IXNReaderMusePlaybackSettings {
  IXNReaderMusePlaybackSettingsAsFastAsPossibleWithSavedTimestamp ,
  IXNReaderMusePlaybackSettingsSimulatedWithSavedTimestamp ,
  IXNReaderMusePlaybackSettingsSimulatedWithSystemClockTimestamp }

7.94.1 Detailed Description

Configures the playback speed and timestamps of a IXNReaderMuse

See also

IXNReaderMuse

IXNReaderMuseBuilder

7.94.2 Enumeration Type Documentation

7.94.2.1 IXNReaderMusePlaybackSettings

enum IXNReaderMusePlaybackSettings

Enumerator

IXNReaderMusePlaybackSettingsAsFastAs↩ PossibleWithSavedTimestamp	The IXNReaderMuse should playback the data as fast as it can be read from the file. There will be no simulated time between packets and the timestamps of the packets created during playback will match the timestamps that are saved in the file.
IXNReaderMusePlaybackSettingsSimulated↩ WithSavedTimestamp	The IXNReaderMuse should playback the data simulating the time between packets. For example, if the second packet in the file has a timestamp that is 20ms later than the timestamp of the first packet, then LibMuse will wait for 20ms after the first packet is sent before sending the second packet. The timestamps of the packets created during playback will match the timestamps that are saved in the file. Using this setting requires that the IXNReaderMuse be constructed with an IXNEventLoop that will be used to simulate the time between packets. If this setting is used, but the IXNReaderMuse does not have an IXNEventLoop then playback will not read the file and a warning will be logged.
IXNReaderMusePlaybackSettingsSimulated↩ WithSystemClockTimestamp	The IXNReaderMuse should playback the data simulating the time between packets. For example, if the second packet in the file has a timestamp that is 20ms later than the timestamp of the first packet, then LibMuse will wait for 20ms after the first packet is sent before sending the second packet. The timestamps of the packets created during playback will use the current times of the system clock rather than the timestamp of the packet saved in the file. Using this setting requires that the IXNReaderMuse be constructed with an IXNEventLoop that will be used to simulate the time between packets. If this setting is used, but the IXNReaderMuse does not have an IXNEventLoop then playback will not read the file and a warning will be logged.

7.95 IXNReaderMusePlaybackSettings.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from sdk_bridge.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00015 typedef NS_ENUM(NSInteger, IXNReaderMusePlaybackSettings)
00016 {
00024     IXNReaderMusePlaybackSettingsAsFastAsPossibleWithSavedTimestamp,
00042     IXNReaderMusePlaybackSettingsSimulatedWithSavedTimestamp,
00061     IXNReaderMusePlaybackSettingsSimulatedWithSystemClockTimestamp,
00062 };
```

7.96 IXNReaderPlaybackListener.h File Reference

Classes

- protocol <IXNReaderPlaybackListener>

7.97 IXNReaderPlaybackListener.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from listeners.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00006
00011 @protocol IXNReaderPlaybackListener
00012
00014 - (void)receivePlaybackDone;
00015
00017 - (void)receivePlaybackInterrupted;
00018
00019 @end
```

7.98 IXNResult.h File Reference

Classes

- class IXNResult

7.99 IXNResult.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from validation.djinni
00003
00004 #import "Muse/api/IXNResultLevel.h"
00005 #import <Foundation/Foundation.h>
00006
00012 @interface IXNResult : NSObject
00013 - (nonnull instancetype)initWithLevel:(IXNResultLevel)level
00014                                  type:(nonnull NSString *)type
00015                                  code:(int32_t)code
00016                                  info:(nonnull NSString *)info;
00017 + (nonnull instancetype)resultWithLevel:(IXNResultLevel)level
00018                                    type:(nonnull NSString *)type
00019                                    code:(int32_t)code
00020                                    info:(nonnull NSString *)info;
00021
00026 @property (nonatomic, readonly) IXNResultLevel level;
00027
00032 @property (nonatomic, readonly, nonnull) NSString * type;
00033
00038 @property (nonatomic, readonly) int32_t code;
00039
00044 @property (nonatomic, readonly, nonnull) NSString * info;
00045
00046 @end
```

7.100 IXNResultLevel.h File Reference

Enumerations

- enum IXNResultLevel {
  IXNResultLevelRNone ,
  IXNResultLevelRSuccess ,
  IXNResultLevelRInfo ,
  IXNResultLevelRWarn ,
  IXNResultLevelRError ,
  IXNResultLevelRDebug }

7.100.1 Detailed Description

Represents the level of a result

7.100.2 Enumeration Type Documentation

7.100.2.1 IXNResultLevel

enum IXNResultLevel

Enumerator

IXNResultLevelRNone	default
IXNResultLevelRSuccess	success
IXNResultLevelRInfo	info
IXNResultLevelRWarn	warn
IXNResultLevelRError	error
IXNResultLevelRDebug	debug

7.101 IXNResultLevel.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from validation.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00012 typedef NS_ENUM(NSInteger, IXNResultLevel)
00013 {
00015     IXNResultLevelRNone,
00017     IXNResultLevelRSuccess,
00019     IXNResultLevelRInfo,
00021     IXNResultLevelRWarn,
00023     IXNResultLevelRError,
00025     IXNResultLevelRDebug,
00026 };
```

7.102 IXNSeverity.h File Reference

Enumerations

- enum IXNSeverity {
  IXNSeveritySevVerbose ,
  IXNSeveritySevInfo ,
  IXNSeveritySevWarn ,
  IXNSeveritySevError ,
  IXNSeveritySevFatal ,
  IXNSeveritySevDebug ,
  IXNSeverityTotal }

7.102.1 Enumeration Type Documentation

7.102.1.1 IXNSeverity

enum IXNSeverity

The log message severity.

Enumerator

IXNSeveritySevVerbose	Verbose logs. These provide lots of details that are probably irrelevant except for tracing or debugging problems with the headband or library.
IXNSeveritySevInfo	Informational logs. These messages are sent when significant but expected events happen (e.g. a connection started or was completed successfully.)
IXNSeveritySevWarn	Warning logs. These messages indicate that something out of the ordinary but recoverable happened (e.g. a connection attempt failed but will be retried automatically.)
IXNSeveritySevError	Error logs. These messages indicate that something has gone wrong – for instance, a connection terminated unexpectedly or a corrupted packet was received.
IXNSeveritySevFatal	Fatal logs. These are sent when the library is in a completely unrecoverable state from which the only reasonable outcome is termination of the running process. The process will be aborted as soon as the log handler returns.
IXNSeveritySevDebug	Debug-only logs. These are only interesting to developers trying to track down problems in the library.
IXNSeverityTotal	Enum size sentinel; not an actual severity.

7.103 IXNSeverity.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from logging.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00007 typedef NS_ENUM(NSInteger, IXNSeverity)
00008 {
00013     IXNSeveritySevVerbose,
00018     IXNSeveritySevInfo,
00024     IXNSeveritySevWarn,
00030     IXNSeveritySevError,
00037     IXNSeveritySevFatal,
00042     IXNSeveritySevDebug,
00044     IXNSeverityTotal,
00045 };
```

7.104 IXNStringify.h File Reference

Classes

- class IXNStringify

7.105 IXNStringify.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from packets.djinni
00003
00004 #import "Muse/api/IXNConnectionState.h"
00005 #import "Muse/api/IXNMuseDataPacketType.h"
00006 #import <Foundation/Foundation.h>
00007 @class IXNStringify;
00008
00009
00018 @interface IXNStringify : NSObject
00019
00024 + (nullable IXNStringify *)instance;
00025
00035 - (nonnull NSString *)packetType:(IXNMuseDataPacketType)type;
00036
00042 - (nonnull NSString *)connectionState:(IXNConnectionState)state;
00043
00044 @end
```

7.106 IXNTimestampMode.h File Reference

Enumerations

- enum IXNTimestampMode {
  IXNTimestampModeLegacy ,
  IXNTimestampModeCurrent ,
  IXNTimestampModeExplicit }

7.106.1 Detailed Description

File writer timestamp mode.

7.106.2 Enumeration Type Documentation

7.106.2.1 IXNTimestampMode

enum IXNTimestampMode

Enumerator

IXNTimestampModeLegacy	Legacy mode. Use the current time for everything except data packets. Use the data packet's timestamp field for those.
IXNTimestampModeCurrent	Use the current time for timestamps.
IXNTimestampModeExplicit	Use set_timestamp for timestamps.

7.107 IXNTimestampMode.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from muse_file.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00012 typedef NS_ENUM(NSInteger, IXNTimestampMode)
00013 {
00020     IXNTimestampModeLegacy,
00022     IXNTimestampModeCurrent,
00024     IXNTimestampModeExplicit,
00025 };
```

7.108 IXNUltraViolet.h File Reference

Enumerations

- enum IXNUltraViolet {
  IXNUltraVioletUvIndex ,
  IXNUltraVioletUvA ,
  IXNUltraVioletUvB }

7.108.1 Detailed Description

Represents the data mapping in a UltraViolet data packet

The ultra violet sensor will measure UVA and UVB wavelength. The UV index is the average of UVA and UVB calculated index values from raw sensor data.

See also

IXNMuseDataPacketTypeUltraViolet

IXNMuseDataPacket::getUvValue

7.108.2 Enumeration Type Documentation

7.108.2.1 IXNUltraViolet

enum IXNUltraViolet

Enumerator

IXNUltraVioletUvIndex	
IXNUltraVioletUvA	
IXNUltraVioletUvB	

7.109 IXNUltraViolet.h

Go to the documentation of this file.

```
00001 // AUTOGENERATED FILE - DO NOT MODIFY!
00002 // This file generated by Djinni from packets.djinni
00003
00004 #import <Foundation/Foundation.h>
00005
00018 typedef NS_ENUM(NSInteger, IXNUltraViolet)
00019 {
00020     IXNUltraVioletUvIndex,
00021     IXNUltraVioletUvA,
00022     IXNUltraVioletUvB,
00023 };
```

7.110 IXNComputingDeviceConfigurationFactory.h File Reference

Classes

- class IXNComputingDeviceConfigurationFactory

7.111 IXNComputingDeviceConfigurationFactory.h

Go to the documentation of this file.

```
00001 // Copyright 2017 InteraXon, Inc.
00002
00003 #import <Foundation/Foundation.h>
00004 #import "Muse/api/IXNComputingDeviceConfiguration.h"
00005
00010 @interface IXNComputingDeviceConfigurationFactory : NSObject
00011
00017 + (nonnull IXNComputingDeviceConfigurationFactory *)getInstance;
00018
00022 @property (nonatomic, readwrite, nonnull) IXNComputingDeviceConfiguration * computingDeviceConfiguration;
00023
00024 @end
```

7.112 IXNMuseFileFactory.h File Reference

Classes

- class IXNMuseFileFactory

7.113 IXNMuseFileFactory.h

Go to the documentation of this file.

```
00001 // Copyright 2015 InteraXon, Inc.
00002
00003 #import <Foundation/Foundation.h>
00004 #import "Muse/api/IXNMuseFile.h"
00005 #import "Muse/api/IXNMuseFileReader.h"
00006 #import "Muse/api/IXNMuseFileWriter.h"
00007
00011 @interface IXNMuseFileFactory : NSObject
00012
00024 + (IXNMuseFileWriter *)museFileWriterWithPathString:(NSString*)filePath;
00025
00032 + (IXNMuseFileReader *)museFileReaderWithPathString:(NSString*)filePath;
00033
00039 + (id<IXNMuseFile>)museFileWithPathString:(NSString*)filePath;
00040
00041 @end
```

7.114 IXNMuseManagerIos.h File Reference

Classes

- class IXNMuseManagerIos

7.115 IXNMuseManagerIos.h

Go to the documentation of this file.

```
00001 // Copyright 2015 InteraXon, Inc.
00002
00003 #import <ExternalAccessory/ExternalAccessory.h>
00004 #import <Foundation/Foundation.h>
00005 #import "Muse/api/IXNMuseManager.h"
00006
00010 @interface IXNMuseManagerIos : NSObject<IXNMuseManager>
00011
00015 + (IXNMuseManagerIos *)sharedManager;
00016
00030 - (void)showMusePickerWithCompletion:
00031     (EABluetoothAccessoryPickerCompletion)completion;
00032
00038 @property (nonatomic, readwrite) id<IXNMuseListener> museListener;
00039
00040 @end
```

7.116 IXNRunEventLoop.h File Reference

Classes

- class IXNRunEventLoop

7.117 IXNRunEventLoop.h

Go to the documentation of this file.

```
00001 // Copyright 2016 Interaxon, Inc.
00002
00003 #import <Foundation/Foundation.h>
00004
00005 #import "Muse/api/IXNEventLoop.h"
00006
00010 @interface IXNRunEventLoop : NSObject <IXNEventLoop>
00016 - (instancetype)initWithRunLoop:(NSRunLoop *)runLoop;
00017 @end
```